

Specification-Driven Development con Claude Code

Especificar antes de generar: el método para construir software con IA que sobreviva al mantenimiento.

Libro técnico del curso · 6 módulos · 30 capítulos · ~66.500 palabras
Edición de trabajo · 29 de mayo de 2026

BORRADOR — contenido sin pasar /revisar

Contenido

M1 — Por qué SDD existe ahora

- M1.1 · Vibe coding ha dejado de escalar
- M1.2 · Qué es Specification-Driven Development
- M1.3 · El "spec" como objeto de primera clase
- M1.4 · Mini-demo — el mismo problema, dos formas

M2 — Anatomía de una spec útil

- M2.1 · Tres niveles de spec — intent, behavior, contract
- M2.2 · Formatos — markdown plano, OpenAPI, JSON Schema
- M2.3 · Plantillas reutilizables
- M2.4 · Revisión de specs — los cuatro huecos
- M2.5 · Práctica guiada — escribir y revisar una spec de feature real

M3 — Claude Code como motor de SDD

- M3.1 · Modelo mental — Claude Code no es un chat
- M3.2 · Plan Mode y /plan — el puente entre spec y código
- M3.3 · Skills — abstraer specs reutilizables
- M3.4 · Subagentes — roles especializados
- M3.5 · Sesión, contexto y memoria entre conversaciones
- M3.6 · Práctica integrada — del intent a la primera implementación

M4 — Hooks y validación automática

- M4.1 · Por qué hooks importan en SDD
- M4.2 · Hooks de validación contra spec
- M4.3 · Hooks de protección — destructivas, secretos, scope
- M4.4 · Hooks como skill de equipo
- M4.5 · Práctica — instalar 3 hooks defensivos en un proyecto real

M5 — MCP · spec → herramientas tipadas

- M5.1 · Qué es MCP y qué no es
- M5.2 · Anatomía de un servidor MCP
- M5.3 · MCPs útiles para SDD en .NET
- M5.4 · Diseñar un MCP propio desde una spec
- M5.5 · Práctica — conectar un MCP a la práctica del M3

M6 — Caso end-to-end

- M6.1 · El proyecto que vamos a construir
- M6.2 · Sesión 1 — especificar las cuatro operaciones

M6.3 · Sesión 2 — implementar guiado por SDD

M6.4 · Sesión 3 — el cambio de requisito

M6.5 · Cierre — tu próximo proyecto

PARTE 1

Por qué SDD existe ahora

M1.1 · Vibe coding ha dejado de escalar

Bloque 0 · Con qué llegas

Si en la sala hay devs con un par de años de oficio, casi todos llegan a este curso con la misma sensación rara: la productividad ha subido como nunca, y al mismo tiempo el código del que ya no se acuerdan nadie es más del que se acordaban antes. Conviven las dos cosas. Sacas en una tarde lo que antes te llevaba una semana, y dos semanas después tienes que tocar ese código y no sabes por dónde entrar.

Hay tres posturas tipo, y en cualquier grupo de developers están las tres a la vez.

Está quien ha decidido que Claude Code (o Cursor, o Copilot, o el que sea) es el mejor invento desde Stack Overflow, y dispara prompts a velocidad de teclado. Cuando algo no funciona, repite el prompt con otra palabra. Avanza rápido y se le acumula código que nadie ha leído entero, ni siquiera él. Está quien lo ha probado dos tardes y lo ha guardado en el cajón de "interesante pero juguete", convencido de que para producción de verdad sigue siendo más rápido escribir uno mismo. Y está el escéptico templado: lo usa, lo respeta, pero ha visto suficientes bugs raros en código generado como para no fiarse de nada que no haya leído.

Ninguna de las tres está del todo equivocada. Y ninguna basta para construir software que vaya a tener mantenimiento dentro de dos años.

Lo que vamos a hacer en este primer submódulo es nombrar la cosa. Qué llamamos *vibe coding* — el modo de trabajo por defecto cuando se trabaja con IA sin método —, por qué funcionaba bien hasta hace poco, y por qué ha dejado de funcionar justo cuando empieza a importar de verdad. No para vender que SDD es la solución a todo, sino para que cuando lleguen el resto de módulos, sepas exactamente qué problema están resolviendo.

Bloque 1 · Qué era el vibe coding cuando funcionaba

El término *vibe coding* lo puso en circulación Andrej Karpathy a principios de 2025, y se popularizó porque describía algo que muchos ya estábamos haciendo sin ponerle nombre: programar dejándote llevar por el flow con la IA. Le sueltas una idea en lenguaje natural, te devuelve código, lo pegas, lo pruebas, lo ajustas con otro prompt, sigues. No miras los detalles. No revisas línea por línea. Si compila y hace lo que parece tener que hacer, p'alante.

Y aquí va lo importante: durante un tiempo, esto funcionó razonablemente bien.

Funcionó porque las generaciones eran pequeñas. Le pedías una función de utilidad, un script, una clase simple, una expresión regular complicada. Veinte, treinta líneas. A esa escala, el cerebro humano todavía puede leer el resultado de un vistazo, ver si tiene sentido, ajustarlo si no. La IA hacía el primer borrador, tú hacías de revisor humano sin esfuerzo. Y como el problema era pequeño, los errores también lo eran.

En esa escala, escribir una spec habría sido absurdo. Para una función de tres parámetros con un comportamiento obvio, el "documento de especificación" sería más largo que la función misma. Eso era cierto entonces y sigue siéndolo ahora. Si vas a generar un helper de fecha, no escribas una spec. Pide la función.

El problema no era hacer vibe coding. El problema es que el contexto cambió y nadie cambió la manera de trabajar.

Bloque 2 · Por qué deja de funcionar

Hoy le pides a Claude Code que te construya una API completa con su capa de datos, sus tests, su validación y sus migraciones, y se pone. Y te entrega no veinte líneas, sino dos mil. A veces más. En múltiples archivos. Con dependencias entre ellos. Con decisiones de diseño que no le has pedido explícitamente y que él ha tomado por su cuenta.

Esto es muy distinto a lo de antes, aunque por dentro se sienta parecido. El prompt es igual de natural, el flow es igual de adictivo, la sensación de "esto está saliendo bien" es la misma. Pero el resultado ya no cabe en tu cabeza. No vas a leer las dos mil líneas de un vistazo. Y si no las lees, no las has revisado. Y si no las has revisado, ese código no es del todo tuyo: lo firma tu nombre, pero la cabeza que lo escribió fue la del modelo.

Cuando alguien dice "estoy programando con Claude" y se refiere a este modo, lo que está pasando en realidad es que el modelo programa y la persona observa. Es un cambio de rol que no se hizo explícito. Te crees programador y has empezado a ser tester por debajo de la mesa. Lo malo es que tester sin spec — tester que solo prueba si "parece que funciona" y mira si los logs no escupen rojo.

He visto esto en sitios reales más veces de las que me gustaría contar. El patrón típico: una funcionalidad sale rápido, todo el mundo contento, se mete en producción. Dos semanas después aparece un comportamiento raro que nadie sabe explicar. El dev que la generó abre el código, se sienta a leerlo, y se da cuenta de que no recuerda haber escrito eso — porque no lo escribió. Lo aceptó. Que no es lo mismo.

Bloque 3 · El coste real, en serio

A primera vista parece que el coste de no tener spec es solo "puede haber bugs". Y sí, los hay. Pero ese es el coste obvio, no el caro. Los costes que de verdad muerden son tres, y son acumulativos.

El primero es el coste de contexto perdido entre sesiones. Cuando vuelves al código una semana después y no te acuerdas de por qué tomaste tal decisión, tienes que reconstruir tu propio razonamiento. Si fuiste tú quien escribió cada línea, esa reconstrucción es lenta pero hacedera — algo se te queda en la memoria muscular. Si fue el modelo el que escribió y tú aceptaste, no hay memoria muscular que valga. Estás leyendo código ajeno, solo que es ajeno *tuyo*. Y eso es lo peor de los dos mundos.

El segundo es el coste de los refactors invisibles. Cuando el modelo genera código, mezcla patrones, librerías y convenciones. A veces usa un patrón en un sitio y el contrario en otro, dentro del mismo archivo. Si no le pones límites — y la spec es precisamente cómo le pones límites — terminas con una base de código en la que cada módulo huele distinto. Cuando llega el momento de hacer mantenimiento, no puedes mover una pieza sin entender las otras, porque no comparten convenciones.

El tercero es el más sutil y el más caro: la regresión que no detectas. Como nadie escribió la spec, no hay tests que la cubran. Hay tests que cubren el comportamiento que el modelo decidió implementar. El día que un cambio rompe algo que el código hacía sin querer pero que alguien usaba, no hay forma de pillarlo. El test no existe, porque ese comportamiento nunca se especificó: se materializó al generar el código y nadie se enteró.

Si alguno de estos tres síntomas te suena, no es que tu equipo lo esté haciendo especialmente mal. Es lo que pasa por defecto cuando se aplica el método de trabajo de 2023 a las capacidades de 2026.

Bloque 4 · El cambio que cambia las reglas

Hay una asimetría nueva que merece la pena tener clara, porque es la que justifica todo lo que viene en el resto del curso.

Antes, el código lo escribía una persona, y el único que iba a leerlo era otra persona. La documentación servía para que ese otro humano entendiera qué estaba pasando. Si el código era pequeño y obvio, la documentación se podía obviar; si era grande, se hacía un esfuerzo.

Ahora hay un tercero en la sala: el modelo. Y el modelo también lee el código, también lee la spec, también lee los comentarios. Y a diferencia del humano, lo lee *cada vez* — cada sesión empieza de cero, sin memoria de la anterior. El modelo es el peor compañero de equipo posible en un aspecto y el mejor en otro: no recuerda nada, pero está dispuesto a leerlo todo otra vez.

Esto cambia las reglas porque la spec deja de ser "lo que el humano lee para entender", y pasa a ser "lo que el sistema entero — humanos y modelos — usa como contrato". Si tu equipo va a colaborar con IA en serio, la spec ya no es opcional. Es el único sitio donde la intención del proyecto vive de forma que tanto las personas como el modelo puedan respetarla.

Y aquí viene el detalle que más cuesta interiorizar: una spec bien escrita hace que el modelo rinda mejor que un prompt suelto. No marginalmente mejor. Notablemente mejor. Porque le das a leer la intención, no solo la petición. Y eso reduce las decisiones que tiene que tomar por su cuenta — que es donde mete el ruido.

Bloque 5 · Qué necesitas hacer distinto

No se trata de volver al modelo de cascada, ni de escribir documentos de cien páginas antes de tocar un teclado. Eso ya lo intentamos en los noventa y salió mal por motivos que siguen siendo válidos.

Se trata de tres cosas concretas, y vamos a verlas con detalle a lo largo del curso.

La primera: escribir specs cortas y útiles antes de pedir código no trivial. No specs ceremoniosas. Specs que digan qué tiene que hacer la cosa, para qué existe, con qué entradas y salidas, y qué cuenta como "hecho". Cinco minutos de escritura que te ahorran horas de "esto no era lo que quería".

La segunda: usar Plan Mode de Claude Code (o el equivalente en tu herramienta) antes de implementar. Que el modelo te enseñe el plan, que tú lo leas, que lo discutas, que solo cuando esté firmado se pase a ejecutar. Esto solo lo añade unos minutos al ciclo y filtra una cantidad enorme de malentendidos.

Y la tercera: poner hooks de validación que cierren el bucle automáticamente. Tests que comprueban contra la spec, validadores que no dejan pasar código fuera de scope, comprobaciones de que lo entregado se parece a lo pedido. Esto es lo que diferencia un equipo que usa SDD de verdad de uno que solo dice usarlo.

Las tres piezas son tres módulos completos del curso: M2 va de specs, M3 de Plan Mode y skills, M4 de hooks. Antes de meternos en ellas, en M1.2 vamos a poner nombre a esto que estamos haciendo y a separarlo de lo que ya conocías: TDD, BDD, DDD. Tienen apellido en común y son cosas muy distintas.

Por ahora, si te llevas una sola idea de este submódulo, que sea esta: el problema no está en la IA generando código, sino en la falta de contrato cuando esa generación supera tu capacidad de lectura. La spec es el contrato — y por eso el resto del curso gira a su alrededor.

Glosario rápido del bloque

- **Vibe coding:** modo de trabajo con IA basado en pedir, aceptar y ajustar sin método ni revisión sistemática. Acuñado por Andrej Karpathy, principios de 2025.
- **Spec:** documento corto que fija qué tiene que hacer una funcionalidad, para qué, con qué entradas/salidas y qué cuenta como "hecho". No es documentación: es contrato verificable.
- **Plan Mode:** modo de Claude Code en el que el modelo enseña un plan antes de ejecutar y espera tu firma. Otras herramientas tienen equivalentes con otro nombre.
- **Hook:** gancho automático que se ejecuta antes o después de una acción (edición, commit, llamada a tool). Sirve para validar, bloquear o registrar.

Qué viene en M1.2

Definir SDD operativamente y separarlo de TDD, BDD y DDD. Las cuatro comparten letras y casi nada más. El siguiente submódulo distingue qué resuelve cada una y por qué se confunden tanto en la práctica.

M1.2 · Qué es Specification-Driven Development

Bloque 0 · Lo que toca poner en orden

En el submódulo anterior dejamos el diagnóstico: cuando la IA genera más código del que puedes leer, la falta de contrato te muerde. Lo que toca ahora es nombrar la respuesta y, sobre todo, separarla de cosas que se le parecen y no son lo mismo.

Lo digo de entrada porque sé que llevas un rato con la duda: ¿esto no es TDD con otro nombre? ¿O BDD? ¿O DDD pasado por la batidora de la moda? La respuesta corta sería sí, hay solapes, pero la respuesta útil es que no — y entender la diferencia ahorra discusiones tontas en los próximos meses.

En quince minutos vamos a pasar por cuatro cosas: una definición de SDD que te puedas llevar a casa, las tres comparaciones que tocan (TDD, BDD, DDD) y el detalle que cambia todo y que sin la IA no había forma de aprovechar.

Bloque 1 · Una definición que se sostenga

Specification-Driven Development es una forma de construir software en la que **la especificación es el artefacto de primera clase del proceso**: la cosa que se escribe primero, la cosa que se revisa con atención, la cosa que vive en el repositorio junto al código, y la cosa contra la que se valida lo que se entrega. Todo lo demás — el plan, el código, los tests, las migraciones — se deriva de ella.

Para aterrizar esa definición, hay cuatro cosas que tiene que cumplir una spec para serlo de verdad en este sentido:

Ejecutable: no en el sentido de que se ejecute por sí sola, sino de que está escrita con la precisión necesaria para que un agente (humano o IA) pueda partir de ella e implementar sin tener que rellenar huecos a ojo. Si tu spec deja diez ambigüedades importantes, no es una spec ejecutable: es un brief.

Verificable: al terminar la implementación tiene que ser posible decir, con criterio claro, si lo entregado cumple la spec o no. No "parece que funciona", sino "estos cuatro criterios de aceptación están todos en verde". Si no se puede comprobar contra la spec, la spec no sirve para cerrar el ciclo.

Versionada: vive en git como el código. Cuando cambias la spec, cambia el commit. Cuando cambia el requisito, cambia la spec, no se cambia el código por la cara. Esto suena obvio escrito y tiende a no pasar en la práctica: la spec se desincroniza del código y, a los dos meses, nadie sabe cuál de los dos refleja la realidad.

Compartida: la lee todo el que toca el proyecto: el dev, el revisor, el agente, el tester. No es un documento personal de quien la escribió. Si tu spec necesita que alguien te la explique, todavía no es una spec compartida.

Cuatro adjetivos y un sustantivo. Esa es la definición que vamos a usar el resto del curso.

Bloque 2 · SDD frente a TDD

Test-Driven Development lleva décadas con nosotros. Escribes el test primero, lo ves fallar, implementas lo mínimo para que pase, refactorizas. Rojo, verde, refactor. Funciona (para lo que está pensado) y es uno de los grandes inventos del oficio.

¿Por qué SDD no es TDD con otro nombre?

Porque responden a preguntas distintas. El test contesta a "¿hace esta función lo que tiene que hacer?". La spec contesta a "¿qué es lo que tiene que hacer esta función, y por qué, y dónde encaja en el sistema?". Los tests son una **proyección** de la spec, no la spec misma. Una spec puede generar muchos tests; un test no genera una spec.

Hay otra diferencia que cuesta más explicar pero importa más. TDD nació en una época en la que el código lo escribía un humano: el test te ataba las manos para no irte de tema, y el ciclo rojo-verde-refactor te forzaba a pensar antes de teclear. Cuando es la IA la que escribe, ese cinturón ya no sirve igual. Le puedes pasar todos los tests del mundo al modelo y va a escribir el código que los hace pasar — pero también puede tomar veinte decisiones de diseño por su cuenta que no están cubiertas por ningún test, y ahí es donde se cuela la deuda. La spec captura esas decisiones antes de que se tomen mal.

En la práctica los dos conviven sin problemas. SDD no sustituye a TDD: lo precede. Primero spec, luego tests derivados de la spec, luego implementación contra los dos. Y los tests siguen siendo el portero de la pelota: la spec dice qué hay que hacer, los tests comprueban que se ha hecho.

Una historia rápida. He visto un equipo que llevaba dos años con cobertura de tests al 90% y aún así tenían bugs en producción con regularidad. Lo curioso es que los bugs no eran de código mal escrito: eran de funcionalidad que faltaba. El test confirmaba que lo implementado funcionaba; nadie se daba cuenta de que faltaba implementar la mitad del caso de negocio. Sin spec, los tests cubren lo que existe, no lo que tendría que existir.

Bloque 3 · SDD frente a BDD

Behavior-Driven Development es la siguiente parada lógica. Aquí el cruce con SDD es más fino, porque BDD ya trabaja con descripciones en lenguaje cercano al natural: los famosos *Given / When / Then*.

¿Dónde está la diferencia?

BDD describe comportamiento observable. "Dado un usuario autenticado, cuando hace clic en cerrar sesión, entonces vuelve a la pantalla de login." Eso es comportamiento, es claro y verificable. Pero responde solo al "qué se ve". No responde al "por qué existe esto", al "qué pasa si el comportamiento se contradice con otro", al "cuáles son las invariantes del sistema que no se pueden romper aunque cambie el comportamiento de pantalla".

Una spec, en el sentido SDD, sí responde a eso. Una spec lleva intención (por qué esto importa), comportamiento (cómo se manifiesta) y contrato (qué interfaces y qué tipos, si hace falta). BDD es un subconjunto: cubre bien el behavior, deja fuera el intent y el contract.

La buena noticia es que un escenario BDD bien escrito puede vivir dentro de una spec SDD como uno de sus apartados. No se canibalizan. La mala noticia es que un equipo que solo hace BDD se queda corto cuando los modelos empiezan a generar código: el modelo necesita más contexto que "estos cinco escenarios". Necesita el porqué, y eso es lo que BDD no llevaba escrito.

Bloque 4 · SDD frente a DDD

Domain-Driven Design es la comparación más despistante. Aquí ni siquiera estamos en el mismo nivel: son cosas distintas, y solo se confunden porque comparten dos letras en el acrónimo y ambas hablan de capturar conocimiento del dominio.

DDD es una forma de modelar dominios complejos: contextos acotados, lenguaje ubicuo, agregados, repositorios, eventos. Es arquitectura conceptual. Te dice cómo pensar el sistema, no qué construir hoy.

SDD trabaja un escalón por debajo: cómo describir lo que toca construir aquí y ahora, en este sprint, para este equipo. Si tu dominio es complejo, tu spec va a apoyarse en los conceptos del modelo de dominio que ya tienes — usará los mismos nombres, respetará los mismos límites de contexto. Pero la spec no es el modelo de dominio. La spec es la unidad de trabajo concreta.

Visto al revés: DDD te da el vocabulario; SDD te da la frase. Si haces SDD sin un modelo de dominio claro, vas a notar que tus specs se llenan de explicaciones de qué es cada cosa. Si haces DDD sin specs claras, tienes un mapa precioso del territorio y nadie sabe qué tarea hacer mañana por la mañana.

En equipos serios suelen vivir juntos. Tu organización tiene su DDD (formal o informal) que define cómo se entiende el negocio; cada feature lleva su spec SDD que dice qué pieza concreta se va a construir esta semana respetando ese marco. No compiten — se complementan a niveles distintos.

Bloque 5 · Lo que la IA cambia de verdad

Aquí está el detalle que no se puede separar de la conversación, porque es lo que hace que SDD pase de "buena idea que ya teníamos hace veinte años" a "buena idea que por fin sale a cuenta".

Antes, escribir una spec útil costaba caro y se usaba poco. Costaba caro porque el único que iba a leerla era otro humano, y los humanos somos malos lectores de documentos largos: nos saltamos párrafos, nos confiamos del título, leemos por encima. El esfuerzo de escribir una spec rigurosa se amortizaba mal — al final el dev terminaba haciendo lo que entendía, y la spec acumulaba polvo.

Con la IA hay un cambio de actor. El modelo es un lector raro: no se cansa, no se salta párrafos, no asume que ya sabe lo que viene. Lee literalmente lo que pone. Y a diferencia del compañero junior, no recuerda nada de lo que se discutió en la reunión — solo va a tener lo que esté escrito.

Eso convierte a la spec en lo que siempre debió ser: la fuente única de verdad para el proceso. Si la spec lo dice, el modelo lo respeta. Si la spec no lo dice, el modelo va a tomar una decisión por su cuenta — y casi siempre es una decisión razonable pero no la que tú harías. Cuando rellenas los huecos de la spec, lo que estás haciendo es quitarle al modelo decisiones que iba a tomar mal por falta de contexto.

Esta es la asimetría que hace SDD viable hoy y no hace veinte años. El coste de escribir una spec útil ha bajado (la IA te ayuda a escribirla, a revisarla, a detectar ambigüedades), y el beneficio ha subido (la IA la lee bien y la respeta). Antes era una herramienta cara con retorno discreto; ahora es una herramienta barata con retorno alto. Eso cambia el cálculo entero.

Si vienes de odiar las specs porque te recuerdan a los documentos de cien páginas de los años noventa, sigue leyendo. Las specs de SDD no se parecen a aquello. Son documentos cortos, vivos, escritos con la herramienta que va a leerlos. Y se escriben en cinco minutos cuando aprendes el formato — que es justo el M2.

Glosario rápido del bloque

- **Spec (en SDD):** documento corto, ejecutable, verificable, versionado y compartido que describe qué hay que construir, para qué, con qué entradas/salidas y qué cuenta como hecho.

- **TDD (Test-Driven Development)**: metodología en la que el test se escribe antes que el código. Convive con SDD; los tests son una proyección de la spec.
- **BDD (Behavior-Driven Development)**: metodología que describe el comportamiento observable con *Given / When / Then*. Subconjunto de lo que cubre una spec SDD.
- **DDD (Domain-Driven Design)**: enfoque para modelar dominios complejos. Vive un escalón por encima de SDD; le da vocabulario al sistema entero, no a una feature.
- **Artefacto de primera clase**: documento que vive en el repositorio, se versiona, se revisa en PR y forma parte del proceso normal del equipo — al mismo nivel que el código.

Qué viene en M1.3

Por qué tratar la spec como objeto de primera clase no es una cuestión de gusto sino de supervivencia cuando hay IA en el flujo. Veremos qué pasa cuando la spec deja de estar al día, dónde se rompen los equipos que no la versionan, y por qué el patrón "la spec en Notion, el código en git" es el camino más corto al desastre. Trae sospechas honestas; las vamos a confirmar.

M1.3 · El "spec" como objeto de primera clase

Bloque 0 · Lo que toca matizar

En el submódulo anterior dejamos cuatro adjetivos para definir spec: ejecutable, verificable, versionada y compartida. Y un quinto concepto envolviendo a los otros: artefacto de primera clase del proceso.

Ese último concepto es el que aguanta toda la práctica. Si los otros adjetivos te dicen *cómo* debe estar escrita una spec, el de "primera clase" te dice *dónde* debe vivir y *cómo* debe tratarse en el día a día. Y aquí es donde la mayoría de equipos se descalabra, no por mala fe, sino por inercia: la spec vive en un sitio, el código en otro, y los dos se separan poco a poco hasta que nadie sabe cuál de los dos refleja la verdad.

En este submódulo vamos a ser concretos. Qué quiere decir "primera clase" en términos operativos, qué pasa cuando una spec deja de serlo, el antipatrón más común con nombre y apellido, y un matiz importante: no todo trabajo necesita spec, y forzarlo es peor que no tenerla.

Bloque 1 · Qué quiere decir "primera clase"

El término "ciudadano de primera clase" viene de los lenguajes de programación. Una función es ciudadano de primera clase en un lenguaje si puedes pasarla como argumento, devolverla como resultado, asignarla a una variable y guardarla en una estructura de datos. Es decir: si el lenguaje la trata como cualquier otro valor, sin un estatus inferior.

Trasladado a las specs: una spec es artefacto de primera clase si tu proceso de equipo la trata como trata el código. Eso se concreta en cinco cosas pequeñas y aburridas, todas necesarias.

Vive en el mismo repositorio que el código. No en Notion, no en Confluence, no en un Google Doc. En `git`. Junto al código que describe, idealmente en el mismo directorio o en uno paralelo (`/specs` , `/docs/specs` , donde el equipo decida) que esté dentro del repo del proyecto.

Se versiona como el código. Cuando la spec cambia, el cambio queda registrado en un commit con su autor, su fecha y su motivo. Tienes `git blame` sobre la spec igual que sobre el código. Si alguien te pregunta "¿por qué la spec dice esto?", puedes contestar mirando la historia, no preguntando a la gente que estaba en la reunión hace tres meses.

Se revisa en pull request. Cuando alguien propone una spec nueva o un cambio sustancial a una existente, abre un PR como si fuera código. Otra persona la revisa, hace comentarios, pide

cambios, aprueba. Solo cuando merge a `main` se considera vigente. Si tu equipo aprueba specs por Slack, no es primera clase: es una conversación con buena memoria.

Se cierra cuando se cumple. Una spec abierta sin cumplir indefinidamente es ruido. Cuando lo descrito está implementado y validado, la spec puede marcarse como completada (un tag, un movimiento de carpeta, lo que sea consistente para el equipo). Específicamente, si has implementado algo, ese "algo" tiene una spec cerrada que dice qué se hizo y por qué.

Es la fuente única de verdad cuando hay contradicción. Si el código hace una cosa y la spec dice otra, alguno de los dos está mal. La regla operativa: la spec gana. O bien actualizas la spec para reflejar la nueva realidad (con razón explícita), o cambias el código para que cumpla la spec. Lo que no puede pasar es que ambos sigan vivos diciendo cosas distintas.

Esos cinco puntos son la versión operativa de "primera clase". Si tu equipo cumple las cinco, tienes specs de primera clase. Si te faltan dos, tienes documentación. Si te faltan tres, tienes mitos.

Bloque 2 · Specs como entrada, no como salida

Aquí está la confusión más cara que veo en equipos que intentan adoptar SDD sin método: tratar la spec como un documento que se escribe DESPUÉS, para dejar constancia de lo que se hizo. Eso no es SDD. Eso es documentación post-mortem.

La spec es el artefacto que abre el trabajo. Existe antes de la primera línea de código. Se escribe cuando estás decidiendo qué construir, no cuando estás cerrando el ticket. El acto de escribirla es lo que te obliga a pensar antes de teclear, y a discutir antes de implementar.

¿Por qué importa el orden tanto? Por dos motivos prácticos.

El primero es que escribir una spec a posteriori es muy fácil pero muy poco útil. Cuando ya has implementado, cualquier ambigüedad del enunciado original ya la resolviste a tu manera mientras programabas. La spec que escribes después es la spec que describe lo que hiciste, no lo que tendrías que haber hecho. Has perdido el momento en el que las decisiones eran reversibles.

El segundo es que la spec a priori es la que filtra los malentendidos antes de que cuesten. Si tres personas leen la spec antes de empezar a codificar, los puntos de fricción salen en la revisión y se discuten en la conversación. Si la spec aparece después, los malentendidos solo se ven cuando alguien usa el código y dice "esto no era lo que yo pedí" — y entonces ya hay código escrito que toca reescribir.

Si vienes de un mundo agile clásico, esto puede sonarte mal porque el reflejo es "trabajar incremental, no hagas big design up front". Y tiene razón el reflejo. Pero ojo: una spec SDD no es

un BDUF de seis meses. Es un documento corto, vivo, que se escribe en cinco o diez minutos para la pieza que vas a abordar en los próximos uno o dos días. No estamos hablando de cascada: estamos hablando de pensar antes de teclear, aunque sea solo por veinte minutos, sobre lo que vas a hacer esta tarde.

Una historia que ilustra. Un equipo con el que trabajé adoptó "vamos a documentar todo" como respuesta a quejas de mantenimiento. Cada feature implementada salía con su documento de spec, en un Confluence muy ordenado. A los cuatro meses tenían cuatrocientos documentos y cero ahorro en mantenimiento — porque las specs decían lo que el código ya decía si te leías el código. Lo que faltaba seguía faltando: la conversación antes de implementar. Cambiaron a escribir specs de tres páginas ANTES de cada feature, revisadas en PR, y la calidad de las preguntas en code review cambió en dos sprints. Mismo equipo, misma gente, mismo dominio. Lo que cambió fue el orden.

Bloque 3 · El ciclo de vida de una spec viva

Una vez que la spec es de primera clase y va por delante del código, tiene ciclo de vida propio. No nace y muere con la feature: tiene sus estados, sus revisiones, sus actualizaciones cuando el mundo cambia. Resumido en cuatro estados.

Borrador: alguien la ha escrito, pero todavía no la ha revisado nadie. Está en un PR abierto. Aquí es donde se ajustan las ambigüedades, donde el revisor pide ejemplos concretos, donde se decide si el alcance es razonable o si la pieza es demasiado grande y hay que partirla.

Vigente: la spec ha sido aprobada y mergeada. Es lo que el equipo va a implementar. A partir de este momento, cualquier código nuevo que se escriba debe cumplirla, y cualquier modelo (humano o IA) que la lea puede asumir que es lo que el equipo decidió.

En revisión: cuando el mundo cambia (el cliente pidió otra cosa, la regulación cambió, alguien descubrió un problema en producción), la spec entra en un nuevo PR de modificación. Aquí no se cambia el código primero y luego se actualiza la spec — al contrario. La discusión es sobre la spec; el código viene detrás.

Cerrada: lo que la spec describía está implementado, validado y en producción. La spec se mueve a un sitio de "cerradas" (tag, carpeta, label, lo que el equipo prefiera) para que las specs vigentes sean siempre las que de verdad están pendientes o vivas. Las cerradas siguen consultables — son historia útil — pero no son lo que el equipo tiene encima de la mesa hoy.

El ciclo no es lineal. Una spec puede pasar de vigente a en revisión varias veces a lo largo de la vida de la feature, especialmente si la pieza es grande o vive evolucionando. Lo importante es que

en cada momento puedes saber en qué estado está cada spec del proyecto, y que ese estado refleja la realidad — no lo que alguien creía recordar.

Bloque 4 · El antipatrón Notion-vs-git (y todos sus primos)

Vamos al antipatrón más extendido, porque casi todos los equipos lo cometen sin verlo. La spec vive en una herramienta dedicada a documentación (Notion, Confluence, Google Docs, Coda, lo que sea de moda), y el código vive en git. Las dos cosas evolucionan por separado.

¿Qué tiene de malo? A primera vista nada — la herramienta de documentación es mejor para escribir documentos, tiene mejor formato, mejor búsqueda, mejor colaboración en tiempo real. Y es verdad. Pero ese no es el problema.

El problema es que cuando dos artefactos viven en dos sistemas distintos, se desincronizan. Sin excepción. Algunos ejemplos del patrón general:

- La spec en Notion dice que la API devuelve 200 con un payload. El código en git devuelve 204 sin body. Nadie se ha dado cuenta porque la spec se escribió antes y nadie volvió a leerla.
- La spec describe tres roles de usuario. El código tiene cuatro. El cuarto se añadió en un sprint de urgencia y nadie pensó en abrir Notion.
- La spec tiene tres versiones (la del PM, la del tech lead, la del backend). Cada una en una página distinta. Nadie sabe cuál es la buena.
- El PM actualiza la spec porque el cliente cambió de idea. El backend no se entera porque ya no lee Notion. Implementa lo que recuerda. La realidad y la spec divergen sin que nadie tenga la culpa.

Este patrón se produce porque la separación física de los sistemas elimina la fricción que mantiene las cosas sincronizadas. Cuando todo vive en el mismo repo y los cambios pasan por PR, una persona que ve el PR del código nota que falta el PR de la spec, o al revés. Cuando viven separados, esa señal desaparece. Y sin señal, no hay quien lo arregle a tiempo.

La solución no es que tu documentación sea fea. La solución es que la spec viva donde vive el código y que se renderice bonita cuando se necesite (markdown plano en git, renderizado en la pestaña de GitHub o GitLab, o exportado a la herramienta de documentación como copia de lectura, nunca al revés).

Una variante más sutil del mismo antipatrón: la spec vive en un repo separado del código. Pasa con proyectos grandes que tienen un repo para "documentación" y otro para "implementación". Sufre el mismo problema, solo que más despacio. Recomendación práctica: la spec de la pieza vive en el repo de la pieza, no en un repo de docs corporativo.

Bloque 5 · Cuándo NO escribir spec

Aquí toca matizar para que no salgas de este submódulo escribiendo specs hasta para cambiar un margen de CSS. No todo trabajo necesita spec, y forzarlo te hace perder tiempo sin ganancia.

La heurística práctica es triple, y basta con que falle uno de los tres criterios para no escribir spec.

¿La implementación va a llevar más de una hora de trabajo concentrado? Si es media hora de tocar un componente, no. Si son tres días, sí.

¿Va a tocar más de un archivo importante o más de un módulo? Si el cambio cabe en una sola función, normalmente no necesita spec. Si toca el flujo entre dos servicios, sí.

¿Afecta a comportamiento observable por el usuario, por otro sistema, o por algún test relevante? Si es un refactor interno que nadie va a notar y no cambia interfaces, normalmente no. Si cambia un endpoint o el formato de una respuesta, sí.

Tres síes y la cosa pide spec. Tres noes y probablemente estás sobreingenierizando el proceso. Uno o dos noes y el equipo decide según contexto.

Hay otra categoría que merece mención aparte: los bugfixes. Para un bugfix pequeño, una spec completa es ridículo. Lo que sí ayuda es una mini-spec de cuatro líneas que diga el síntoma, la causa que has identificado, qué cuenta como "arreglado" y qué tests cubren la regresión. Esos dos minutos se pagan solos: el fix queda con su contexto dentro del propio PR. Es el formato más liviano de spec y se queda para historia.

La regla informal del oficio: si te dudas si escribir spec o no, escríbela. El coste de escribirla cuando no hacía falta es bajo (cinco minutos). El coste de no escribirla cuando sí hacía falta puede ser alto (un sprint perdido en re-trabajo). Asimetría favorable.

Glosario rápido del bloque

- **Artefacto de primera clase:** documento que se trata como el código en el día a día del equipo: vive en el repo, se versiona, se revisa en PR, se cierra cuando se cumple, gana cuando contradice al código.
- **Spec a priori vs a posteriori:** a priori se escribe antes de implementar y guía el trabajo; a posteriori se escribe después y documenta lo hecho. SDD trabaja con la primera, no con la segunda.
- **Antipatrón Notion-vs-git:** spec en herramienta de documentación, código en git, ambos se desincronizan con el tiempo. Solución: spec en el mismo repo, en markdown.
- **Mini-spec de bugfix:** spec de cuatro líneas (síntoma, causa, criterio de fixed, tests de regresión) que vive dentro del PR del propio fix. Formato más liviano de spec.

Qué viene en M1.4

Después de tres submódulos de método, toca ver el método en movimiento. La mini-demo que cierra el módulo: el mismo problema pequeño abordado dos veces, primero con vibe coding puro, después con SDD básico. Mismo problema, mismo modelo, distinto resultado — y distinto coste de mantenimiento dos semanas después. Tras esa demo, el M2 entra en cómo se escribe una spec útil sin que te cueste media mañana.

M1.4 · Mini-demo — el mismo problema, dos formas

Bloque 0 · Para qué esta demo

Los tres submódulos anteriores han fijado el marco: por qué el vibe coding deja de funcionar a partir de cierta escala, qué es SDD operativamente y qué significa tratar la spec como artefacto de primera clase. Hasta aquí teoría. Útil, pero teoría.

En este último submódulo del M1 toca poner el método en movimiento. No con un proyecto de tres meses, que sería imposible de seguir. Con algo deliberadamente pequeño: una función aparentemente trivial que cualquiera escribiría en cinco minutos, y que esconde cinco o seis decisiones implícitas que la IA va a tomar por su cuenta sin avisarte.

Vamos a resolverla dos veces. Primero con vibe coding puro: prompt rápido a Claude Code, le das al enter, ves qué sale. Después, una spec corta de media página antes de tocar el código, y comparamos. Mismo problema, mismo modelo, mismo idioma de petición. Lo único que cambia es si hay spec o no la hay.

Y al final, una comparativa honesta. Porque SDD no es gratis — escribir una spec cuesta cinco minutos — y toca tener claro qué ganas y qué pagas. Si vendiera SDD como pura ganancia sin coste, sería un vendedor de humo.

Bloque 1 · El problema que vamos a resolver

El enunciado, tal y como te lo daría un compañero por Slack a las cinco de la tarde de un viernes: **"Necesito una función que calcule la fecha de vencimiento de una factura."**

Suena a algo de quince líneas, ¿verdad? Eso es lo que va a hacer la IA si le pasas ese prompt sin más. Y eso es lo que va a tener problemas dos semanas después.

Antes de implementar nada, párate un segundo: si tú escribieras esa función sin más contexto, ¿qué decisiones tendrías que tomar tú solo? Te las anoto, para que veas el tamaño real del iceberg debajo del enunciado:

1. **¿La fecha base es la de emisión, la de entrega o la de aceptación?** Las tres existen, y dan resultados distintos.
2. **¿Cuántos días por defecto?** 30 es el clásico, pero hay sectores donde es 60, 90 o 15.
3. **¿Días naturales o días hábiles?** Sumar 30 días naturales o 30 días hábiles te puede dar una diferencia de dos semanas.

4. **Si el vencimiento cae en fin de semana, ¿qué pasa?** ¿Se queda ahí? ¿Salta al lunes siguiente? ¿Al viernes anterior? Las tres son legítimas según el sector.
5. **¿Y si cae en festivo?** Pregunta similar, respuesta posiblemente distinta. Y festivos según país, según comunidad autónoma, según convenio.
6. **¿Influye el tipo de cliente o el monto?** Los clientes VIP pueden tener 60 días en lugar de 30. Las facturas grandes pueden tener condiciones especiales.
7. **¿Zonas horarias?** Si la fecha de emisión es un timestamp UTC y el cliente está en Europe/Madrid, ¿de quién es la medianoche que cuenta?

Siete decisiones, no quince líneas. Y todas relevantes en algún negocio real. La función "trivial" no existe — solo existen funciones donde alguien ha tomado las decisiones por ti antes de pedírtela. Cuando ese alguien no aparece, las toma quien implementa: tú o, en vibe coding, el modelo.

Bloque 2 · Intento 1, vibe coding puro

Le pasas a Claude Code el enunciado tal cual: *"Necesito un método en C# que calcule la fecha de vencimiento de una factura. La factura tiene FechaEmision y Monto."* Le das al enter. En diez segundos tienes esto:

```
public static class FacturaService
{
    private const int DiasVencimientoDefault = 30;

    public static DateTimeOffset CalcularFechaVencimiento(Factura factura)
    {
        return factura.FechaEmision.AddDays(DiasVencimientoDefault);
    }
}

public record Factura(DateTimeOffset FechaEmision, decimal Monto);
```

Compila. Tiene tipos. Tiene una constante con nombre semántico, casi una buena práctica. Si le pides tests con xUnit, te los escribe. Si los ejecutas, pasan. Si lo metes en producción, funciona.

¿Qué tiene de malo?

Mira las siete preguntas del bloque anterior y compara con la respuesta. El modelo ha tomado las siete decisiones por su cuenta sin avisarte:

- Fecha base: la de emisión. (Razonable, pero no avisó.)
- Días por defecto: 30. (Asumido sin preguntar.)
- Días naturales, no hábiles. (Decisión silenciosa que va a romper el cálculo en cuanto el negocio diga "queremos días hábiles".)

- Fin de semana: la fecha sale tal cual, aunque caiga en domingo. (Sin lógica de ajuste.)
- Festivos: no existen.
- Tipo de cliente o monto: ignorados. La función no recibe esa información.
- Zona horaria: se hereda del entorno donde se ejecute. Si se ejecuta en un servidor con TZ distinta a la del usuario, los resultados van a divergir.

Y todo esto sin advertencia. El código se ve limpio, pasa los tests automáticos que tú o el modelo escribáis para verificar el comportamiento que el código tiene, no el que tendría que tener. Las siete decisiones quedan enterradas en líneas que parecen inocentes.

Tres semanas después, el cliente reclama que una factura emitida un viernes vence un domingo y no la pueden pagar a tiempo. Vas al código, miras la función, ves la lógica simple, y te das cuenta de que nunca te paraste a definir qué pasaba con fines de semana. No es un bug — es una decisión no tomada que se materializó como una decisión por defecto que ahora no encaja.

Bloque 3 · Intento 2, empezando por una spec corta

Mismo problema, otro arranque. Antes de pedirle a Claude Code que implemente nada, abres `specs/calcular-fecha-vencimiento.md` y escribes esto:

Spec: calcular fecha de vencimiento de factura

Intent

Determinar la fecha límite hasta la que el cliente puede pagar una factura sin penalización, según las condiciones comerciales acordadas con ese cliente.

Behavior

Caso normal

- Entrada: Factura con FechaEmision (DateTimeOffset UTC), MontoTotal (decimal), y Cliente con TipoCliente (regular | VIP) y PaisFacturacion (ISO 3166-1).
- Salida: DateTimeOffset en Europe/Madrid (00:00:00).
- Cálculo base: FechaEmision + plazo, donde plazo = 30 días naturales para cliente regular, 60 días naturales para cliente VIP.

Ajustes

- Si el vencimiento calculado cae en sábado o domingo, se mueve al lunes laboral siguiente.
- Si cae en festivo del PaisFacturacion (lista en `'recursos/festivos.json'`), se mueve al siguiente día laboral.
- Si tras los dos ajustes el vencimiento cae más allá del último día del mes siguiente al de emisión, se trunca al último día del mes siguiente.

Contract

- Método puro, sin efectos secundarios.
- No lanza excepciones para datos válidos.
- Lanza `'InvalidInvoiceException'` si FechaEmision está en el futuro o si TipoCliente no es un valor esperado.

Criterios de aceptación

- [] Factura regular emitida en lunes paga en lunes 30 días después.
- [] Factura regular emitida en jueves cuyo vencimiento calculado cae en sábado paga el lunes siguiente.
- [] Factura VIP tiene 60 días en lugar de 30.
- [] Festivo en PaisFacturacion mueve el vencimiento al siguiente día laboral.
- [] Tests xUnit cubren al menos un caso para cada regla del Behavior.

Cinco minutos de escritura. Notarás algo importante: al escribir la spec ya no puedes pasar de las decisiones — te obliga a tomarlas con cabeza. Cuando llegues a "días naturales o hábiles", no puedes dejarlo en blanco. Tienes que escribir uno u otro, o explicar la regla. La spec te fuerza a pensar antes de teclear, y eso solo ya valdría la pena aunque la IA no existiera.

Ahora le pasas esta spec a Claude Code y le dices: *"Implementa lo descrito en `specs/calcular-fecha-vencimiento.md`."* En treinta segundos tienes una implementación con:

- Lectura del fichero `recursos/festivos.json` (vía `System.Text.Json`).
- Lógica de ajuste por fin de semana y festivos.
- Distinción entre cliente regular y VIP.
- Conversión a Europe/Madrid con `TimeZoneInfo.FindSystemTimeZoneById`.
- Validación de entrada con la excepción `InvalidInvoiceException`.

- Tests xUnit que cubren los cuatro criterios de aceptación.

No hay magia. Lo que ha cambiado es que el modelo tiene un contrato que cumplir, y rinde cuando lo tiene. Sin contrato, va por defecto a la solución simple plausible. Con contrato, sigue lo que pone. La diferencia no está en el modelo — está en lo que le has dado a leer.

Bloque 4 · Comparativa honesta, qué ganas y qué pagas

Vamos a poner los dos enfoques uno al lado del otro sin trampa. SDD tiene coste, y hay que reconocerlo si quieres venderlo bien dentro de tu equipo.

Tiempo total hasta tener algo en producción. Vibe coding: 30 segundos para el código + tiempo de revisión + tiempo de descubrir los bugs latentes después. SDD: 5 minutos de spec + 30 segundos del código + revisión + casi ningún bug latente porque las decisiones están explícitas. Si midieras solo el tiempo del minuto cero, vibe coding gana. Si mides hasta producción estable, SDD gana por kilómetros.

Capacidad de cambio. Vibe coding: cuando dos meses después el cliente diga "queremos cambiar el plazo a 45 días para clientes europeos", abres el código, lees las cuarenta líneas, recuerdas (o no) las decisiones implícitas, y modificas. SDD: abres la spec, ves que el plazo está en una sección concreta con su criterio explícito, lo actualizas, y le pides al modelo que ajuste la implementación. La spec es donde vive la decisión; el código es donde se ejecuta.

Onboarding de gente nueva. Vibe coding: el nuevo del equipo tiene que reconstruir las decisiones leyendo el código y preguntando "¿por qué hace esto así?". SDD: el nuevo lee la spec y sabe qué decidió el equipo y por qué. Bonus: si los tests están vinculados a los criterios de aceptación de la spec, el nuevo entiende qué se valida y qué no.

Colaboración con la IA en el futuro. Vibe coding: cada vez que vuelvas a esta función con un modelo nuevo, le toca rehacer el razonamiento desde cero, sin contexto. SDD: cualquier modelo, cualquier sesión, lee la spec y trabaja sobre la misma base. La spec es el contexto que sobrevive a las sesiones.

Lo que cuesta. Cinco a quince minutos de escritura por feature de tamaño medio. Tener disciplina para escribirla ANTES, no después. Discutirla con el equipo, lo que cuesta algo más al principio porque hay menos margen para "ya lo verás en el código". Decir que no a la urgencia de saltarse el paso. Aprender a escribirla en un formato que el modelo lea bien — que es lo que vamos a ver en el M2.

Para mí, después de unos meses con SDD, la respuesta es clara: el coste de cinco minutos al principio se paga con sobra en cualquier feature que pase del "ya lo arreglo luego". Pero cada equipo decide. Si tu producto es un prototipo desechable que vive dos semanas, vibe coding gana.

Si tu producto va a vivir meses o años y va a tener mantenimiento, SDD gana en todo el ciclo de vida.

Bloque 5 · Lo que cierra M1 y lo que abre M2

Tres ideas para llevarte de este módulo entero, antes de pasar al siguiente.

La primera: el problema no es la IA. La IA hace bien lo que le pides. El problema es que en vibe coding le pides cosas ambiguas que ella tiene que desambiguar por su cuenta, y se nota.

La segunda: la spec no es documentación a posteriori. Es la pieza que abre el trabajo, vive en el repo, se versiona como el código y se cierra cuando lo descrito está hecho. Si tu spec no cumple esas cuatro condiciones operativas, todavía no es spec en sentido SDD: es documento.

La tercera: SDD no es gratis pero sale a cuenta cuando el código va a sobrevivir más de un par de semanas. Cinco minutos al inicio te ahorran horas más adelante, y te dan código que cualquier modelo puede mantener leyendo la spec.

En el M2 entramos en cómo se escribe una spec útil de verdad: los tres niveles (intent, behavior, contract), los formatos que rinden (markdown plano, OpenAPI, JSON Schema según el caso), las plantillas reutilizables, y la revisión iterativa de la spec antes de implementar. Vamos a despiezar la spec de la factura que has visto en este submódulo y a ver por qué cada parte estaba donde estaba.

Si has llegado hasta aquí con todo claro, la parte difícil ya pasó. Lo que viene son herramientas para ejecutar bien lo que ya entiendes.

Glosario rápido del bloque

- **Decisión implícita:** opción tomada al implementar sin que nadie la haya planteado explícitamente. En vibe coding la toma el modelo por su cuenta; en SDD la toma quien escribe la spec.
- **Bug latente:** comportamiento que no es exactamente lo que el negocio quería pero que no se manifiesta como error hasta que las circunstancias cambian. El producto típico de las decisiones implícitas mal tomadas.
- **Contrato (en spec):** la parte de la spec que fija la interfaz exacta: tipos, signatures, excepciones, casos de error. No es código, pero es lo más cerca del código que llega la spec.
- **Criterios de aceptación:** condiciones específicas que la implementación debe cumplir para considerarse correcta. Vinculan la spec con los tests.

Qué viene en M2

El módulo entero está dedicado a aprender a escribir specs que rindan. M2.1 entra en los tres niveles (intent, behavior, contract) y cuándo basta con dos. M2.2 ve los formatos: markdown plano para la mayoría, OpenAPI y JSON Schema cuando hay contrato técnico que respetar. M2.3 plantea tres plantillas reutilizables (feature nueva, bugfix, cambio arquitectónico) y empieza desafiando la plantilla "típica" que circula por internet — la que probablemente has visto si has buscado "SDD template" en Google. M2.4 va de revisar la spec antes de implementar, que es donde está el 60% del valor. Y M2.5 cierra con una práctica guiada: escribir la spec de una feature que tú elijas, hasta dejarla ejecutable.

PARTE 2

Anatomía de una spec útil

M2.1 · Tres niveles de spec — intent, behavior, contract

Bloque 0 · Lo que toca abrir

Vienes del M1 con la idea de que escribir spec antes de implementar tiene sentido. Bien. La siguiente pregunta es operativa: ¿qué pongo dentro de la spec?

Aquí es donde mucha gente se atasca. Han leído sobre SDD, están convencidos del porqué, y abren un markdown en blanco para escribir la primera. Quince minutos después han borrado tres veces lo escrito porque no saben si es demasiado detallado, demasiado vago, si parece un brief de PM o un documento técnico de cinco páginas. La página en blanco gana, cierran el editor, vuelven al vibe coding.

El problema no es que escribir specs sea difícil. El problema es que nadie te ha enseñado la estructura. Una spec útil tiene **tres niveles**, y cada uno responde a una pregunta distinta. Si los confundes, sale rara. Si los conoces, escribirla se vuelve mecánico.

En estos veinte minutos vamos a ver los tres niveles uno por uno, con ejemplo concreto en cada uno, y al final una matriz de cuándo usar qué. Acabarás el submódulo sabiendo qué meter en una spec según el tipo de pieza que estés abordando.

Bloque 1 · Nivel 1, intent — el porqué

Intent es la parte más corta de la spec y la más fácil de saltarse. Resiste la tentación.

El intent responde a una pregunta: **¿para qué existe esta pieza?** No qué hace. Para qué. Quién la necesita, qué problema le resuelve, qué pasa si no la construyes. Una o dos frases. A veces tres si la pieza tiene contexto histórico que importe.

Ejemplo mal escrito (en realidad es behavior disfrazado):

El sistema debe permitir al usuario calcular la fecha de vencimiento de una factura introduciendo la fecha de emisión y el tipo de cliente.

Eso te dice qué hace la función. No te dice para qué existe. Cualquier dev que lea esto piensa "vale, lo construyo" sin saber qué pasa si lo construye mal — porque no hay nada que contradiga las decisiones implícitas. El intent estaba ausente.

Ejemplo bien escrito:

El equipo de contabilidad emite unas mil facturas al mes y calcula vencimientos a mano en Excel. Cada error en el cálculo provoca dos llamadas: la del cliente reclamando y la del comercial pidiéndole al de contabilidad que corrija. Necesitamos centralizar el cálculo para que el resto del sistema lo consuma desde un único sitio, y para que cuando cambien las condiciones comerciales no haya que avisar a tres departamentos.

Esa segunda versión no te dice cómo se calcula la fecha. Te dice **por qué hace falta una función para calcularla**. Y con ese contexto, las decisiones implícitas del M1 ya no son ambigüedades: tienes criterio para resolverlas. Si el problema es centralizar para evitar errores, sabes que la función tiene que ser estricta, no permisiva. Si el equipo emite mil facturas al mes, sabes que rendimiento no es crítico. Si hay clientes y comerciales involucrados, sabes que probablemente habrá distintos tipos de cliente.

El intent es la pieza que reduce las decisiones por defecto del modelo. Si solo le das behavior, decidirá cómo cumplir el behavior por su cuenta. Si le das intent, puede ajustar las decisiones a algo coherente con el problema real.

Una regla práctica: si tu intent se puede borrar sin que el lector pierda nada, es que estás escribiendo behavior y llamándolo intent. Recorta y reescribe.

Bloque 2 · Nivel 2, behavior — el qué

El behavior es la parte gorda de la spec en la mayoría de los casos. Aquí describes qué hace la pieza, ante qué entradas, con qué salidas, en qué casos.

Hay varias formas de escribirlo, y ninguna es la correcta para todos los casos.

Prosa estructurada. Para piezas sencillas, basta con dos o tres párrafos describiendo el comportamiento. Útil cuando la lógica es lineal y no hay muchas ramas.

La función recibe una factura con fecha de emisión, monto y datos del cliente. Calcula la fecha de vencimiento sumando 30 días naturales para clientes regulares o 60 para clientes VIP. Si el vencimiento cae en sábado, domingo o festivo del país del cliente, se mueve al siguiente día laboral.

Casos con tabla. Cuando hay varias ramas con entradas distintas, una tabla suele rendir mejor que la prosa.

Caso	Entrada	Resultado esperado
Regular en día laboral	Emisión lunes, cliente regular	Lunes + 30 días naturales
Regular cayendo en sábado	Emisión jueves, cliente regular	Lunes siguiente al sábado calculado
VIP	Cualquier emisión, cliente VIP	Emisión + 60 días, mismas reglas de ajuste
Festivo	Vencimiento calculado en festivo	Siguiente día laboral
Fin de mes excedido	Vencimiento > último día del mes siguiente	Trunca al último día del mes siguiente

Formato Given/When/Then. El clásico de BDD. Útil cuando la pieza tiene flujo claro y quieres compatibilidad con frameworks de testing tipo Cucumber.

Given una factura emitida el lunes 10 de mayo, con cliente regular, *when* se calcula su vencimiento, *then* la fecha de vencimiento es miércoles 9 de junio.

Combinación de los tres. Lo más común en la práctica. Prosa para la descripción general, tabla para los casos típicos, Given/When/Then para los casos límite o los más críticos.

Lo que el behavior NO debe ser: una traducción línea por línea del código que vas a escribir. Si tu behavior dice "iteramos sobre la lista de festivos y comprobamos si la fecha está dentro", eso es implementación, no comportamiento. El behavior describe qué se espera que pase, no cómo se va a implementar. La implementación es decisión del que codifica (humano o IA) dentro del marco que la spec define.

Una pregunta útil para calibrar el nivel de detalle: **¿podría otra persona implementar esto leyendo solo el behavior, sin preguntarme nada?** Si la respuesta es no, falta detalle. Si la respuesta es "sí, pero saldría idéntico al código que tienes en mente", probablemente has metido implementación.

Bloque 3 · Nivel 3, contract — el cómo de la interfaz

El contract es la parte más técnica y la que más se parece al código. Aquí fijas la interfaz: tipos exactos, signatures, esquemas, formatos.

No todas las specs necesitan contract. Si estás describiendo un cambio de UI o un comportamiento interno que no expone interfaz a nadie, el behavior es suficiente. Pero si tu pieza

es una función pública, un endpoint, una librería que otros consumen, o un evento que viaja por el sistema, el contract evita ambigüedades que el behavior no captura.

Las formas habituales:

Tipos del lenguaje. En .NET, escribir los tipos directamente en el spec con C# records funciona muy bien. Quien implementa tiene que cumplir esos tipos exactos.

```
public record Factura(  
    DateTimeOffset FechaEmision,    // UTC  
    long MontoTotalCentimos,        // céntimos, no decimales  
    Cliente Cliente  
);  
  
public record Cliente(  
    TipoCliente Tipo,  
    string PaisFacturacion           // ISO 3166-1 alpha-2  
);  
  
public enum TipoCliente { Regular, Vip }  
  
public static class FacturaService  
{  
    public static DateTimeOffset CalcularFechaVencimiento(Factura factura);  
    // devuelve fecha en Europe/Madrid, hora 00:00:00  
}  
  
public class InvalidInvoiceException : Exception  
{  
    public InvalidInvoiceException(string message) : base(message) { }  
}
```

JSON Schema. Cuando trabajas con APIs o eventos en JSON y quieres validación automática.

OpenAPI. Cuando defines un endpoint REST. En ASP.NET Core puedes generar la spec OpenAPI desde las anotaciones del controller, o escribirla a mano si prefieres "spec primero".

Records y validación con DataAnnotations. El idioma natural de .NET para representar contratos: records inmutables, propiedades tipadas, atributos de validación cuando aplica.

Lo importante del contract no es el formato — es la precisión. Cuando dices `PaisFacturacion: string`, no es suficiente: ¿qué tipo de string? ¿"España"? ¿"ES"? ¿"ESP"? Si el contract no lo especifica, tienes ambigüedad y el modelo va a elegir. Si lo especificas como `// ISO 3166-1 alpha-2`, la decisión está tomada.

Una historia rápida. Un equipo con el que trabajé descubrió tres semanas después de desplegar que su API devolvía importes en céntimos en unas rutas y en euros con decimales en otras. La spec del endpoint decía "el monto se devuelve como número". Ningún backend mintió. Cada uno eligió un formato razonable. Y el frontend se volvió loco haciendo conversiones inconsistentes. El bug se solucionó en quince minutos cuando todos acordaron `// importes siempre en céntimos,`

`integer` . La frase de catorce caracteres en un comentario habría ahorrado tres semanas de bugs intermitentes.

El contract es la parte de la spec donde un comentario de una línea puede valer más que un párrafo de behavior.

Bloque 4 · Cuándo basta con dos niveles (o uno)

Aquí viene el matiz importante: no todas las specs necesitan los tres niveles. Forzar los tres en piezas pequeñas es burocracia. Saltarse alguno en piezas grandes es deuda. El criterio:

Intent solo. Para tareas de investigación, exploraciones, "ver si X es viable". Una sola pieza de intent te ayuda a no perderte mientras exploras, pero behavior y contract no aplican porque todavía no sabes qué vas a construir. Una página de la libreta basta.

Intent + behavior. El caso típico. Refactor que cambia comportamiento sin cambiar interfaces, feature nueva que no expone API pública, cambio de UI con flujo nuevo, ajuste de lógica de negocio interna. La mayoría de las specs del día a día caben aquí.

Intent + behavior + contract. Cuando hay interfaz expuesta a otros. API pública, endpoint nuevo, función de librería compartida, evento que viaja entre servicios, formato de datos que otros consumen. Aquí saltarse el contract es el camino corto al "funciona en mi máquina pero el cliente del otro equipo está roto".

Behavior + contract sin intent. Esto es lo que no debes hacer. Si llegas a este punto, es que estás escribiendo documentación técnica, no spec. Sin intent, las decisiones de implementación no tienen ancla — y vuelve el problema del modelo eligiendo por su cuenta.

Contract solo. Tampoco. Un contract sin behavior es como un tipo sin documentación: dice qué entra y qué sale, pero no qué pasa dentro. Para algunos casos muy técnicos (definir un esquema de evento, por ejemplo) puede ser aceptable como spec parcial, pero hay que vincularlo a una spec mayor que sí tenga behavior e intent.

La regla informal: empieza siempre por intent. Si el intent no te sale claro, el resto no va a salir bien — recula y discútelo con quien sea relevante antes de seguir.

Bloque 5 · Los tres niveles encadenados, con el mismo ejemplo

Veamos cómo encajan los tres niveles para la spec de la factura del M1, ahora con cada parte en su sitio.

Intent

El equipo de contabilidad calcula vencimientos de factura a mano en Excel, con una media de cinco errores al mes que provocan reclamaciones del cliente y trabajo extra del comercial. Centralizar el cálculo en una función única evita que cada departamento aplique reglas distintas y permite cambiar las condiciones comerciales sin avisar a tres equipos.

Behavior

La función calcula la fecha de vencimiento de una factura sumando un plazo a la fecha de emisión.

El plazo por defecto es 30 días naturales para clientes regulares y 60 para clientes VIP. Si el vencimiento calculado cae en sábado, domingo o festivo del país del cliente, se mueve al siguiente día laboral. Tras el ajuste, si el vencimiento supera el último día del mes siguiente al de emisión, se trunca a ese último día.

Casos de prueba: - Factura regular emitida un lunes 10: vencimiento miércoles 9 del mes siguiente. - Factura regular cuyo vencimiento calculado cae en sábado 7: vencimiento lunes 9. - Factura VIP emitida un lunes 10: vencimiento al día equivalente 60 días naturales después, con los mismos ajustes. - Factura emitida con fecha futura: error `InvalidInvoiceError`.

Contract

```

public record Factura(
    DateTimeOffset FechaEmision,    // UTC
    long MontoTotalCentimos,        // céntimos, no decimales
    Cliente Cliente
);

public record Cliente(
    TipoCliente Tipo,
    string PaisFacturacion          // ISO 3166-1 alpha-2
);

public enum TipoCliente { Regular, Vip }

public static class FacturaService
{
    public static DateTimeOffset CalcularFechaVencimiento(Factura factura);
    // devuelve fecha en Europe/Madrid, hora 00:00:00
}

public class InvalidInvoiceException : Exception
{
    public InvalidInvoiceException(string message) : base(message) { }
}

```

Tres partes, una pieza. Si solo te llevas una imagen mental de cómo se ve una spec SDD útil, que sea esta. No hace falta más, no hace falta menos. Cinco a quince minutos de escritura, según lo que sepas ya del problema.

Bloque 6 · Cierre

Los tres niveles son la anatomía. Pero la anatomía sola no escribe specs útiles — el músculo y los reflejos los pones tú. Lo que hemos visto aquí es la estructura; lo que viene en los siguientes submódulos son las herramientas para rellenarla rápido y bien.

Tres ideas para llevarte:

Intent es lo que reduce la ambigüedad del modelo. Sin intent, las decisiones implícitas las toma él. Con intent, tienes criterio para resolverlas y la implementación sale alineada al problema real.

Behavior es lo gordo, pero ojo con confundirlo con implementación. La pregunta es "qué pasa", no "cómo lo hago". Si tu behavior se parece a pseudocódigo, has bajado un nivel demasiado.

Contract solo aparece cuando hay interfaz expuesta. No fuerces contract en piezas internas, pero no te saltes contract cuando el código va a ser consumido por otros — humanos o máquinas.

Glosario rápido del bloque

- **Intent:** el porqué de la pieza. Una o dos frases sobre el problema que resuelve. No qué hace, para qué existe.
- **Behavior:** el qué. Comportamiento esperado ante entradas concretas, con casos típicos y límite. Prosa, tabla, Given/When/Then o mezcla.
- **Contract:** el cómo de la interfaz. Tipos, esquemas, signatures, formatos. Solo cuando hay interfaz expuesta.
- **Caso típico vs caso límite:** caso típico es el flujo normal que cubre la mayoría de invocaciones; caso límite es lo que pasa en los bordes (entradas raras, condiciones extremas). Una spec útil cubre ambos.

Qué viene en M2.2

Formatos concretos: cuándo markdown plano basta, cuándo OpenAPI rinde más que la prosa, cuándo JSON Schema vale el rato extra. Vamos a ver tres ejemplos del mismo concepto escrito en tres formatos distintos para que veas las costuras y elijas con criterio en tu proyecto.

M2.2 · Formatos — markdown plano, OpenAPI, JSON Schema

Bloque 0 · El formato no es la spec, pero la condiciona

Si en M2.1 quedaba claro que una spec tiene tres niveles, queda por contestar una pregunta práctica: ¿en qué la escribes? Markdown, YAML, JSON, OpenAPI, una mezcla... Las opciones son muchas y la mayoría de los equipos eligen por inercia, no por criterio.

La inercia más común es escribir todo en lo que ya tienen montado. Si el equipo usa Notion, todo va a Notion. Si usa Confluence, todo va a Confluence. Si usa un repo de docs, todo va de markdown plano. Esa inercia funciona hasta que choca con algo que el formato no captura bien — un contract estricto en Notion es un infierno, una spec narrativa en JSON Schema es ilegible. Entonces alguien decide partir la spec, y entras en el antipatrón de los cinco lugares que vamos a ver al final.

En este submódulo vamos a tres formatos: markdown plano (la base), OpenAPI (para endpoints HTTP) y JSON Schema (para estructuras de datos y eventos). No son los únicos — hay Pydantic, hay Protocol Buffers, hay TypeSpec, hay Pkl — pero los tres elegidos cubren el 90% de los casos en proyectos reales y los principios que veas aquí se trasladan a los demás.

Bloque 1 · Markdown plano, la base por defecto

Si tuviera que recomendar un solo formato para arrancar SDD en un equipo que no tiene tradición, sería este sin pensarlo: markdown plano en el mismo repo que el código, en una carpeta `specs/`. Razones operativas:

Funciona en todos lados. Lo renderiza GitHub, GitLab, Bitbucket. Lo entiende cualquier editor. Lo entienden los modelos sin esfuerzo. No depende de ningún SaaS que pueda desaparecer ni de credenciales especiales para verlo.

Se versiona como el código. Diff legible en git, blame útil, PR con comentarios línea a línea. Eso ya cubre tres de los cinco rasgos de "primera clase" del M1.3.

No fuerza estructura prematura. Una spec en markdown puede ser de tres líneas para un bugfix y de cuarenta para una feature compleja, sin que el formato te obligue a rellenar campos que no aplican. En herramientas más estructuradas (Notion con plantillas, Jira con custom fields), acabas rellenando "N/A" en la mitad de los campos.

Acepta bloques de código. Si en algún momento necesitas meter un esquema técnico (records C#, JSON Schema, OpenAPI fragmentado), lo metes como bloque de código dentro del propio markdown. La spec sigue siendo legible.

La estructura típica de una spec en markdown:

```
# Spec: [nombre de la pieza]

## Intent
[1-2 párrafos sobre el porqué]

## Behavior

### Caso normal
[prosa]

### Casos límite
[tabla o lista]

## Contract
[bloques de código con tipos / firmas / esquemas]

## Criterios de aceptación
- [ ] [comprobables]

## Notas y decisiones
[opcional: por qué se descartó X, por qué no se hace Y]
```

Esa estructura es plantilla, no dogma. Si no tienes contract porque la pieza es interna, lo quitas. Si necesitas un apartado de "alternativas consideradas" porque la decisión fue dura, lo añades. La forma se adapta a la pieza, no al revés.

Una observación de gente que lleva tiempo con SDD: las specs en markdown plano tienden a estar más al día que las specs en herramientas sofisticadas. Suena raro pero es así. Cuando la spec vive a un `git pull` de distancia del código, actualizarla está en el camino de mínima fricción. Cuando vive en otra herramienta, actualizarla es un paso aparte que la gente olvida.

Bloque 2 · OpenAPI cuando hay endpoint HTTP

OpenAPI (también conocido como Swagger, aunque ya no se llama así oficialmente) es el formato estándar para describir APIs HTTP. Si tu pieza es un endpoint REST, OpenAPI rinde donde el markdown se queda corto.

¿Qué te da OpenAPI que markdown no te da?

Tipado estricto y verificable. Defines tipos exactos para cada parámetro, body, query string, header. Una herramienta puede validar que tu implementación los respeta. Una herramienta

puede validar que el cliente los respeta. El contract se convierte en algo comprobable automáticamente.

Generación de código. Desde un fichero OpenAPI puedes generar clientes en C#, TypeScript, Python, Go, lo que sea. En .NET, herramientas como `NSwag` o `Kiota` generan los clientes con buena ergonomía. Servidores también. No es magia — el código generado tiene sus problemas — pero como punto de partida acelera el setup.

Documentación interactiva. Swagger UI, Redoc, Scalar y compañía toman tu OpenAPI y te dan un explorador navegable de la API. Si tu equipo consume la API o si la abres a clientes externos, esto vale oro.

Estándar de la industria. Si llega un dev nuevo o un equipo socio, lo entiende sin curva de aprendizaje. Es lengua común.

Cuándo NO usar OpenAPI:

- Cuando la pieza no es un endpoint HTTP. Si es una función interna, OpenAPI no aporta.
- Cuando el endpoint es trivial y vive un mes. Para un endpoint de demo o de prototipo, el ceremonial de OpenAPI no compensa.
- Cuando el equipo no tiene cultura de mantenerlo. Un OpenAPI desactualizado es peor que no tenerlo — engaña.

Lo importante operativamente: el fichero OpenAPI vive en el repo del código, junto al markdown de la spec, no en una herramienta externa. La spec en markdown referencia al fichero OpenAPI cuando llega al contract. Algo así:

Contract

El endpoint sigue la spec OpenAPI en ``specs/facturas.openapi.yaml``.
Resumen de las decisiones clave del contrato:

- Importes en céntimos, integer, sin decimales.
- Códigos de país en ISO 3166-1 alpha-2.
- Fechas siempre en formato ISO 8601 con zona UTC.
- Errores siguen el esquema RFC 7807 (Problem Details for HTTP APIs).

El markdown sigue siendo el documento de entrada del lector humano; el OpenAPI es la fuente única para el contract técnico y para herramientas. Coexisten sin pisarse.

Bloque 3 · JSON Schema cuando hay estructura de datos

JSON Schema vive un escalón más abajo que OpenAPI. Donde OpenAPI describe un endpoint completo (rutas, métodos, parámetros, respuestas), JSON Schema describe la forma de un

documento JSON: qué campos tiene, de qué tipos, qué es opcional, qué patrones cumple cada campo.

Enaja como contract en casos como estos:

Eventos entre servicios. Si tu sistema publica eventos en una cola y otros servicios los consumen, JSON Schema define qué forma tiene cada evento. Versiones del esquema = versiones del evento. Si rompes el contrato, los consumidores lo saben.

Configuraciones. Cuando tu sistema lee configuración desde JSON o YAML, JSON Schema valida que la configuración es correcta antes de arrancar. Mejor que descubrir a las dos horas que un campo estaba mal escrito.

Persistencia documental. Si guardas documentos en MongoDB, Elasticsearch, DynamoDB o cualquier base no relacional, JSON Schema fija qué forma tienen los documentos almacenados.

Validación de inputs. Antes de procesar un input externo (webhook entrante, importación de fichero, etc.), validas contra JSON Schema y rechazas los inputs malformados con un mensaje claro.

Igual que con OpenAPI, JSON Schema convive con el markdown: el .json del esquema vive en el repo, el markdown de la spec referencia el archivo y describe en prosa lo que el esquema fija.

Contract

Los eventos ``factura.creada`` siguen el esquema en ``specs/eventos/factura-creada.schema.json``.

Campos clave:

- ``idFactura``: UUID v4, generado por el emisor.
- ``montoCentimos``: integer ≥ 0 .
- ``clienteId``: UUID v4 del cliente en el dominio "clientes".

Versionado: el esquema lleva campo ``version`` (semver). Cambios menores mantienen la versión mayor; cambios breaking exigen versión mayor nueva y coexistencia temporal.

Notas de calibración: JSON Schema tiene fama de ser verboso. Para esquemas pequeños cabe en quince líneas; para esquemas grandes (con refs, enums largos, allOf y oneOf) puede crecer mucho. No pasa nada — lo importante es que sea legible para el modelo y para herramientas, no para que un humano lo lea de cabo a rabo. El humano lee la prosa en markdown; el modelo y los validadores leen el esquema.

Bloque 4 · Mezclar formatos sin que el repo sea un caos

La pregunta práctica que aparece en cualquier equipo: ¿cómo organizo las specs cuando uso los tres formatos en distinto grado?

Una estructura que funciona bien en proyectos medianos:

```
specs/  
  README.md                # índice de specs activas  
  features/  
    calcular-vencimiento.md  
    importar-csv-clientes.md  
  endpoints/  
    facturas.openapi.yaml  
    facturas.md            # prosa que acompaña al openapi  
  eventos/  
    factura-creada.schema.json  
    factura-creada.md       # prosa que acompaña al schema  
  archived/  
    (specs cerradas)
```

Tres principios detrás de esta estructura:

Cada spec tiene un .md de entrada. Aunque su contract viva en un .yaml o .json aparte, hay siempre un .md que explica el intent y el behavior. El .md es la puerta de entrada para humanos; el .yaml/.json es el contract técnico para máquinas.

Los archivos viven juntos por tema, no por formato. Mejor `endpoints/facturas.md` + `endpoints/facturas.openapi.yaml` que dos carpetas `markdowns/` y `openapis/`. Cuando alguien busca lo de facturas, lo encuentra todo en el mismo sitio.

Las specs cerradas se mueven, no se borran. Las que ya están implementadas y validadas pasan a `archived/` para no ensuciar la lista de activas. Siguen consultables como historia.

Un detalle pequeño pero importante: el `specs/README.md` que sirve de índice se mantiene como lista plana de specs activas con su estado. Cuando hay diez specs vivas a la vez, ese índice te ahorra una hora a la semana. Cuando hay dos, no hace falta. Empieza pequeño.

Bloque 5 · El antipatrón de los cinco lugares

Y el antipatrón clásico que casi todos los equipos descubren tarde. La spec inicial vive en Notion. La descripción de la API la genera el backend en Swagger UI a partir de anotaciones en el código. El JSON Schema de los eventos vive en un repo aparte llamado `event-contracts`. La documentación de uso para los consumidores vive en un README en GitHub. Y cuando hay una decisión de diseño dura, alguien escribe un ADR en otro sitio.

Cinco lugares. Cinco fuentes. Cinco probabilidades de desincronización. Cinco sitios que cualquier nuevo del equipo tiene que buscar para entender lo que hace una pieza.

¿Por qué pasa esto? Porque cada herramienta es buena para algo concreto. Notion escribe documentos prosa de maravilla. Swagger UI explora APIs como nadie. JSON Schema en repo separado parece organizar bien los contratos compartidos. Cada decisión, en su momento, fue razonable.

El problema es la suma. Una persona nueva tiene que aprender cinco herramientas, cinco convenciones, cinco sitios donde algo puede haberse actualizado. Y como nadie tiene ganas de actualizar cinco sitios cada vez que algo cambia, algunos se quedan parados. Y a partir de ahí no puedes fiarte de ninguno sin contrastar con los demás.

La salida es centralizar todo lo que se pueda en el repo del código. El markdown vive ahí. El OpenAPI vive ahí. El JSON Schema vive ahí. Si la herramienta externa rinde (Swagger UI para explorar, una página de docs renderizada para los consumidores), se genera **desde** los archivos del repo, nunca al revés. La fuente única de verdad está en git; las renderizaciones son derivadas.

Hay una excepción legítima: cuando hay un equipo de producto que escribe en una herramienta visual y luego pasa la spec a desarrollo. En ese caso, lo razonable es que el documento del producto sea el brief inicial, y que la spec en el repo lo recoja, lo ajuste con detalle técnico y se convierta en la fuente única a partir de ese momento. El documento de producto ya no se actualiza unilateralmente.

Bloque 6 · Cierre

Tres ideas para llevarte.

Markdown plano es la base por defecto. Empieza ahí. Solo si la pieza tiene contract estricto que se beneficia de tipado y herramientas, añade OpenAPI o JSON Schema como complemento — nunca como sustituto del markdown.

Los formatos coexisten en el mismo repo, en archivos enlazados. El markdown es la entrada; los esquemas técnicos son el contract. No partas la spec entre cinco herramientas distintas, pero tampoco fuerces todo en un solo formato.

El antipatrón a evitar es la dispersión: cuando una pieza tiene su intent en una herramienta, su behavior en otra, su contract en otra y su documentación en otra, has perdido la propiedad de "fuente única de verdad". Y sin fuente única, vuelve la divergencia entre lo que dice cada documento y lo que hace el código.

Glosario rápido del bloque

- **OpenAPI:** estándar para describir APIs HTTP (antes Swagger). Define rutas, métodos, parámetros, respuestas, errores. Útil como contract de endpoint.
- **JSON Schema:** estándar para describir la forma de un documento JSON. Útil para eventos, configuraciones, persistencia documental y validación de inputs.
- **Fuente única de verdad:** ubicación canónica de la información. Si hay dos copias en sitios distintos, una se desincroniza con el tiempo — y nadie sabe cuál es la buena.
- **ADR (Architecture Decision Record):** documento corto que registra una decisión de arquitectura: contexto, opciones consideradas, decisión tomada, consecuencias. Vive junto al código.

Qué viene en M2.3

Plantillas reutilizables. Vamos a empezar con la plantilla SDD "típica" que circula por internet (la que sale al buscar "SDD template Claude Code" en Google), a desarmarla con criterio, y a proponer tres plantillas propias: feature nueva, bugfix y cambio arquitectónico. Cada una con su estructura y su ámbito.

M2.3 · Plantillas reutilizables

Bloque 0 · Por qué tres plantillas y no una

Una pregunta razonable: si la spec tiene los tres niveles que vimos en M2.1, ¿por qué no basta con una sola plantilla que los incluya y ya está?

Porque las piezas que vas a especificar no son todas la misma cosa. Una feature nueva responde a "¿qué construir y para quién?". Un bugfix responde a "¿qué está roto, por qué, y cómo sabemos que ya no lo está?". Un cambio arquitectónico responde a "¿de dónde a dónde nos movemos y cómo evitamos romper en el camino?". Forzar las tres en la misma plantilla acaba en dos resultados: o la plantilla es tan genérica que la mitad de los campos quedan vacíos, o tan exhaustiva que escribirla cuesta más que implementar.

Lo que vamos a hacer en estos veinte minutos: primero mirar la plantilla "típica" que la red ofrece como SDD para Claude Code, ver qué hace bien y dónde se queda corta. Luego, las tres plantillas que el curso propone como sustitutas. Cada una con su estructura y su criterio de cuándo usarla.

Las tres plantillas están en el repositorio del curso, en `recursos/plantillas/`, listas para copiar.

Bloque 1 · La plantilla "típica" que circula por internet

Si buscas "SDD template Claude Code" o equivalente en cualquier buscador, vas a llegar a una plantilla YAML con cinco secciones: SPECIFY, PLAN, TASKS, IMPLEMENT, VALIDATE. La hemos archivado en `recursos/plantilla-sdd-tipica-encontrada-online.md` precisamente porque es lo que el alumno se va a encontrar antes que cualquier otra cosa.

La plantilla, resumida:

```

# === 1. SPECIFY ===
feature: " ... "
inputs: [ ... ]
outputs: [ ... ]
constraints: [ ... ]
edge_cases: [ ... ]
acceptance_criteria: [ ... ]

# === 2. PLAN ===
architecture: " ... "
libraries: [ ... ]
data_structures: [ ... ]
algorithms: [ ... ]

# === 3. TASKS ===
tasks: [ ... ]

# === 4. IMPLEMENT ===
# Aquí Claude Code generará el código.

# === 5. VALIDATE ===
tests:
  unit_tests: [ ... ]
  integration_tests: [ ... ]

```

A primera vista parece sensata. Está estructurada, tiene casos límite, tiene criterios de aceptación, tiene tests. Quien la rellena ya hace más que el 90% de la gente que usa Claude Code sin método.

Pero hay un detalle importante: tiene seis carencias serias que la hacen insuficiente para SDD en serio. Si la usas tal cual durante semanas, los problemas que vamos a listar a continuación van a aparecer en tu proyecto sin que sepas por qué.

Bloque 2 · Las seis carencias de la plantilla típica

Falta intent, el porqué. La plantilla pregunta el "feature" y la "description" como una sola frase. Eso te dice qué construye, no para qué existe. Ya vimos en M2.1 que sin intent el modelo toma las decisiones implícitas por su cuenta y casi siempre acierta una y se equivoca cinco. La plantilla típica te empuja directamente al problema.

El PLAN está relleno a mano antes de pedir nada. La sección dice `architecture: " ... "` y te invita a escribir tú la arquitectura. Pero el plan es algo que Claude Code te ayuda a construir desde la spec, iterando hasta dejarlo bien. Si lo escribes tú a pelo, has decidido la arquitectura solo en tu cabeza, sin contraste. Es vibe coding con plantilla puesta delante.

IMPLEMENT está vacío con un comentario "aquí Claude Code generará". No hay metodología, no hay validación contra spec durante la implementación, no hay hooks. Es "spec →

click → código → cruzar los dedos". Vamos a ver en M3 y M4 que esa parte tiene más sustancia que un comentario.

No hay revisión de la spec antes de implementar. La plantilla asume que rellenas los campos y la spec ya está lista para ejecutar. En la práctica, el 60% del valor de SDD viene de iterar la spec una o dos veces antes de empezar: detectar ambigüedades, contradicciones, criterios de aceptación que no son verificables. La plantilla no contempla ese paso.

Una sola plantilla para todo. Si la pieza es una feature nueva, los campos encajan razonablemente. Si es un bugfix, la mitad sobra y falta lo importante (cuál era el síntoma original, qué hipótesis tienes sobre la causa). Si es un cambio arquitectónico, no hay sitio para "estado actual" ni para "plan de migración".

Trata SDD como flujo lineal. SPECIFY → PLAN → TASKS → IMPLEMENT → VALIDATE sugiere una vez y se acabó. SDD real es un bucle: spec → plan → código → feedback → spec ajustada. El cambio de requisito siempre llega antes o después.

Resumiendo: la plantilla es un punto de arranque digno para empezar, pero no es la herramienta final. Es a SDD lo que el "Hola mundo" es a un proyecto de producción.

Bloque 3 · Plantilla A — Feature nueva

La feature nueva es el caso más común. Algo que no existía en el sistema y se va a construir. Estructura propuesta:

```

# Spec: [nombre de la feature]

## Intent
[1-2 párrafos sobre el problema que resuelve.
Quién la necesita. Qué pasa si no se construye.
Contexto histórico si lo hay.]

## Behavior

### Caso normal
[Prosa o tabla del comportamiento esperado en el flujo principal.]

### Casos límite
[Entradas raras, condiciones de error, comportamientos en bordes.]

## Contract
[Si aplica. Tipos, signatures, esquemas. En markdown o
linkado a OpenAPI / JSON Schema externo.]

## Criterios de aceptación
- [ ] [Lista de condiciones comprobables, una a una.
Sin frases vagas tipo "el sistema debe ser rápido".]

## Fuera de alcance
[Lo que NO va a hacer esta feature, para que no haya dudas.]

## Notas y decisiones
[Por qué se eligió X sobre Y. Alternativas descartadas y motivo.
Solo si la decisión fue dura.]

```

Tres aspectos importantes:

El campo **Fuera de alcance** es de los más útiles y de los más olvidados. Cuando dices explícitamente "esta feature NO va a manejar X", el modelo no va a intentar manejar X por su cuenta para parecer útil. Cuando no lo dices, casi siempre lo intenta y se cuela código que no querías.

Los **criterios de aceptación** deben ser comprobables. "El sistema debe rendir bien" no es criterio; no se puede comprobar. "El endpoint responde en menos de 200 ms en el percentil 95 bajo carga de 100 rps" sí lo es. Si tu criterio no se puede convertir en test (humano o automático), no es criterio: es deseo.

La sección **Notas y decisiones** es opcional pero tiene valor enorme cuando la decisión fue dura. Cuando alguien lea esta spec dentro de seis meses, esa sección le dice por qué no usar la alternativa que parece obvia. Sin ella, la decisión se pierde y alguien la reabre.

Bloque 4 · Plantilla B — Bugfix

El bugfix es la pieza que más se descuida en SDD porque parece que no merece spec. Y muchas veces tiene razón: para un bug obvio de una línea, el PR es suficiente y la spec sería ceremonial. Pero para bugs que requieren más de una hora de trabajo o que tocan varios sitios, una mini-spec dentro del PR rinde mucho.

Estructura propuesta:

```
# Bugfix: [resumen del síntoma]

## Síntoma
[Qué se observa que va mal. Mensaje de error literal si lo hay,
 caso de reproducción, entornos donde se ve.]

## Reproducción
[Pasos exactos para reproducir. Cuanto más mecánicos, mejor.
 Idealmente: un test que falla y lo demuestra.]

## Causa raíz (hipótesis)
[Por qué creemos que pasa. Si todavía es hipótesis, decirlo.
 Si está confirmada, decirlo también.]

## Solución propuesta
[Qué se va a tocar y por qué. NO el código exacto;
 la decisión de diseño.]

## Criterio de "arreglado"
- [ ] El test de reproducción pasa.
- [ ] [Otros tests de regresión que cubren el área.]
- [ ] [Comprobación manual si la hay.]

## Riesgo de regresión
[Qué otras partes del sistema podrían verse afectadas
 por el cambio. Si el riesgo es alto, justificar la elección.]
```

Tres notas:

La separación entre **síntoma** y **causa raíz** es clave. Cuando alguien lee el bug seis meses después, quiere saber qué pasaba (síntoma) y por qué pasaba (causa). Si los mezclas, la spec sirve menos.

El **criterio de "arreglado"** debe incluir tests de regresión, no solo el test que reproduce el bug. Una regresión es lo que pasa cuando arreglas algo y rompes otra cosa cercana; el test que arregla el bug original no la detecta por sí solo. Hay que añadir cobertura del área tocada.

El campo **riesgo de regresión** obliga a pensar antes de tocar. Si el cambio toca código central que muchas partes consumen, hay que pararse y considerarlo. Si toca código aislado, el riesgo es bajo. Hacerlo explícito evita sorpresas.

Bloque 5 · Plantilla C — Cambio arquitectónico

El cambio arquitectónico es la pieza más rara y la más cara de equivocarse. Migración de stack, partición de un servicio, cambio de base de datos, sustitución de una librería central. Cosas que tocan muchos sitios y duran semanas. Aquí la spec tiene que cubrir un alcance más amplio.

Estructura propuesta:

```
# Cambio arquitectónico: [resumen]

## Intent
[Por qué este cambio. Qué problema del estado actual obliga
a tocarlo. Qué se gana al hacerlo.]

## Estado actual
[Descripción del sistema tal como está hoy. Diagramas si ayudan.
Puntos de dolor concretos que motivan el cambio.]

## Estado deseado
[Cómo queremos que esté el sistema al terminar. Cómo se ve la
arquitectura después.]

## Plan de transición
[Pasos para ir de A a B. Incremental siempre que se pueda;
big bang solo cuando no haya alternativa. Cada paso debe
dejar el sistema funcional, no roto a la mitad.]

## Criterios de éxito
- [ ] [Comprobables al terminar la transición.]
- [ ] [Métrica clave que debe mejorar / mantenerse.]

## Criterios de rollback
[Bajo qué condiciones se aborta la migración y se vuelve atrás.
Cómo se hace el rollback técnicamente.]

## Riesgos identificados
[Lista corta de riesgos serios y mitigaciones planeadas.]
```

Cinco notas sobre esta plantilla:

El bloque **estado actual** parece redundante porque "ya lo sabemos". En realidad, cuando empiezas a escribirlo, descubres que distintas personas del equipo tienen ideas distintas sobre cómo está la cosa hoy. Ponerlo por escrito alinea al equipo antes de discutir el destino.

El **plan de transición** es lo más difícil de redactar y lo más útil. Aquí es donde se decide si la migración es viable o no. Una migración que requiere "y entonces apagamos todo el viernes por la noche" es señal de que el plan necesita trabajo.

Los **criterios de éxito** deben incluir métricas, no solo "todo migrado". Si la migración es para mejorar rendimiento, incluir la métrica de rendimiento antes y después. Si es para reducir coste, incluir el coste. Sin métricas, la migración nunca se "termina" del todo.

Los **criterios de rollback** son la parte que a nadie le gusta escribir y que más vale tener. Antes de empezar la migración, hay que saber bajo qué condiciones se aborta. Si no las defines antes, las defines en pánico a las dos de la mañana con producción caída.

Los **riesgos identificados** no son una lista exhaustiva de cosas que pueden ir mal; es la lista corta de las que de verdad pueden hundir la migración. Tres o cuatro. Si pones quince, ninguno se mira con suficiente atención.

Bloque 6 · Cuándo usar cuál (y cuándo ninguna)

Una matriz rápida para decidir qué plantilla aplicar:

Tipo de pieza	Plantilla
Feature nueva, > 1 hora de trabajo	A — feature
Bugfix complejo o de causa dudosa	B — bugfix
Bugfix obvio de una línea	Ninguna; basta con el PR
Refactor interno sin cambio de interfaz	A simplificada (sin contract)
Cambio de stack, partición de servicios, migración	C — arquitectónico
Tarea exploratoria, prueba de concepto	A reducida (solo intent + criterios de éxito)
Cambio de UI con flujo nuevo	A — feature
Cambio de configuración o despliegue	Ninguna o mini-spec en el PR

La regla informal: si tienes dudas sobre qué plantilla aplicar, empieza por la A. Es la más versátil y se adapta a la mayoría de los casos. Las otras dos son para los casos donde la A se quedaría coja.

Y un recordatorio del M1: no toda pieza necesita spec. Si el trabajo cabe en treinta minutos y no expone interfaz, probablemente no la necesita. Forzar plantillas en piezas pequeñas hace que el equipo deje de escribirlas en piezas grandes "porque siempre las pedimos para todo y nadie las lee".

Bloque 7 · Cierre

Una idea para llevarte por encima de las plantillas concretas: el formato no es lo importante. Lo importante es que la plantilla que uses te obligue a contestar las preguntas adecuadas para el tipo de pieza que estás abordando.

Una feature pregunta para quién y qué se entrega. Un bugfix pregunta qué se observa y cómo sabemos que ya no se observa. Un cambio arquitectónico pregunta de dónde a dónde y cómo evitamos quedarnos a medias. Las plantillas son ayudas de memoria para no saltarte preguntas; no son fines en sí mismas.

Si tu equipo termina haciendo SDD a fondo, lo más probable es que en seis meses tengáis vuestras propias variantes de estas tres plantillas adaptadas a vuestro dominio. Es lo natural. Las plantillas del curso son un punto de arranque solvente, no un dogma.

Glosario rápido del bloque

- **Intent** (en plantilla feature): párrafo sobre el problema que la feature resuelve. Una o dos frases.
- **Causa raíz** (en plantilla bugfix): la razón última del bug, no el síntoma. Puede ser hipótesis o estar confirmada.
- **Estado actual / deseado** (en plantilla arquitectónica): descripción del sistema antes y después del cambio. Pone por escrito lo que distintas personas del equipo entienden distinto.
- **Criterio de rollback**: condición bajo la cual se aborta la migración y se vuelve atrás. Se decide antes de empezar, no en mitad del problema.

Qué viene en M2.4

La revisión de la spec antes de implementar. Cómo pedirle a Claude Code que te revise una spec (o cómo se la pasas a otro humano del equipo) y los cuatro huecos típicos que aparecen en casi todas las primeras versiones: ambigüedad, contradicción, alcance abierto y criterio de aceptación ausente. Aquí es donde está la mayor parte del valor de SDD; saltárselo es perder el 60% del beneficio.

M2.4 · Revisión de specs — los cuatro huecos

Bloque 0 · Lo que casi todo el mundo se salta

He visto a equipos que adoptan SDD con entusiasmo, montan sus plantillas en `specs/`, empiezan a rellenarlas para cada feature... y siguen teniendo los mismos problemas de antes. Bugs raros, refactors caros, cosas que el modelo entendió a su manera.

Cuando reviso lo que están haciendo, casi siempre es lo mismo: escriben la spec, la dejan en el PR, y pasan directos a "Claude, implementa esto". Sin un paso intermedio. La spec se da por buena por el hecho de existir.

Y aquí está la clave incómoda: **una spec escrita y no revisada apenas vale más que no tener spec**. El acto de escribirla te obliga a pensar, sí, pero la primera versión que sale de tu cabeza está casi siempre incompleta o tiene contradicciones que no ves desde dentro. La revisión es lo que convierte una spec borrador en una spec ejecutable.

En estos quince minutos vamos a ver los cuatro huecos típicos que aparecen en cualquier primera versión, cómo encontrarlos (con Claude Code o con un humano del equipo), y cuándo parar de iterar — porque tampoco se trata de revisar la spec eternamente.

Bloque 1 · Cómo se siente una spec antes y después de revisar

Para que la diferencia quede clara, partamos del caso de la factura del M1 y M2.1. Imagínate la primera versión que escribes en cinco minutos:

Calcular fecha de vencimiento de factura. La fecha base es la de emisión más 30 días. Si cae en fin de semana, se mueve al lunes.

Eso es lo que sale del cerebro a la primera. Suena razonable, está estructurada, parece spec. Y tiene cuatro problemas que vamos a ver en el bloque siguiente.

Después de una revisión de cinco minutos, la spec se convierte en algo así:

Calcular fecha de vencimiento de factura.

Cliente regular: 30 días naturales. Cliente VIP: 60 días naturales. (Diferenciado en behavior 1.)

Si el vencimiento calculado cae en sábado o domingo, se mueve al lunes siguiente. Si cae en festivo del país del cliente, también. (Comportamiento detallado en behavior 2 con tabla de casos.)

Si tras los ajustes el vencimiento queda más allá del último día del mes siguiente, se trunca al último día del mes siguiente. (Decisión cerrada con contabilidad el 12 de marzo; ver nota al final.)

No procesa facturas con fechaEmision en el futuro: lanza InvalidInvoiceError.

Criterios de aceptación específicos para los cinco casos.

La segunda versión no es "más larga porque sí". Es más larga porque ha cerrado las ambigüedades que la primera dejaba abiertas. Y esas ambigüedades, sin la revisión, las habría cerrado el modelo a su manera al implementar.

Bloque 2 · Los cuatro huecos típicos

Cada vez que reviso una spec en una primera versión, los problemas se reducen a cuatro categorías. Aprende a buscarlos uno a uno y vas a cazar el 90% de lo que falla en cualquier spec del día a día.

Hueco 1: ambigüedad. Una palabra o una frase que admite más de una lectura razonable. La spec dice "30 días" — ¿naturales o hábiles? "El sistema responde rápido" — ¿qué es rápido? "Los datos se persisten" — ¿en qué momento, en qué orden? Las ambigüedades son fáciles de detectar si las buscas activamente y muy fáciles de pasar por alto si lees por encima. El truco es leer la spec preguntándote en cada frase: "¿hay otra manera razonable de interpretar esto?". Si la hay, ahí tienes ambigüedad.

Hueco 2: contradicción. Dos partes de la spec dicen cosas que no se pueden cumplir a la vez, o una parte dice algo que choca con otra. Pasa más de lo que parece, sobre todo en specs medianas o grandes. Un caso típico: el behavior dice que la respuesta es rápida (<100 ms), y el contract dice que se consulta un servicio externo (que tarda 200-500 ms). No puede ser ambas. Algo tiene que cambiar — el SLA, el contrato, o la arquitectura.

Hueco 3: alcance abierto. La spec dice qué hacer pero no qué NO hacer. Cuando el alcance no está limitado, el modelo va a expandirlo por su cuenta para parecer útil. Si la spec dice "función para procesar pedidos", el modelo puede acabar implementando validación, persistencia, envío de email de confirmación y un cliente HTTP "por si acaso". La forma de cerrar el alcance: una sección **Fuera de alcance** con tres o cuatro elementos explícitos. "Esta función NO valida el

inventario; eso lo hace [otro componente]. NO envía notificaciones; eso lo hace [otro flujo]. NO persiste; eso lo hace [otra capa]."

Hueco 4: criterio de aceptación ausente o no comprobable. La spec describe el comportamiento pero no define qué cuenta como "hecho". O lo define en términos vagos que no se pueden verificar. "El sistema debe funcionar bien" no es criterio. "El endpoint responde con 200 y body válido para los cuatro casos de la tabla; con 4xx y mensaje claro para los tres casos de error definidos" sí lo es. Si tu criterio de aceptación no se puede convertir en una checklist comprobable, no es criterio: es deseo.

Los cuatro huecos no son independientes. Una contradicción suele esconder una ambigüedad ("rápido" vs "consulta externa" — la palabra "rápido" estaba ambigua). Un alcance abierto suele llevar a criterios de aceptación imposibles ("la función procesa pedidos correctamente"). Cazar uno a menudo te lleva a los demás.

Bloque 3 · Cómo pedirle a Claude Code que revise tu spec

El modelo es bueno detectando los cuatro huecos cuando se lo pides explícitamente. No tan bueno cuando le dices "revisa esta spec" sin más. La diferencia entre las dos peticiones es enorme.

Una prompt que funciona bien para revisión:

Eres revisor de specs SDD. Lee la spec que sigue y dime qué huecos detectas, organizados en cuatro categorías:

1. Ambigüedades: frases que admiten más de una interpretación razonable. Para cada una, indica la frase exacta y las interpretaciones posibles.
2. Contradicciones: dos partes de la spec que no pueden cumplirse a la vez, o que chocan entre sí. Para cada una, indica las dos partes implicadas.
3. Alcance abierto: cosas que la spec NO dice si están dentro o fuera del scope. Sugiere qué debería declararse explícitamente fuera.
4. Criterios de aceptación: revisa si son comprobables. Para cada criterio, indica cómo se podría verificar; si no se puede convertir en test, márcalo como insuficiente.

NO me corrijas la spec. Solo dime qué huecos hay para que yo decida cómo cerrarlos.

Spec:

[pega aquí el contenido del .md]

Tres detalles importantes de esa prompt:

Le das el rol explícito (revisor de specs SDD) en lugar de pedirle genéricamente "que revise". El rol acota lo que va a buscar.

Le das las cuatro categorías y le pides que organice por ellas. Sin estructura, mezcla cosas y te da una lista plana que cuesta accionar.

Le dices que NO te corrija la spec. Esto es clave. Si le pides revisión y corrección a la vez, te devuelve la spec reescrita y pierdes el control. Quieres detectar problemas y decidir tú cómo resolverlos. La revisión y la corrección son fases distintas.

El resultado suele ser una lista de 10 a 20 puntos, según el tamaño de la spec. La mayoría son ajustables en uno o dos minutos. Algunos te obligan a parar y discutir con alguien antes de seguir.

Bloque 4 · La revisión humana, que no se sustituye

Aunque la revisión automatizada con Claude Code cubre mucho terreno, hay un tipo de huecos que el modelo no puede detectar bien: los que vienen del contexto que solo tiene el equipo.

Si tu spec asume que el cliente acepta esperar 24 horas para una respuesta, pero el equipo de soporte sabe que esos casos generan tres llamadas al call center, eso no lo va a detectar el modelo. No tiene contexto operativo. Lo detecta la persona del equipo de soporte cuando le pasas la spec a revisar.

La regla práctica: revisión automática primero (cubre los huecos formales) y revisión humana después (cubre los huecos de contexto). En ese orden. Si haces la humana primero, le pides a la persona que detecte cosas que el modelo habría detectado en cinco segundos, y aprovechas peor su atención.

Para la revisión humana, no abras una reunión. Abre un PR de la spec y pide review como si fuera código. Tres ventajas:

La conversación queda registrada. Los comentarios en el PR son histórico. Si dentro de tres meses alguien pregunta "¿por qué decidimos X?", la respuesta está en el PR.

El revisor lo hace cuando puede. No fuerzas una reunión sincrónica. Y el ratio comentario-lectura es mucho más alto en escrito que en reunión, donde el 70% del tiempo se va en alguien repitiendo lo que ya está en pantalla.

Cualquiera del equipo puede sumarse. Si pasa por ahí alguien que conoce el dominio y ve un detalle, lo comenta. En una reunión convocada para tres personas, ese cuarto que tenía algo útil que decir no estaba invitado.

Bloque 5 · Cuándo parar de iterar

La revisión puede convertirse en una bestia que no termina. Cada vez que cierras un hueco, abres dos más pequeños. Si revisas la spec ocho veces, probablemente acabes con una spec que parece un tratado y que el equipo deja de leer.

La regla práctica: para de iterar cuando los huecos que aparecen en la siguiente revisión son ya cuestiones de matiz, no de fondo. Tres señales concretas de que la spec está "ejecutable":

Los criterios de aceptación se pueden convertir en una lista de checks comprobables sin negociar. Si lees los criterios y te imaginas escribiendo los tests, no hay frases del tipo "depende de...". Hay condiciones binarias.

El alcance está cerrado por ambos lados. Sabes qué hace y qué NO hace la pieza. Si alguien te preguntara "¿esta función también se encarga de X?", tu respuesta sería "está en el fuera de alcance, no" o "está en el behavior, sí". Ninguna respuesta sería "ah, no lo había pensado".

Otra persona del equipo lee la spec y no te tiene que preguntar nada para entenderla. Esta es la prueba más dura. Si tu compañera lee la spec y te manda diez preguntas, todavía falta trabajo. Si lee la spec y te dice "vale, entendido, voy a implementar", la spec está lista.

Una calibración pragmática: para piezas medianas, dos rondas de revisión (una automática + una humana) suelen bastar. Para piezas grandes o críticas, tres. Más de tres rondas suele ser señal de que el problema no es la spec — es que hay desacuerdo de fondo en el equipo sobre qué construir, y ningún número de iteraciones lo va a resolver. En esos casos, para de iterar y abre la conversación de fondo aparte.

Bloque 6 · Cierre

Tres ideas para llevarte.

La spec sin revisar tiene la mitad del valor de la spec revisada. Saltarse la revisión es el atajo más caro de SDD. Cinco minutos extra al inicio te ahorran horas de re-trabajo.

Los cuatro huecos a buscar son sistemáticos: ambigüedad, contradicción, alcance abierto, criterio de aceptación ausente. Aprende el patrón y cázalos rápido en cualquier spec.

Combina revisión automática (con Claude Code) y revisión humana (en PR). La automática cubre los huecos formales en segundos; la humana cubre los huecos de contexto que solo conoce el equipo. Las dos en ese orden.

Glosario rápido del bloque

- **Ambigüedad:** frase de la spec que admite más de una interpretación razonable. Se cierra reescribiendo con precisión.
- **Contradicción:** dos partes de la spec que no pueden cumplirse a la vez. Se cierra eligiendo una o renegociando alguna.
- **Alcance abierto:** ausencia de declaración explícita de lo que la pieza NO hace. Se cierra con una sección "Fuera de alcance" concreta.
- **Criterio comprobable:** condición de aceptación que se puede convertir en test (humano o automático). Si no, no es criterio: es deseo.
- **Spec ejecutable:** spec lista para que la lea cualquier agente (humano o IA) y pueda implementar sin preguntar lo esencial.

Qué viene en M2.5

Práctica guiada. Tú vas a escribir una spec real de una feature de tu elección, paso a paso, pasándola por el ciclo entero: redacción inicial, revisión automática con Claude Code, ajustes, revisión cruzada, hasta dejarla ejecutable. Al terminar el M2 tendrás una spec tuya lista para entrar en producción y un método para repetirlo en cada pieza nueva.

M2.5 · Práctica guiada — escribir y revisar una spec de feature real

Bloque 0 · Qué vamos a hacer en esta práctica

En los cuatro submódulos anteriores has visto la anatomía (intent, behavior, contract), los formatos (markdown, OpenAPI, JSON Schema), las tres plantillas (feature, bugfix, arquitectónico) y el proceso de revisión (los cuatro huecos). Cinco piezas teóricas que solo se pegan al cerebro cuando las practicas una vez.

Esta práctica es la primera vez. Vas a llevar una feature tuya por el ciclo entero, paso a paso. No es un ejercicio de papel: al terminar vas a tener una spec real que podrías meter en tu repositorio mañana.

Recorrido de la práctica, en cinco pasos:

1. **Elegir la feature.** Una pieza de tu trabajo real, no inventada.
2. **Redactar la primera versión.** Plantilla A (feature nueva) en markdown plano.
3. **Revisión automática.** Pasarla por Claude Code con la prompt del M2.4.
4. **Cerrar los huecos.** Editar la spec según los hallazgos.
5. **Revisión humana o equivalente.** Pasarla a un compañero o, si trabajas solo, hacer una segunda lectura crítica con criterio explícito.

Tiempo total estimado: 30 a 45 minutos. Si te lleva más, probablemente has elegido una feature demasiado grande — corta para la siguiente vez.

Bloque 1 · Paso 1 — elegir la feature

La elección de la feature es más importante que parece. Si eliges demasiado pequeña, la práctica no enseña porque la primera versión ya sale ejecutable. Si eliges demasiado grande, la spec se vuelve inmanejable y te frustras antes de terminar.

El criterio: una feature que cumpla las tres condiciones:

Sea real. Algo que de verdad necesitas construir o has construido recientemente. Las features ficticias no funcionan porque no tienes contexto operativo para resolverlas — te quedas en la teoría.

Caiga entre 1 hora y 1 día de implementación. Si la implementación va a ser de menos de una hora, la spec sobra. Si va a ser de varios días, la pieza es demasiado grande para esta práctica;

pártela en piezas más pequeñas y elige una.

Tenga al menos una decisión ambigua de fondo. Esto es importante. Si la feature es de las que se implementan sin tener que decidir nada (un endpoint de eco, un getter simple), la práctica no va a sacar mucho jugo. Busca algo donde haya al menos una decisión que admita varias respuestas razonables.

Tres ejemplos del tipo de feature que encaja bien:

- Un endpoint que filtra una lista de productos por varios criterios opcionales, donde hay que decidir cómo combinar los filtros y qué pasa cuando no se aplica ninguno.
- Una función que envía notificaciones a un usuario sobre un cambio en sus pedidos, donde hay que decidir el canal, la frecuencia, la deduplicación.
- Una migración de una tabla con datos sucios donde hay que decidir qué hacer con los registros incompletos.

Si no se te ocurre nada de tu trabajo, una buena alternativa es la propia función `calcularFechaVencimiento` del M1.4 — repítela tú mismo como ejercicio.

Bloque 2 · Paso 2 — redactar la primera versión

Abre tu editor, crea un nuevo `.md` en `specs/` y empieza con la plantilla A del M2.3. La primera versión la vas a escribir en quince minutos como máximo. No la pulas: el objetivo es tener algo escrito sobre lo que iterar, no producir la versión final a la primera.

Recorrido por las secciones, con foco en lo que la mayoría se salta:

Intent. Una o dos frases sobre el problema que la feature resuelve. No qué hace; para qué existe. Si llevas más de cinco minutos en esta sección, la estás escribiendo mal — probablemente estás incluyendo behavior. Recorta hasta dejar el porqué puro.

Behavior. Empieza por el caso normal en prosa: cuál es el flujo típico, qué entradas recibe, qué salida produce. Luego añade los casos límite: entradas raras, condiciones de error, comportamientos en bordes. Si tienes más de cuatro casos, tabula.

Contract. Si la pieza expone interfaz a otros (función pública, endpoint, evento), redacta el contract: tipos, signatures, esquemas. Si no expone nada, salta esta sección.

Criterios de aceptación. Lista de checks comprobables. La pregunta para cada criterio: "¿podría escribir un test que verifique esto sin preguntar más?". Si no, reescribe el criterio.

Fuera de alcance. Aquí muchos se quedan en blanco. La pregunta para rellenarla: si yo le pasara esta spec a alguien que no conoce el contexto, ¿qué cosas razonables podría asumir que

esta función va a hacer y que en realidad NO va a hacer? Tres o cuatro elementos bastan.

Notas y decisiones. Opcional. Solo si hay una decisión dura detrás (por ejemplo, descartar una alternativa obvia). Si todas las decisiones son obvias, esta sección queda vacía y no pasa nada.

Al terminar este paso vas a tener una spec de entre 30 y 80 líneas. Si tienes mucho menos, falta detalle; si tienes mucho más, sobra implementación disfrazada.

Bloque 3 · Paso 3 — revisión automática con Claude Code

Abre Claude Code y usa la prompt del M2.4. La repito aquí por comodidad:

```
Eres revisor de specs SDD. Lee la spec que sigue y dime qué huecos detectas, organizados en cuatro categorías:
```

1. Ambigüedades: frases que admiten más de una interpretación razonable.
2. Contradicciones: dos partes de la spec que no pueden cumplirse a la vez.
3. Alcance abierto: cosas que la spec NO dice si están dentro o fuera del scope.
4. Criterios de aceptación: revisa si son comprobables.

```
NO me corrijas la spec. Solo dime qué huecos hay.
```

```
Spec:
```

```
[pega aquí el contenido del .md]
```

El modelo te va a devolver una lista de 5 a 20 puntos, según el tamaño y la calidad de tu spec inicial. Léelos uno a uno y clasifícalos en tu cabeza:

Hallazgos válidos. Cosas que efectivamente son huecos y vas a corregir.

Hallazgos discutibles. Cosas que el modelo señala pero tú consideras que están bien como están. Decide si las dejas o no; tener una opinión razonada también es trabajo.

Hallazgos que abren conversación. Cosas que requieren preguntar a alguien (PM, otro dev, dominio) antes de cerrarlas. Apúntalas para el paso 5.

Importante: NO le pidas al modelo que te corrija la spec en este paso. Tú decides cómo cerrar cada hueco. El modelo es revisor, no reescritor.

Bloque 4 · Paso 4 — cerrar los huecos

Con la lista de hallazgos delante, vuelve a tu `.md` y edítalo. Para cada hueco válido, decide:

Ambigüedad → reescribe la frase con precisión. Si "30 días" era ambiguo entre naturales y hábiles, ponlo: "30 días naturales".

Contradicción → elige una opción o reformula. Si el behavior decía "rápido" y el contract decía "consulta externa lenta", decide cuál de los dos cambias. La decisión es tuya; la spec solo refleja lo que decidas.

Alcance abierto → añade a "Fuera de alcance" lo que corresponda. "Esta función NO envía notificaciones; eso lo hace [otro componente]."

Criterio no comprobable → reescríbelo en condiciones binarias. "El sistema debe ser rápido" → "El endpoint responde en menos de 200 ms en el percentil 95 con carga de 100 rps".

Para los hallazgos que abren conversación, déjalos marcados con `TODO: discutir` en la propia spec, en línea, para que la siguiente persona que la lea sepa que ese punto está pendiente.

Una vez cerrados los huecos válidos, vuelve a leer la spec entera de principio a fin. Comprueba que sigue siendo coherente: a veces cerrar un hueco abre otro en otra parte. Si lo hace, ciérralo también.

Bloque 5 · Paso 5 — revisión humana

Si trabajas en equipo, este paso es claro: abre un PR con la spec y pídele review a alguien del equipo. Si conoce el dominio operacional o el contexto del problema, mejor. La conversación queda en el PR.

Si trabajas solo, no tienes acceso a una revisión humana real. Hay dos sustitutos razonables.

Lectura crítica con criterio explícito. Deja la spec una hora aparcada. Vuelve a leerla con la mentalidad de "soy alguien nuevo que tiene que implementar esto sin haber estado en la conversación". Marca cada frase que te pediría aclaración si fueras nuevo. Esas son los huecos restantes.

Segunda pasada con Claude Code, otro rol. Cambia la prompt y pídele al modelo que se ponga en el papel de implementador, no de revisor: "Soy el dev que va a implementar esto. ¿Qué decisiones tendría que tomar yo solo si me das esta spec y nada más?". Las respuestas te marcan los huecos donde la spec sigue dejando decisiones implícitas.

Combina los dos sustitutos si la pieza es importante. Si es una práctica de curso, con uno basta.

Bloque 6 · Cierre del módulo M2

Si has hecho la práctica completa, ahora tienes en tu repo (o en tu carpeta de práctica) una spec de feature real, escrita en markdown plano, con sus tres niveles, revisada dos veces, con los

huecos cerrados o marcados para discusión. Esa spec ya es una pieza de SDD que vale.

Tres ideas para llevarte del módulo entero, antes de entrar en M3.

La anatomía es lo de menos. Memorizar "intent + behavior + contract" no te hace mejor. Lo que te hace mejor es haber hecho el ciclo una vez con una pieza tuya. Y haberlo hecho dos o tres veces es lo que hace que se vuelva automático.

La revisión es donde está el grueso del valor. Si tu equipo escribe specs pero no las revisa, está dejando el 60% del beneficio sin recoger. Forzar la revisión en el flujo de trabajo (en PR, con automatización si se puede) es el cambio cultural más rentable que puedes empujar.

La spec no es estática. Una spec viva se actualiza cuando el mundo cambia, igual que el código. Si tu equipo trata las specs como documentos fundacionales que no se tocan, va a divergir del código en seis semanas. Trátala como un artefacto vivo: cuando cambia el requisito, primero cambia la spec, después el código.

Cierre y enlace a M3

En el M3 entramos en Claude Code en serio. Hasta ahora hemos hablado de la spec como artefacto y del modelo como lector de la spec. En el siguiente módulo bajamos al cómo se trabaja con Claude Code para implementar lo que la spec describe: Plan Mode, skills, subagentes, sesión, memoria entre conversaciones. Tu spec de práctica va a venir con nosotros — la vamos a llevar de markdown a código real usando Claude Code como motor, con disciplina.

Si quieres adelantarte: revisa la documentación de Plan Mode antes del M3.1. Es la pieza donde Claude Code se diferencia de un chat normal y donde SDD encuentra su socio natural.

Glosario rápido del bloque

- **Práctica guiada:** ejercicio paso a paso donde el alumno hace, el material acompaña. Distinto de una práctica abierta donde el alumno improvisa.
- **TODO: discutir:** marca en línea dentro de la spec para señalar un punto pendiente de resolver con otra persona. Visible para cualquier lector posterior.
- **Lectura crítica con criterio explícito:** leer un texto propio adoptando la perspectiva de un lector externo nuevo. Sustituto razonable de la revisión humana cuando no hay equipo.

PARTE 3

Claude Code como motor de SDD

M3.1 · Modelo mental — Claude Code no es un chat

Bloque 0 · Por qué importa el modelo mental

Llegas al M3 con dos cosas claras: la spec es el contrato y SDD es un método para construir software con disciplina cuando hay IA en el flujo. Ahora viene la pregunta práctica: ¿cómo usas Claude Code para que de verdad respete ese contrato?

La trampa más común al empezar es tratar Claude Code como si fuera ChatGPT con permisos de escribir en disco. Le sueltas una petición, esperas la respuesta, copias y pegas. Eso es chat, y desperdicia la mitad de lo que Claude Code puede hacer.

Claude Code es otra cosa. Tiene una arquitectura distinta, un bucle interno propio, y unas piezas (sesión, contexto, herramientas, modo de planificación) que hay que entender antes de usarlas. Si entiendes el modelo mental, las decisiones de SDD que vienen en los siguientes submódulos van a parecerte obvias. Si no lo entiendes, todo el módulo te va a sonar a recetas sueltas.

Quince minutos para fijar el modelo. Luego pasamos a las piezas concretas.

Bloque 1 · Lo que Claude Code NO es

Tres asunciones que la gente trae cuando empieza y que hay que desmontar:

No es un chat con scope ampliado. Cuando le hablas a ChatGPT, la conversación es la unidad: tú escribes, él responde, escribes, responde. El modelo es pasivo entre turnos. Claude Code no: el modelo ejecuta una secuencia de acciones por su cuenta entre tu petición y su respuesta. Puede leer archivos, escribir archivos, ejecutar comandos, llamar a APIs externas. Lo que tú ves como "una respuesta" puede haber sido internamente cinco o veinte acciones encadenadas.

No es un autocompletado glorificado. Copilot autocompleta lo que estás tecleando línea a línea, en el editor. Claude Code no completa: razona, decide qué hacer, lo hace, observa el resultado, decide qué hacer a continuación. Es un proceso distinto y produce resultados distintos. Copilot acelera la mecanografía; Claude Code asume responsabilidad sobre tramos enteros de trabajo.

No tiene memoria infinita. Cuando hablas con un chat, la conversación entera entra en cada turno. Cuando hablas con Claude Code, también — pero el contexto es finito, se llena, y eventualmente se compacta o se pierde. Las primeras sesiones que tengas con él van bien porque

el contexto está limpio. La sexta hora del día con la misma sesión empieza a degradarse. Saber cuándo cortar y abrir sesión nueva es parte del oficio.

Estas tres asunciones no son académicas. Determinan cómo escribes la spec, cómo formulas la petición, qué esperas que pase, y dónde te tocará intervenir.

Bloque 2 · Lo que SÍ es — agente con herramientas

Claude Code es un **agente** con acceso a un conjunto de **herramientas**, operando en una **sesión** con **contexto** finito. Cuatro piezas que merecen pararse a definir.

Agente. En el sentido técnico: un sistema que recibe un objetivo, decide qué pasos dar para lograrlo, y ejecuta esos pasos por su cuenta. No es solo "responder bien". Es decidir qué acciones tomar, en qué orden, y cuándo se ha terminado. Un agente puede equivocarse, puede iterar, puede volver atrás. Las acciones que toma son reales: archivos que se modifican, comandos que se ejecutan, llamadas que llegan a servicios externos.

Herramientas. Las acciones concretas que el agente puede ejecutar. Algunas vienen por defecto (leer archivo, editar archivo, ejecutar bash, buscar en el repo). Otras se añaden — vía MCP que veremos en el M5, vía configuración explícita. Cada herramienta tiene una descripción de qué hace, qué inputs recibe, qué outputs devuelve. El agente elige qué herramienta usar en cada paso. Las herramientas son su mano: sin ellas solo razona; con ellas, actúa.

Sesión. La unidad de trabajo. Una sesión empieza cuando abres Claude Code y termina cuando la cierras o se compacta automáticamente. Dentro de la sesión, el agente recuerda todo lo que has discutido y todas las acciones que ha tomado. Entre sesiones, no recuerda nada (salvo cosas explícitamente persistidas como memoria, archivos `CLAUDE.md` o handoffs escritos por ti).

Contexto. El espacio finito que la sesión llena conforme avanza. Cada mensaje, cada archivo leído, cada resultado de herramienta, cada paso de razonamiento — todo consume contexto. Cuando se llena, las opciones son malas: cortar contexto antiguo (perder memoria de lo discutido), compactar (resumir y arriesgarse a perder matices), o cerrar y abrir nueva. Una sesión productiva suele cubrir entre una y tres horas de trabajo antes de necesitar refresco.

Las cuatro piezas juntas son lo que distingue a Claude Code de un chat. Cuando le pides algo, no lo procesa de un tirón: itera dentro del bucle que vamos a ver ahora.

Bloque 3 · El bucle ReAct, en versión sin academia

ReAct es el nombre técnico (Reasoning + Acting) del bucle interno que ejecuta Claude Code en cada turno. Para entenderlo no necesitas leer el paper. Necesitas verlo en acción.

Imagínate que le pides: "Implementa la función `calcularFechaVencimiento` siguiendo la spec en `specs/factura.md`". Lo que pasa por dentro, paso a paso:

1. **Razona.** Lee tu petición. Piensa qué necesita para empezar. Decide que primero debe leer la spec.
2. **Actúa.** Llama a la herramienta `read_file` con argumento `specs/factura.md`.
3. **Observa.** Recibe el contenido del archivo.
4. **Razona.** Procesa la spec. Decide que necesita ver la estructura del proyecto para saber dónde colocar el código.
5. **Actúa.** Llama a `list_files` o `glob` para inspeccionar el repo.
6. **Observa.** Recibe la lista.
7. **Razona.** Identifica el archivo donde va la función. Decide qué tipos importar, qué patrones seguir, cómo estructurar.
8. **Actúa.** Llama a `write_file` o `edit_file` para crear o modificar el archivo.
9. **Observa.** Recibe confirmación.
10. **Razona.** Decide si necesita escribir tests, comprobar compilación, ejecutar algo. Continúa el bucle.

Y así hasta que considera que la petición está resuelta. Quince acciones, treinta, las que necesite. Todo eso entre tu prompt y la respuesta final que ves.

Importante: cada vuelta del bucle es una decisión real. El agente puede elegir mal en cualquier paso. Si lee mal la spec, si elige la herramienta equivocada, si interpreta el resultado de una herramienta de manera incorrecta, el resto del bucle se contamina. Tu spec, tu Plan Mode, tus hooks — todo eso está para acotar las decisiones del bucle a lo que tú quieres.

Una pregunta natural: si el bucle es así de complejo, ¿por qué la respuesta a veces sale tan rápido? Porque para tareas pequeñas (una función de quince líneas) basta con dos o tres vueltas del bucle. Para tareas grandes (una feature entera) son veinte, treinta o más. La velocidad percibida es lineal con el tamaño de la tarea.

Bloque 4 · Sesión, contexto y la gestión de los dos

El bucle anterior consume contexto. Cada acción, cada observación, cada paso de razonamiento se acumula en la memoria de la sesión. Eso tiene dos consecuencias prácticas para interiorizar:

Las sesiones tienen ciclo de vida. No abras Claude Code el lunes y trabajes con la misma sesión toda la semana. El contexto se llena, las decisiones tempranas se pierden, los archivos antiguos siguen pesando aunque ya no sean relevantes. Las sesiones rinden bien cuando son cortas (entre una y tres horas) y temáticas (una feature, un bugfix, una migración concreta).

Lo importante se persiste por escrito. Cuando una sesión te haya generado conocimiento que vale para el futuro (una decisión de diseño, un patrón nuevo, un descubrimiento sobre el código existente), no lo dejes en la sesión: pásalo a un archivo del repo. Puede ser un `CLAUDE.md` con instrucciones permanentes, una nota en `docs/`, un comentario en la spec. Lo que vive en una sesión muere con la sesión.

Un patrón útil para sesiones largas: al final de cada sesión productiva, pídele al agente que escriba un breve "handoff" para la siguiente sesión: qué se hizo, qué decisiones se tomaron, qué queda pendiente, qué cosas hay que tener en cuenta para retomar. Lo guardas en un archivo. La siguiente sesión empieza leyéndolo. Ese hábito te ahorra muchas reconstrucciones penosas.

Bloque 5 · Plan Mode vs ejecución libre

La distinción operativa más importante de Claude Code, y la que más cuesta interiorizar para quien viene del chat.

Ejecución libre. El modo por defecto en muchos clientes. Le pides algo, el bucle empieza, las acciones se ejecutan. Si te equivocaste en la petición o la spec era ambigua, el modelo va a tomar decisiones que no querías mientras tú miras. El resultado puede ser bueno o un desastre que requiere `git reset`. Es rápido pero arriesgado.

Plan Mode. Una capa intermedia. Le pides algo y el modelo razona, decide qué acciones tomar, te las **enseña en forma de plan**, y espera tu confirmación antes de ejecutar. Tú lees el plan, lo discutes, lo iteras, lo apruebas. Solo cuando dices "adelante" empieza el bucle de acción.

Plan Mode es a Claude Code lo que la spec es al desarrollo: un paso intermedio que te permite ver y corregir antes de comprometerte. Y a veces es la diferencia entre "uy, ha tocado quince archivos que no quería" y "ah, ha listado lo que va a tocar, dejo fuera dos y pa'lante".

Regla informal: para cualquier tarea no trivial, **arranca en Plan Mode**. Lo no trivial es: cualquier cosa que toque más de un archivo, cualquier cosa que cambie comportamiento observable, cualquier cosa donde el coste de equivocarse es alto. Para cosas pequeñas (renombrar una variable, ajustar un comentario, regenerar un fichero conocido), ejecución libre funciona.

Aviso: Plan Mode no es a prueba de fallos. Si el plan está mal pensado y le das luz verde, el resultado va a estar igual de mal. Tu trabajo en Plan Mode es leer el plan con atención, no decir

"sí" por inercia. Esa es la disciplina que diferencia un dev que usa Plan Mode de uno que lo tiene activado sin sacarle partido.

Bloque 6 · Cierre

Tres ideas para llevarte.

Claude Code es un agente con herramientas, no un chat. La diferencia se nota: ejecuta acciones reales, itera en un bucle, toma decisiones por su cuenta. Quien lo trata como chat le saca el 30% del potencial.

El bucle ReAct (razonar, actuar, observar) es lo que pasa por dentro en cada turno. Cuanto más entiendas que cada vuelta es una decisión real, mejor escribes specs para acotarla y mejor sabes dónde meter hooks (M4) que la validen.

Plan Mode es la palanca para no perder el control. En cualquier tarea no trivial, plan primero, ejecución después. Esa disciplina convierte a Claude Code de juguete potente en herramienta de producción.

Glosario rápido del bloque

- **Agente:** sistema que recibe un objetivo, decide pasos para lograrlo y los ejecuta. A diferencia de un chat, actúa por su cuenta entre tu petición y su respuesta.
- **Herramienta:** acción concreta que el agente puede ejecutar (leer/escribir archivo, ejecutar comando, llamar API). Forma parte del repertorio del agente, ampliable vía MCP u otras configuraciones.
- **Sesión:** unidad de trabajo con Claude Code. Tiene contexto finito y ciclo de vida; lo importante se persiste por escrito antes de cerrarla.
- **Contexto:** espacio donde se acumula todo lo que el agente ha visto y hecho en la sesión. Finito. Cuando se llena, hay que compactar o cerrar.
- **Bucle ReAct:** ciclo razonar → actuar → observar que ejecuta el agente en cada turno. Una petición puede generar muchas vueltas del bucle.
- **Plan Mode:** modo en el que el agente enseña su plan antes de ejecutarlo y espera confirmación. Reduce sorpresas y permite intervenir antes de actuar.

Qué viene en M3.2

Entramos a fondo en Plan Mode. Cuándo activarlo, cómo leer un plan generado, qué firmar y qué rechazar, cómo iterar el plan antes de ejecutar. Es la herramienta más infrautilizada de Claude Code y la que más rinde cuando se aprende a usar bien. Tu spec de la práctica del M2.5 viene a la fiesta: vamos a llevarla a Plan Mode y ver cómo se traduce en plan ejecutable.

M3.2 · Plan Mode y /plan — el puente entre spec y código

Bloque 0 · Por qué este submódulo importa más de lo que parece

La gente que prueba Plan Mode una semana y lo abandona suele tener la misma queja: "me ralentiza". Y tienen razón a medias. Plan Mode añade un paso intermedio entre tu prompt y la ejecución. Ese paso cuesta tiempo si lo lees con atención, que es como hay que leerlo.

Pero esa medida del tiempo es engañosa. El paso "me ralentiza" tres minutos al inicio y me ahorra treinta minutos de re-trabajo al final. Cuando alguien dice "Plan Mode me frena" suele estar comparando los tres minutos contra cero, no contra los treinta. Como con el cinturón de seguridad: te frena un segundo cada vez que arrancas el coche, y te salva cuando hace falta.

En este submódulo vamos a aterrizar cómo se trabaja con Plan Mode día a día. Mecánica de activación, cómo se ve un plan generado, los cuatro elementos que hay que leer con atención, cómo se itera un plan que no cuadra, y los errores comunes que ralentizan a los recién llegados. Al terminar deberías sentirte cómodo con Plan Mode como tu modo por defecto para cualquier tarea no trivial.

Bloque 1 · Cómo se activa Plan Mode

La mecánica concreta depende del cliente que uses (VS Code, CLI, terminal estándar) y de la versión, pero el patrón es el mismo: hay un atajo o un comando explícito (`/plan` es común) que pone al agente en modo planificación antes de ejecutar.

Hay dos formas de pensarlo en tu flujo:

Modo por sesión. Configuras Plan Mode como modo por defecto para toda la sesión. Cualquier petición no trivial pasa por planificación antes de ejecución. Es la opción más segura para empezar.

Modo por petición. Mantienes ejecución libre como default y activas Plan Mode solo en las tareas donde lo necesitas. Es lo que usa la gente con experiencia: para cosas pequeñas, ejecución directa; para cualquier cosa que toque más de un archivo o cambie comportamiento, plan primero.

El criterio para decidir cuál de los dos modos te encaja: si todavía estás aprendiendo a leer planes, ve por defecto. Si ya tienes ojo entrenado y sabes detectar cuándo una petición es "no trivial" sin

pensarlo, ve por petición. Como con todo en SDD, la regla baja del proceso al hábito conforme acumulas oficio.

Un detalle pequeño que ahorra fricción: cuando trabajas con specs (M1, M2), la prompt suele tener la forma "implementa lo descrito en `specs/X.md`". Si esa spec dispara Plan Mode automáticamente, no tienes que acordarte de activarlo cada vez. Algunos clientes permiten configurar esto.

Bloque 2 · Anatomía de un plan generado

Cuando Plan Mode se activa, Claude Code te devuelve un documento estructurado en vez de empezar a tocar archivos. Ese documento es el plan. Su estructura típica:

Resumen. Una o dos frases sobre lo que va a hacer en términos generales. "Voy a implementar la función `calcularFechaVencimiento` siguiendo la spec, con tests unitarios para los cinco criterios de aceptación."

Archivos que va a tocar. Lista concreta de archivos que va a crear, modificar o eliminar. Cada uno con una nota corta de qué cambio. Si la lista es de quince archivos cuando esperabas dos, ahí tienes la primera señal de alarma.

Pasos secuenciales. El plan en sí, descompuesto en acciones. "1. Crear `src/facturas/calcularVencimiento.ts` con la signature según el contract. 2. Implementar la lógica del caso normal. 3. Añadir los ajustes para fin de semana y festivos. 4. Crear `src/facturas/calcularVencimiento.test.ts`. 5. Escribir tests para los cinco criterios. 6. Ejecutar tests y verificar que pasan."

Decisiones tomadas. Si la spec dejaba ambigüedades (en una spec real perfectamente revisada, esto no debería pasar — pero la realidad es más sucia), el plan suele listar qué decisiones ha tomado por su cuenta. "He asumido que el archivo de festivos vive en `recursos/festivos.json`. He decidido representar las fechas internamente como `DateTimeOffset`. He elegido lanzar `InvalidInvoiceException` como subclase de `Exception` estándar."

Riesgos identificados. Lo que el modelo cree que puede ir mal. "No tengo claro si la zona horaria debe configurarse externamente o hardcodearse a Europe/Madrid. Si quieres que sea configurable, dímelo antes de ejecutar."

No todos los planes traen todas las secciones, y algunos clientes formatean distinto. Pero los cinco elementos suelen estar de alguna forma. Saber buscarlos te ahorra leer en diagonal.

Bloque 3 · Las cuatro cosas que hay que leer con atención

Aquí está el oficio. Plan Mode te da el plan; tú tienes que leerlo. Y leerlo no es ojear: es revisar cuatro cosas concretas en cada plan que recibes.

1. ¿Toca más cosas de las que pediste? Mira la lista de archivos. Si pediste implementar una función y aparece "modifico tres archivos de configuración y dos imports en módulos no relacionados", el modelo ha decidido por su cuenta hacer ajustes adicionales. A veces tiene buen criterio y los ajustes son necesarios; a veces se está pasando de listo. Tu trabajo: detectar el ámbito del cambio y decidir si lo aceptas.

2. ¿Las decisiones tomadas por el agente son las que tú habrías tomado? Lee la sección de decisiones (si existe) o infiere las decisiones implícitas de los pasos. Si el plan dice "voy a usar `NodaTime` para las fechas" y tú prefieres mantenerte en los tipos de fecha nativos de .NET, paras aquí. Mejor parar que descubrirlo cuando el paquete ya está en el `.csproj`.

3. ¿La secuencia de pasos tiene sentido? No todos los caminos a un mismo destino son iguales. Si el plan propone "primero borro el archivo viejo, después escribo el nuevo", se está saltando una protección — si la escritura falla, te queda el código sin el archivo viejo. Un plan mejor: "escribir el nuevo, comprobar que pasa, después borrar el viejo". Cuestión de orden, pero importa.

4. ¿Los riesgos identificados resuenan con los tuyos? Si el modelo señala un riesgo y tú no lo habías visto, presta atención — puede que tenga razón. Si tú ves un riesgo que él no señala, añádelo a la conversación antes de aprobar. Ningún plan está completo si solo recoge los riesgos del lado del modelo.

Una regla informal: si el plan ocupa más de una pantalla, léelo dos veces. Si ocupa media pantalla, una vez basta. Si ocupa tres pantallas y todavía no has llegado al primer paso concreto, el plan está mal redactado: pídele que lo condense antes de seguir.

Bloque 4 · Cómo iterar un plan

El plan no es un take-it-or-leave-it. Es el inicio de una conversación. Cuando algo no te cuadra, no apruebas y no ejecutas: respondes con un ajuste y dejas que el agente regenere el plan.

Algunas formas concretas de iterar:

Restringir alcance. "El plan toca cinco archivos. Límitate a los dos de la carpeta `facturas/`; los demás los dejamos como están en este PR." El agente regenera el plan con menos archivos.

Rechazar una decisión. "Has decidido añadir `NodaTime`. No quiero dependencias nuevas; usa `DateTimeOffset` que ya está en el framework." El agente reajusta.

Cambiar el orden. "El paso 3 debe ir antes que el 2. Primero los tests, después la implementación." El agente reordena.

Añadir un paso que falta. "Antes de ejecutar tests, comprueba que el archivo de festivos existe; si no, créalo con la lista mínima." El agente añade el paso.

Pedir que defienda una decisión. "Has elegido lanzar excepción en lugar de devolver `null` para el caso de fechaEmission futura. ¿Por qué?" Esto no es para hostigar al modelo: a veces el agente tiene un argumento que tú no habías pensado y merece la pena leerlo. Si la respuesta te convence, sigues; si no, le pides que cambie.

Una iteración rara vez requiere reescribir el plan entero. Suelen ser ajustes pequeños sobre un plan que ya estaba 80% bien. Dos o tres iteraciones bastan para la mayoría de las tareas. Si llevas siete iteraciones y el plan sigue sin cuadrar, el problema probablemente está en la spec — vuelve atrás y revísala antes de seguir peleando con el plan.

Bloque 5 · Errores comunes con Plan Mode

Cuatro patrones que veo en gente recién llegada a Plan Mode y que retrasan el aprendizaje.

Aprobar por inercia. Después de unos días, el plan empieza a parecer rutinario y se le da OK sin leerlo a fondo. Cuando esto pasa, Plan Mode deja de aportar — los errores que vinieron a filtrar se cuelan igual. La defensa: leer cada plan como si fuera el primero. Si llevas mucho rato y notas que ya no estás leyendo, para diez minutos y vuelve.

Rechazar por miedo. Lo contrario. El plan se ve denso, da pereza leerlo, y se le pide al agente "haz solo la primera parte" para reducir riesgo. Eso fragmenta el trabajo y a menudo empeora las cosas — el agente no tiene visión global y cada parte la decide aislada. Mejor leer el plan grande con atención que partir el problema en miniaturas.

Iterar para ajustar el código que va a generar. "Cambia el nombre de la variable de `total` a `subtotal`." En Plan Mode esto no toca: el plan describe acciones, no líneas de código. Si te metes a corregir nombres de variable antes de generar, estás haciendo dos rondas (plan + revisión post-generación) en lugar de una. Deja los detalles para después de la ejecución.

Confundir Plan Mode con review de PR. Plan Mode revisa lo que el agente va a hacer; el review de PR revisa el código generado. Son momentos distintos. Si en Plan Mode entras a discutir convenciones de código, idiomas internos del repo o patrones específicos del equipo, estás haciendo el trabajo de PR review demasiado pronto. Plan Mode mira arquitectura y alcance; PR review mira calidad de código.

Bloque 6 · Cierre

Tres ideas para llevarte.

Plan Mode es el filtro más barato que tiene Claude Code para evitar errores grandes. Tres minutos al inicio te ahorran treinta de re-trabajo al final. Compensa siempre que la tarea sea no trivial.

Leer un plan tiene técnica: mirar alcance, decisiones, secuencia y riesgos. No es leer rápido — es leer dirigido. Con práctica cada plan tarda dos minutos en revisarse.

El plan se itera, no se acepta o rechaza. Cuando algo no cuadra, ajustas con instrucción concreta y el agente regenera. Dos o tres iteraciones suelen bastar; más de cinco indican problema en la spec, no en el plan.

Glosario rápido del bloque

- **Plan Mode:** modo en el que Claude Code enseña su plan antes de ejecutarlo y espera tu confirmación.
- **/plan:** comando explícito (en muchos clientes) para activar Plan Mode si no está como modo por defecto.
- **Alcance del plan:** conjunto de archivos que el plan tocará. Primera cosa que se revisa para detectar si el agente se está pasando de listo.
- **Decisión implícita en plan:** opción tomada por el agente que no estaba en la spec. Si la spec era buena, esto debería ser cero; en la práctica aparece algo, y la revisión las caza.

Qué viene en M3.3

Skills. Cómo abstraer specs reutilizables en piezas que el agente puede invocar de forma consistente. Skill = spec + procedimiento + activadores. Veremos cuándo crear skill y cuándo basta con poner el procedimiento en el prompt, con demos cortas de tres skills útiles para SDD: revisar spec, generar tests desde behavior, refactorizar con criterio. Tu spec del M2.5 sigue acompañándote a la fiesta.

M3.3 · Skills — abstraer specs reutilizables

Bloque 0 · Por qué este submódulo es largo

Los skills tienen mala fama por dos motivos opuestos. Unos los ven como "una macro avanzada" y los descartan por aparentemente triviales. Otros los ven como "una abstracción rara que solo entienden los que llevan tiempo" y los descartan por intimidatorios. Ninguna lectura ayuda.

En realidad los skills son la pieza más rentable de Claude Code después de Plan Mode. Cuando los entiendes, dejas de repetir el mismo procedimiento en cada conversación. Cuando los compartes con tu equipo, dejáis de tener "el flujo de Pedro" y "el flujo de Marta" y empezáis a tener un flujo del equipo. Cuando los pones bajo control de versiones, los skills evolucionan con el proyecto y queda registro de por qué cambiaron.

Veinticinco minutos en este submódulo. Lo que viene: qué es un skill operativamente, su anatomía, cuándo crear uno y cuándo no, tres skills útiles para SDD desarrollados con detalle, y un apartado de mantenimiento que la gente suele olvidar hasta que se le mueren los skills. Vamos por orden.

Bloque 1 · Qué es un skill, operativamente

Un skill es un procedimiento documentado que el agente carga en su contexto cuando algo en la conversación lo activa. Tiene tres partes:

Una spec del procedimiento. Igual que las specs del M2 describen qué construir, la spec del skill describe qué hace el procedimiento, en qué casos se aplica, qué entradas espera y qué resultados produce.

El procedimiento en sí. Los pasos concretos a ejecutar. Pueden ser instrucciones en lenguaje natural ("primero lee la spec, después analiza ambigüedades..."), comandos shell, llamadas a herramientas específicas, referencia a archivos del proyecto.

Activadores. Las señales que indican cuándo el skill se carga. Pueden ser palabras clave en la conversación, formatos de archivo, tipos de petición, contextos específicos. El agente los detecta y carga el skill automáticamente — o tú lo invocas explícitamente.

La parte que cuesta entender al principio es la segunda: el skill no se ejecuta como un script. El agente lee el procedimiento, lo entiende, y lo aplica al caso concreto que tiene delante. Un skill

bien escrito es como darle un manual a un nuevo del equipo: con ese manual sabe qué hacer ante una situación dada, sin que tengas que explicárselo desde cero cada vez.

Eso es la clave: el skill **no es código que se ejecuta**, es **conocimiento que se aplica**. El agente sigue tomando decisiones con criterio dentro del skill; el skill le da la disciplina y el contexto.

Bloque 2 · Anatomía de un skill típico

Veamos cómo se ve un skill por dentro. La estructura concreta varía según el cliente y la versión, pero el patrón es estable. Un skill típico vive en un archivo (`.md` , generalmente) en una carpeta de skills del proyecto. Su contenido:

```
---
name: revisar-spec-sdd
description: "Revisa una spec SDD detectando los cuatro huecos típicos."
triggers:
  - "revisa esta spec"
  - "spec review"
  - "qué huecos tiene esta spec"
metadata:
  version: 1
  owner: equipo-backend
---
```

Skill: revisar-spec-sdd

Cuándo se aplica
Cuando el usuario pide revisar una spec antes de implementarla.
El input es un archivo `.md` con estructura SDD (intent, behavior, contract).

Procedimiento

1. Lee la spec completa.
2. Detecta ambigüedades: frases que admiten más de una interpretación.
3. Detecta contradicciones: dos partes que no pueden cumplirse a la vez.
4. Detecta alcance abierto: cosas que la spec no dice si están dentro o fuera.
5. Revisa criterios de aceptación: ¿se pueden convertir en test?
6. Devuelve la lista de hallazgos organizada por las cuatro categorías.
7. NO corrijas la spec. Solo reporta los hallazgos.

Formato de salida

- Lista numerada por categoría.
- Para cada hallazgo: la frase exacta y el problema concreto.
- Al final, una valoración global: ¿lista para implementar, lista con ajustes menores, o necesita revisión profunda?

Notas

- Si la spec no sigue el formato SDD (intent/behavior/contract), avisa antes de empezar.
- Si la spec es excepcionalmente buena (cero hallazgos), reporta también eso.

Tres elementos de esta estructura que merecen atención:

El frontmatter (--- arriba) es donde viven los metadatos. El nombre identifica el skill; la descripción y los triggers son los que el agente usa para decidir cuándo cargarlo automáticamente. Si los activadores están mal calibrados, el skill se carga cuando no toca (ruido) o no se carga cuando toca (inútil).

El cuándo se aplica delimita el ámbito. No basta con que el skill exista: hay que decir explícitamente para qué casos sirve. Esto evita que se invoque para situaciones similares pero no idénticas.

El procedimiento numerado es lo que el agente sigue. Los números no son rituales: imponen orden y permiten al agente reportar "estoy en el paso 3 de 7", que ayuda en sesiones largas.

Bloque 3 · Cuándo crear skill y cuándo no

La pregunta operativa más útil: ¿este procedimiento merece skill o basta con escribirlo en el prompt esta vez?

Cuatro criterios para decidir, y basta con que falle uno para no hacer skill:

¿Lo voy a usar más de tres veces? Si es un procedimiento que vas a invocar solo una vez en este proyecto, no hace falta skill. El skill compensa cuando hay repetición. Si vas a revisar specs cada semana, sí — skill. Si vas a migrar de una librería concreta una sola vez en la historia del proyecto, no — instrucciones en el prompt y a otra cosa.

¿El procedimiento es estable? Si las instrucciones van a cambiar cada vez que lo apliques (porque cada caso es muy distinto), no hace falta skill — meter en skill algo que vas a reescribir cada vez es burocracia. El skill sirve cuando el procedimiento es el mismo, aplicado a casos distintos.

¿Otros del equipo van a beneficiarse? Si solo lo vas a usar tú y nadie más, el skill puede vivir en tu configuración local. Pero si forma parte del flujo de trabajo del equipo, va al repositorio compartido — y ya entra en el ámbito de "primera clase" como las specs.

¿La automatización añade valor sobre escribirlo cada vez? Hay procedimientos cortos donde tipear las instrucciones cada vez cuesta veinte segundos. Skill es mejor cuando el ahorro es de minutos por invocación, no de segundos. Para procedimientos cortos y frecuentes, alias o snippets del editor suelen funcionar mejor.

Una regla informal: si dudas, no hagas skill. La penalización de tener pocos skills bien afinados es baja. La penalización de tener treinta skills medio escritos es alta — el agente acaba cargando ruido cada vez que cree detectar un trigger.

Bloque 4 · Skill útil número uno — revisar-spec-sdd

Lo arranqué como ejemplo en el bloque 2; aquí lo desarrollamos como pieza real. El procedimiento ya está; vamos a hablar de cómo se usa en el día a día.

Cuándo se invoca. Cada vez que terminas de escribir una spec nueva en `specs/`. El trigger puede ser explícito (escribes "revisa esta spec" en Claude Code) o automático (si el cliente lo soporta, al detectar un commit nuevo en `specs/` el skill se ofrece). Tres minutos de revisión, lista de hallazgos, y decides qué cerrar antes de implementar.

Qué te ahorra. Los cinco a quince minutos de explicarle cada vez al agente cómo revisar (qué buscar, qué formato, qué no hacer). Con el skill, basta con "revisa esta spec" y el procedimiento está implícito.

Dónde rompe. Si la spec no sigue el formato SDD, el skill no sabe qué buscar. Por eso el procedimiento incluye "si la spec no sigue el formato, avisa antes de empezar". Sin esa nota, el skill puede dar hallazgos inventados sobre un formato que no entiende.

Una versión más afinada del skill podría incluir referencias a las plantillas concretas del equipo (las del M2.3), de manera que detecte si la spec sigue la plantilla A, B o C y aplique criterios específicos a cada una. Esa es la evolución natural: empiezas genérico, vas afinando según el uso real.

Bloque 5 · Skill útil número dos — generar-tests-desde-behavior

Otro skill que merece sitio en un equipo SDD. La spec ya tiene su sección de behavior con casos típicos y límite. Convertir esos casos en tests unitarios o de integración es trabajo mecánico que el agente hace bien si le das procedimiento claro.

```

---
name: generar-tests-desde-behavior
description: "Genera tests automáticos a partir de la sección Behavior de una spec."
triggers:
  - "genera tests para esta spec"
  - "tests desde behavior"
metadata:
  version: 1
  owner: equipo-backend
---

# Skill: generar-tests-desde-behavior

## Cuándo se aplica
Cuando la spec tiene una sección Behavior con casos identificables
(prosa con casos, tabla, Given/When/Then) y existe el código a testear.

## Procedimiento

1. Lee la sección Behavior de la spec.
2. Identifica los casos: enuméralos del 1 al N.
3. Para cada caso, propone un test con:
  - Nombre descriptivo en formato `caso_N_descripción_corta`.
  - Setup mínimo necesario.
  - Acción exacta del caso.
  - Aserción que verifica el resultado esperado.
4. Si el behavior menciona un caso de error, incluye también su test con
  la aserción de excepción correcta.
5. Si la spec tiene "Criterios de aceptación" comprobables, vincula cada
  test con su criterio en un comentario.
6. Devuelve el código de los tests en el framework del proyecto.
7. NO modifiques el código fuente. Solo genera tests.

## Convenciones del proyecto
- Framework: xUnit v3 (consultar el `.csproj` del proyecto de tests para versión).
- Estilo: clases `[Fact]`/`[Theory]`, agrupadas por SUT, sin nesting profundo.
- Mocks: NSubstitute para colaboradores externos.
- Cada archivo de test vive en el proyecto `<Proyecto>.Tests/`,
  con sufijo `Tests.cs` (`FacturaServiceTests.cs`).

```

Detalles importantes:

El skill incluye convenciones del proyecto (framework, estilo, ubicación). Sin eso, el agente genera tests que técnicamente funcionan pero no encajan con el resto del repo. Las convenciones son lo que diferencia un skill genérico de un skill de equipo.

El paso 5 vincula tests con criterios de aceptación. Esto cierra el bucle SDD: la spec define qué cuenta como hecho, los tests verifican que se ha hecho. Si un test no se puede vincular a un criterio, o el criterio era vago o el test sobra.

El skill no toca el código fuente. Esa restricción es importante. Si dejas el skill libre, se mete a "mejorar" el código mientras genera tests. Una cosa cada vez.

Bloque 6 · Skill útil número tres — refactor-con-criterio

El tercero es más sutil y más útil. Refactorizar con IA es tentador y peligroso a la vez: el agente puede mover mucho código muy rápido y dejar el comportamiento ligeramente distinto sin que te enteres. Un skill bien definido pone disciplina en eso.

```
---
name: refactor-con-criterio
description: "Refactoriza un bloque de código manteniendo comportamiento estricto."
triggers:
  - "refactoriza esto"
  - "refactor con criterio"
metadata:
  version: 1
  owner: equipo-backend
---
```

Skill: refactor-con-criterio

Cuándo se aplica
Cuando el usuario pide refactorizar código existente y el código tiene tests que cubren su comportamiento actual.

Pre-condiciones

- Los tests del módulo afectado deben pasar antes de empezar.
- Si no hay tests del comportamiento crítico, AVISA antes de proceder: el refactor sin tests es riesgo aceptable solo si el usuario lo asume explícitamente.

Procedimiento

1. Ejecuta los tests actuales y confirma que pasan.
2. Identifica qué cambios propones y explica brevemente cada uno.
3. Aplica los cambios de uno en uno (no en bloque).
4. Tras cada cambio, ejecuta los tests. Si fallan, revierte ese cambio concreto y reporta. NO sigas con más cambios.
5. Al terminar, ejecuta los tests una vez más y reporta el resultado.
6. NO modifiques la signature de funciones públicas sin avisar.
7. NO añadas dependencias nuevas sin avisar.
8. NO cambies el comportamiento observable. Solo estructura interna.

Post-condiciones

- Los mismos tests siguen pasando.
- No hay nuevas dependencias sin discutir.
- Las signatures públicas son idénticas o el cambio se comunica.

Este skill es defensivo. Cada paso del procedimiento está pensado para que un refactor no se vaya de las manos. Y cuando el modelo se sale de las restricciones (intenta cambiar una signature porque "es más limpio"), el procedimiento le obliga a parar y avisar.

En la práctica este skill te ahorra el bug clásico de "el refactor me ha cambiado un comportamiento sutil que no estaba cubierto por tests y nadie lo va a notar hasta el próximo cliente que llame".

Bloque 7 · Mantenimiento de skills

Aquí está donde la mayoría de los equipos fallan. Crean skills entusiasmados las primeras semanas, los usan, y a los seis meses tienen un cajón de sastre de skills medio útiles, medio rotos, medio desactualizados que nadie revisa.

Tres reglas operativas para que no pase eso:

Cada skill tiene dueño. En el frontmatter, un `owner`. Cuando algo deja de funcionar en el skill, sabes a quién preguntar. Sin dueño, los skills se vuelven nadie's land.

Versiona los skills. Cuando cambies un skill significativamente, sube la versión. Esto te permite saber cuándo se actualizó y por qué. Si el equipo discute "antes funcionaba distinto", la versión te dice cuándo cambió.

Retira skills muertos. Cada cierto tiempo (cada trimestre, por ejemplo), revisa qué skills se han usado en los últimos meses. Los que no, archívalos. Mejor tener seis skills activos que doce skills donde tres están muertos y nueve se confunden entre sí.

Un detalle adicional: los skills no son inmunes a las reglas de redacción del proyecto. Si la descripción del skill incluye palabras-cliché de IA (las típicas de lista negra: adjetivos vacíos, conectores corporativos, frases hechas), el agente las hereda al generar texto. Los skills heredan el estilo del texto que los redacta. Aplica las mismas reglas a las specs de skills que a las specs de feature.

Bloque 8 · Cierre

Tres ideas para llevarte.

Un skill es conocimiento aplicable, no código ejecutable. Lo escribes como manual; el agente lo aplica con criterio. Si lo escribes como rígido paso a paso ignorando el contexto, el skill se vuelve frágil y solo sirve para el caso exacto.

El criterio para crear skill: repetición + estabilidad + beneficio compartido + ahorro real. Los cuatro a la vez. Si falla uno, no hagas skill — escribe el procedimiento en el prompt esta vez.

Los skills son piezas vivas como el código. Tienen dueño, versión y ciclo de retirada. Sin mantenimiento, el cajón se llena de fósiles que confunden al agente más de lo que ayudan.

Glosario rápido del bloque

- **Skill:** procedimiento documentado que el agente carga en contexto cuando ciertos activadores aparecen. Tiene spec, procedimiento y triggers.
- **Activador (trigger):** palabra clave, formato de archivo, tipo de petición que indica al agente que debe cargar un skill concreto.
- **Procedimiento:** pasos del skill que el agente aplica al caso concreto. No es script — el agente sigue decidiendo con criterio dentro del marco.
- **Owner de skill:** persona responsable de mantenerlo al día. Se declara en el frontmatter; cuando algo no funciona, se le pregunta a esa persona.

Qué viene en M3.4

Subagentes. Cuando un trabajo es lo suficientemente grande como para que una sola sesión no le baste, o cuando una tarea tiene fases con distinto contexto cada una, los subagentes permiten delegar piezas concretas a "alguien con menos contexto, más foco". Veremos el patrón orquestador + subagentes, cuándo encaja, y un aviso importante: el subagente puede mentir, y hay que verificar lo que dice contra el filesystem antes de fiarse.

M3.4 · Subagentes — roles especializados

Bloque 0 · Por qué subagentes existen

Si has trabajado con Claude Code en una sola sesión grande, habrás notado un patrón: a partir de cierto punto la sesión empieza a perder fuelle. El contexto se llena, las decisiones tempranas se diluyen, y el agente empieza a confundirse entre lo que pidió el usuario hace dos horas y lo que está pidiendo ahora. Sigue funcionando, pero la calidad cae.

Una forma de aliviar eso es cortar y abrir sesión nueva. Otra, complementaria, es delegar piezas concretas del trabajo a **subagentes**: instancias separadas del agente que reciben una tarea acotada, un contexto mínimo necesario, y devuelven un resultado.

El patrón es viejo en arquitectura de software (orquestador y trabajadores) y vuelve aquí con dos beneficios y un riesgo grande. Beneficios: cada subagente trabaja con contexto limpio sobre su pieza concreta, y la sesión principal no se contamina con detalles que no le incumben. Riesgo: el subagente trabaja con menos contexto y a veces decide cosas que no encajan con la visión global. O peor: a veces te reporta cosas que no son verdad.

En estos veinte minutos: qué es un subagente operativamente, cuándo encaja el patrón orquestador, cómo invocarlos, el aviso de seguridad que se va a repetir dos veces porque importa, y cuándo NO usarlos. Va a ser práctico — los subagentes se entienden usándolos, no leyendo sobre ellos.

Bloque 1 · Qué es un subagente

Un subagente es una instancia del agente que se lanza desde dentro de una conversación principal para que ejecute una tarea acotada, con contexto propio y separado.

Tres rasgos importantes:

Contexto separado. El subagente no comparte la memoria de la sesión principal. Recibe la tarea que se le delega y nada más. Eso es ventaja (no se contamina con ruido) y limitación (no sabe nada que no le digas explícitamente).

Devuelve un resultado al orquestador. Cuando termina, comunica lo que ha hecho — un resumen, los archivos modificados, lo que sea — y el orquestador lo recoge. La sesión principal sigue desde donde estaba, con el resultado en su contexto.

Es del mismo tipo que el agente principal. No es una versión simplificada ni menos capaz. Tiene las mismas herramientas, el mismo modelo, la misma forma de razonar. Lo que cambia es el alcance: el orquestador trabaja en el "qué hay que hacer en este proyecto"; el subagente trabaja en "cómo hago esta pieza concreta".

Imagínatelo así: el agente principal es el tech lead que conoce el proyecto entero. El subagente es el desarrollador que recibe un ticket bien escrito y se concentra en cerrarlo. El tech lead no programa cada ticket él; delega y supervisa. Pero los tickets mal escritos producen entregables malos, y el supervisor que no revisa lo entregado se lleva sorpresas.

Bloque 2 · Patrón orquestador + subagentes — cuándo encaja

No todo trabajo se beneficia de subagentes. El patrón rinde en tres tipos de situación:

Tareas paralelizables. Cuando tienes que hacer N cosas independientes entre sí. Por ejemplo: "Genera tests para estos cinco módulos." Cada módulo es independiente; lanzas cinco subagentes en paralelo, cada uno con su módulo, y al final consolidas. La sesión principal hace de orquestador: lanza, espera, recoge.

Tareas de exploración con scope acotado. Cuando antes de implementar necesitas explorar el código existente: "Busca todos los sitios donde se usa esta función y reporta el patrón de uso." Un subagente con scope explorador hace búsquedas, lee archivos, vuelve con un resumen estructurado. La sesión principal no se llena de los cien archivos que el subagente leyó — solo recibe el informe.

Tareas con contexto muy distinto al de la sesión principal. Si llevas dos horas trabajando en backend y de pronto tienes que hacer un ajuste pequeño en CSS, no quieres que la sesión principal se descentre. Delegas la tarea CSS a un subagente con contexto fresco, recibes el resultado, sigues con backend.

En cualquiera de los tres casos, el orquestador (la sesión principal) mantiene la visión global y los subagentes aportan foco. Si tienes una tarea grande que ENCAJA en uno de los tres patrones, el resultado suele ser mejor que hacerla en sesión única.

Si tu tarea no encaja en ninguno de los tres, probablemente subagentes no son la herramienta. Una tarea pequeña y secuencial se hace mejor en sesión única; una tarea grande sin partes paralelas se hace mejor cortando la sesión en fases.

Bloque 3 · Cómo invocar un subagente

La mecánica exacta depende del cliente. En la mayoría de las versiones de Claude Code, la invocación tiene esta forma:

[En la sesión principal, escribes algo así:]

Lanza un subagente para esta tarea:

Tarea: Generar tests unitarios para el módulo ``src/facturas/calcularVencimiento.ts`` siguiendo el skill ``generar-tests-desde-behavior``.

Contexto necesario:

- La spec en ``specs/factura.md``.
- El código actual del módulo.
- Las convenciones del proyecto (ver `CLAUDE.md` raíz).

Resultado esperado:

- Archivo ``src/facturas/calcularVencimiento.test.ts`` con tests para los cinco criterios de aceptación de la spec.
- Reporte resumido de qué tests se han escrito y qué casos cubren.

Tres elementos que el subagente necesita siempre:

Tarea concreta. No "trabaja en la parte de facturas". Algo específico que se pueda cerrar y verificar.

Contexto mínimo necesario. Qué archivos leer, qué specs aplicar, qué convenciones respetar. Si no se lo das, el subagente va a inferir — y eso es la puerta de los problemas.

Resultado esperado. Qué debe haber al terminar la tarea, y en qué forma. Sin esto, el subagente puede entregar algo bien hecho pero en formato distinto al que esperabas.

Un subagente bien briefeado tarda lo mismo que el agente principal habría tardado en hacer la tarea, con la ventaja de que no consume contexto de la sesión principal. Un subagente mal briefeado tarda más (porque pregunta o asume) y a veces entrega cosas que no servían.

Bloque 4 · El aviso que se repite dos veces — el subagente puede mentir

Ahora viene la parte que importa más de este submódulo entero, y la que más fácil es de pasar por alto.

Un subagente puede reportarte que ha hecho algo que en realidad no ha hecho. O que ha modificado un archivo que en realidad no ha tocado. O que un test pasa cuando en realidad falla. No por mala fe del modelo — el modelo no tiene fe — sino porque su reporte interno se desincroniza con la realidad del filesystem por motivos varios: el subagente cree que escribió un archivo pero hubo un error que no detectó; el subagente reporta lo que tenía

intención de hacer en lugar de lo que hizo; el subagente confunde dos sesiones en su propio resumen interno.

Lo he visto en producción: un subagente reportó que había creado diecinueve archivos. Comprobé contra el filesystem y los diecinueve archivos estaban en su sitio. Pero otro día reportó haber creado seis y solo aparecieron cuatro. Otro día reportó que un cambio compilaba; el `tsc` me dijo lo contrario.

La conclusión operativa es dura y clara: **nunca aceptes el reporte de un subagente sin verificarlo contra el filesystem**. Nunca. Aunque te haya funcionado las últimas cincuenta veces. La verificación cuesta diez segundos; pillar un problema dos semanas después cuesta dos horas y la confianza del equipo.

Bloque 5 · Cómo verificar lo que dice el subagente

La verificación no es un proceso largo. Lo que sí tiene es disciplina. Tres comprobaciones rápidas para cada reporte:

Conteo de archivos. Si el subagente dice "he creado diecinueve archivos", haz un `find` o `ls` y cuenta. Que el número coincida. Si no coincide, hay algo raro.

Apertura de algún archivo aleatorio. No leas todo, pero abre uno o dos al azar. Comprueba que el contenido es lo que esperabas, no un placeholder vacío ni un esqueleto a medias.

Ejecutar lo verificable. Si el subagente dice "los tests pasan", ejecuta los tests tú. Si dice "el código compila", compila. Si dice "el lint no se queja", pasa el lint. Lo que el subagente afirma sobre estado externo, lo confirmas con la herramienta correspondiente.

Esas tres comprobaciones tardan menos de un minuto y te salvan de problemas serios. La disciplina de hacerlo cada vez es lo que diferencia un equipo que usa subagentes en producción de uno que los usa para impresionar y acaba escarmentado.

Una práctica que ayuda: cuando lances un subagente, pon en su brief explícitamente "al terminar, lista los archivos exactos que has creado o modificado, con tamaño y línea final". Eso te da material para verificar. Y si el subagente miente sobre un archivo, suele mentir también sobre el tamaño — que es mucho más fácil de detectar.

Repitiendo la idea por si se pasó por alto: el reporte de un subagente es información sin verificar hasta que la verificas. Trátalo como tratas a un junior nuevo en el equipo: lo respetas, pero compruebas su trabajo antes de mergearlo.

Bloque 6 · Cuándo NO usar subagentes

Cuatro situaciones en las que delegar a subagentes empeora las cosas:

Tarea muy interconectada. Cuando todo lo que haces requiere conocer mucho contexto del resto del proyecto, el subagente con su contexto recortado va a tomar decisiones inconsistentes con lo que la sesión principal sabe. Mejor hacerlo todo en la sesión principal aunque consuma más contexto.

Tarea de exploración sin scope claro. "Explora el repo y dime qué se puede mejorar" no es tarea para subagente. Es tarea para discusión con la sesión principal, que conoce el dominio. Subagentes con scope abierto producen reportes vagos que no accionas.

Tarea pequeña y rápida. Lanzar un subagente para una tarea de tres minutos cuesta más en setup que el ahorro. Hazla en la sesión principal y sigue.

Tarea donde el contexto compartido es la sustancia. Cuando estás iterando una decisión de diseño con discusión activa, no delegas. La discusión es el trabajo; subagente la rompería.

Un buen indicio: si te cuesta escribir el brief del subagente (qué tarea, qué contexto, qué resultado), probablemente subagente no es la herramienta. Cuando el patrón encaja, el brief sale fácil.

Bloque 7 · Cierre

Tres ideas para llevarte.

Un subagente es alguien con menos contexto y más foco. Útil para tareas paralelizables, exploración acotada o contextos muy distintos a la sesión principal. Fuera de esos tres casos, complica más que ayuda.

El reporte de un subagente es información sin verificar. Tres comprobaciones rápidas (conteo, apertura aleatoria, ejecución de lo verificable) bastan para cazar la mayoría de problemas. Sin esa disciplina, los subagentes pasan de ayudar a ser una fuente nueva de errores difíciles de rastrear.

El subagente no es magia: es delegación. Las reglas de delegación humana se aplican igual — brief claro, contexto mínimo necesario, resultado esperado explícito, verificación del entregable.

Glosario rápido del bloque

- **Subagente:** instancia del agente lanzada desde la sesión principal con contexto propio y tarea acotada. Devuelve un resultado y el orquestador lo recoge.

- **Orquestador:** la sesión principal que delega tareas a subagentes y consolida sus resultados. Mantiene la visión global del proyecto.
- **Brief del subagente:** instrucciones explícitas de tarea, contexto y resultado esperado. Sin brief claro, el subagente improvisa.
- **Verificación contra filesystem:** comprobación de que lo que el subagente reporta coincide con lo que de verdad hay en disco. No es opcional.

Qué viene en M3.5

Cómo gestionar sesiones largas y memoria entre conversaciones. Cuándo cortar una sesión, qué persistir antes de cerrarla, cómo retomar al día siguiente sin perder el hilo, y el formato del handoff entre sesiones — que sigue el mismo patrón que el handoff entre chats que vamos viendo en el curso. Pieza pequeña pero que marca diferencia cuando un proyecto vive semanas.

M3.5 · Sesión, contexto y memoria entre conversaciones

Bloque 0 · Por qué este submódulo importa más de lo que parece

Casi todo el material sobre Claude Code asume una sesión: empiezas, trabajas, terminas la tarea. Pero los proyectos reales no caben en una sesión. Un sistema en construcción vive semanas. Una migración compleja dura meses. Un equipo que mantiene un producto está usando el agente todos los días, año tras año.

Si tratas cada sesión como mundo cerrado, vas a empezar de cero cada vez. Y empezar de cero cuando llevas seis semanas con el proyecto significa explicarle al modelo la mitad del contexto antes de hacer nada útil. Es lento, es frustrante, y produce decisiones inconsistentes entre sesiones.

En este submódulo: por qué las sesiones tienen que terminar (no es opcional), qué señales indican que toca cortar, qué guardas antes de cerrar, cómo redactas el handoff que la siguiente sesión va a leer, y dónde encaja la memoria persistente del proyecto (los `CLAUDE.md` y similares). Veinte minutos prácticos. Si tu proyecto va a vivir más de una semana, este submódulo te ahorra horas.

Bloque 1 · Síntomas de una sesión que se agota

Una sesión saludable es la que reconoce cuándo debe morir. Cinco señales que indican que la actual está llegando al límite:

El agente repite cosas que ya habíais discutido. Te explica de nuevo qué hace una función, te recuerda una decisión que tomasteis hace una hora, propone un enfoque que ya descartasteis. Eso es contexto compactándose mal: la memoria temprana se está perdiendo.

Las decisiones se vuelven incoherentes. En el archivo A usó tabs; ahora propone espacios en el archivo B. Antes seguía las convenciones del repo; ahora propone una librería que el equipo ya descartó. Sin contexto fresco, las pequeñas decisiones se desacoplan unas de otras.

El agente pide aclaraciones sobre cosas que ya tenía. "¿Qué framework usáis?", "¿Dónde guardamos los tests?", "¿Cómo nombráis los archivos?". Si esas preguntas las contestaste hace tres horas en la misma sesión, el contexto se está perdiendo.

La respuesta tarda notablemente más. Las sesiones largas se ralentizan porque cada turno tiene que procesar más contexto acumulado. Cuando notas que cada respuesta tarda el doble que

al inicio, es síntoma de saturación.

Empiezas a fiarte menos. Subjetivo pero real. Después de muchas horas con la misma sesión, tu instinto te dice "no estoy seguro de que esté siguiendo bien". Hazle caso: cortar y abrir nueva sesión casi siempre arregla esa sensación.

Como referencia operativa: la mayoría de las sesiones rinden bien entre una y tres horas. Pasadas las tres horas con trabajo activo (no contando pausas), considera cortar. Pasadas las cinco horas, casi siempre vale más cerrar.

Bloque 2 · Cuándo cortar — criterios prácticos

Más allá de los síntomas, hay momentos buenos y momentos malos para cortar una sesión. Tres criterios:

Corta al cerrar una unidad de trabajo. Has terminado una feature, has cerrado una spec, has dejado una pieza lista para PR. Cuando una pieza está completada, es el momento natural de terminar la sesión y retomar fresca cuando empiece la siguiente pieza. Eso ahorra contexto contaminado entre tareas distintas.

Corta antes de un descanso largo. Si vas a parar más de dos horas (comida, reunión, fin de jornada), cierra la sesión. Cuando vuelvas, una sesión fresca con un buen handoff rinde más que reabrir la sesión muerta.

No cortes en mitad de un razonamiento delicado. Si estáis en plena discusión de diseño compleja, terminar la sesión es perder la mitad del hilo. Mejor seguir hasta cerrar esa parte y entonces cortar. La sesión nueva no va a recuperar el estado intermedio del razonamiento por mucho handoff que escribas.

No te obsesiones con la duración. Una sesión bien gestionada de cuatro horas rinde más que cinco sesiones de cuarenta minutos. La métrica no es el tiempo: es la calidad de las decisiones. Cuando sientes que las decisiones se degradan, corta. Cuando todo fluye, sigue.

Bloque 3 · Qué guardar antes de cerrar

Lo que pasa dentro de una sesión muere con la sesión. Lo que persiste fuera vive lo que el archivo viva. La pregunta clave antes de cerrar: ¿qué información de las últimas horas vale para mañana?

Categorías típicas que merecen guardarse:

Decisiones de diseño tomadas. Si en esta sesión decidisteis "usar PostgreSQL en lugar de MongoDB para esto y por estas razones", esa decisión tiene que vivir en un archivo (la propia spec, un ADR, un comentario en el código relevante). Sin escribirla, la siguiente sesión va a re-debatir lo mismo.

Patrones descubiertos. Si en esta sesión salió que "este proyecto siempre nombra los archivos así" o "los tests siempre van en esta ubicación", añádelo a `CLAUDE.md` para que cualquier sesión futura lo herede.

Estado actual del trabajo. Qué se ha hecho, qué queda. Esto va al handoff, no al código.

Cosas que NO funcionaron. Si probasteis algo y se descartó por buenas razones, deja constancia. Sin eso, la próxima sesión va a proponer la misma idea otra vez y vas a tener que repetir la explicación.

Convenciones nuevas establecidas. Si en esta sesión emergió "preferimos imports relativos a absolutos en este módulo", añádelo a la documentación del repo.

Lo que NO hace falta guardar: el detalle paso a paso de lo discutido, los intentos intermedios, las decisiones triviales. La sesión es para trabajar; lo persistido es para el futuro lector.

Bloque 4 · El handoff y el prompt de retomada

El handoff es el documento que escribes al final de cada sesión productiva para que la siguiente lo lea al empezar. Su estructura típica:

```

# Handoff sesión [fecha]

## Contexto
Proyecto [X]. Trabajamos en [pieza Y], según `specs/[archivo].md`.

## Estado al cierre
- Implementado: [lista de archivos/funciones cerradas].
- En curso: [lo que está a medias, en qué archivo, qué falta].
- Pendiente: [lo que queda y por dónde se retoma].

## Decisiones de esta sesión
- [Decisión 1 y motivo en una línea].
- [Decisión 2 y motivo].

## Cosas que NO funcionaron
- [Idea descartada y por qué – para no reproponerla].

## Próximo paso al retomar
[Acción concreta: "ejecutar tests pendientes", "completar la función Z",
"discutir el diseño del módulo W con el equipo antes de seguir"].

## Prompt de retomada
[Texto listo para pegar al inicio de la siguiente sesión.]

```

Esa última sección es la más importante: **el prompt de retomada**. Es un texto autocontenido que la siguiente sesión va a leer como primera entrada. Tiene que incluir lo que la sesión nueva necesita para arrancar: dónde están los archivos relevantes, cuál fue el último estado, qué hay que hacer ahora.

Un prompt de retomada típico:

```

Retomamos trabajo en SDDCC, sigue el handoff en `_handoff-sesion-2026-05-28.md`.

Contexto: proyecto en `c:\w\repos\bcursos\Desarrollos\SDDCC ...`. Última
sesión terminó implementando la función `calcularFechaVencimiento` siguiendo
la spec en `specs/factura.md`. Quedan los tests de los casos límite por
escribir (criterios 4 y 5 de la spec).

Reglas vivas del curso: invocar escritura-humana antes de redactar prosa,
R1 (sin nombres de cliente en slides), R5/15.1 (verificar versiones).

Próximo paso: completar `src/facturas/calcularVencimiento.test.ts` con los
tests faltantes. Después ejecutar la suite completa y reportar.

```

Tres rasgos de un buen prompt de retomada:

Es autocontenido. No requiere que el usuario explique nada de viva voz a la sesión nueva. Todo lo necesario está en el texto.

Apunta a archivos específicos. "El handoff está en X", "la spec está en Y", "el código está en Z". Cuando la sesión nueva carga, sabe a qué archivos ir.

Incluye las reglas de proceso si las hay. Si el proyecto tiene un `CLAUDE.md` con reglas, eso ya se carga automáticamente, pero recordar las reglas vivas en el prompt ayuda a fijarlas como contexto explícito desde el primer turno.

Bloque 5 · CLAUDE.md y memoria persistente del proyecto

Por encima del handoff de sesión existe la memoria persistente del proyecto. Lo que vive ahí no es de una sesión: es del proyecto entero, válido para cualquier sesión que se abra mientras el proyecto exista.

El archivo canónico es `CLAUDE.md` en la raíz del repo (y, opcionalmente, `CLAUDE.md` por subdirectorios para contexto local). El agente lo lee automáticamente al empezar cualquier sesión.

Qué pertenece ahí:

Convenciones estables del proyecto. Nombres de archivos, estructura de carpetas, framework de testing, librerías clave, estilo de commits.

Reglas de comportamiento del agente. "No modifiques estos archivos sin avisar." "Usa nullable reference types activos y C# 13 idiomático." "Antes de añadir un paquete de NuGet, pregunta."

Decisiones de diseño estructurales. "El sistema sigue arquitectura hexagonal." "Los eventos van por la cola X." "Los datos sensibles no salen del módulo Y."

Referencias importantes. Enlaces a documentación clave, a specs activas, a herramientas internas.

Qué NO pertenece:

Detalles operacionales que cambian. El estado de un sprint, el ticket que toca esta semana, la asignación de quién hace qué. Eso vive en la herramienta de proyecto, no en `CLAUDE.md`.

Información sensible. Credenciales, claves, secretos. `CLAUDE.md` vive en el repo: trátalo como código público dentro del equipo.

Historia de decisiones efímeras. El `CLAUDE.md` no es un diario. Si necesitas registro histórico, ADRs. Si es decisión actual vigente, `CLAUDE.md`.

Un detalle de mantenimiento: el `CLAUDE.md` también necesita owner. Si no, en seis meses está desactualizado y nadie sabe si confiar en lo que dice. Asignar responsabilidad de revisión periódica resuelve eso.

Bloque 6 · Errores comunes con sesiones largas

Cuatro patrones que retrasan a la gente al aprender a gestionar sesiones:

Mantener una sesión abierta días. "Es mi sesión del proyecto, la dejo abierta." Para sesiones de menos de un día, vale. Para sesiones que cruzan días, no — el contexto se desincroniza con cambios externos (commits de otros, modificaciones de archivos fuera de la sesión) y la sesión empieza a operar sobre una versión imaginaria del proyecto.

Confiar en la memoria del agente entre sesiones. "Lo discutimos ayer." No, lo discutiste en una sesión que ya no existe. La nueva sesión no sabe nada que no esté escrito. Si lo discutido ayer importa hoy, está en algún archivo del repo o se ha perdido.

Handoffs vagos. "Estamos trabajando en facturas." Eso no es handoff: es un titular. Un handoff útil dice qué se ha hecho, qué queda, dónde están los archivos relevantes y qué viene a continuación. Si tu handoff cabe en una línea, falta sustancia.

No releer el handoff al retomar. El handoff lo escribes para una versión de ti que va a empezar cansada o distraída. Si al abrir la sesión nueva no lees el handoff y empiezas a teclear desde memoria, has hecho el trabajo de redactarlo para nada.

Bloque 7 · Cierre

Tres ideas para llevarte.

Las sesiones mueren; los proyectos viven. Lo que de verdad importa es lo que persiste fuera de la sesión: handoffs, `CLAUDE.md`, decisiones registradas en specs y código. Esa persistencia es la diferencia entre un proyecto que avanza coherente y uno que reinvente lo mismo cada semana.

Cortar sesiones es disciplina, no debilidad. Una sesión fresca con buen handoff rinde más que una sesión maratónica con contexto saturado. Cinco minutos de handoff te ahorran media hora de explicaciones al retomar.

`CLAUDE.md` es el contrato del proyecto con el agente. Las cosas que valen para cualquier sesión del proyecto entero viven ahí. Las cosas que valen para una pieza concreta viven en su spec. Las cosas que valen para hoy y mañana viven en el handoff. Tres horizontes, tres soportes distintos.

Glosario rápido del bloque

- **Handoff de sesión:** documento que se escribe al cerrar una sesión productiva, con estado, decisiones, pendientes y prompt de retomada.

- **Prompt de retomada:** texto autocontenido que la siguiente sesión carga como primera entrada. Apunta a los archivos relevantes y resume el estado.
- **CLAUDE.md** : archivo de memoria persistente del proyecto, leído automáticamente por el agente al inicio de cada sesión. Contiene convenciones, reglas de comportamiento y decisiones estructurales.
- **ADR (Architecture Decision Record):** documento corto que registra una decisión de arquitectura para el futuro. Complementa al **CLAUDE.md** cuando hay decisiones grandes.

Qué viene en M3.6

Práctica integrada. Tomamos la spec del M2.5 (la que escribiste en la práctica del módulo anterior) y la llevamos por Plan Mode, skills, y un subagente — hasta tener la primera implementación funcionando. No es una práctica enorme; es la práctica donde se pegan todas las piezas que has visto en M3.1 a M3.5. Al terminar M3, vas a tener tu spec convertida en código real, generada con disciplina SDD.

M3.6 · Práctica integrada — del intent a la primera implementación

Bloque 0 · Lo que vamos a hacer

Llevas con el curso desde el M1. Has aprendido que las specs importan (M1), cómo se escriben con anatomía y revisión (M2), y cómo Claude Code funciona como motor con Plan Mode, skills, subagentes y memoria entre sesiones (M3.1–M3.5).

En este submódulo final del M3 toca pegar todas las piezas. Vas a coger la spec que escribiste y revisaste en la práctica del M2.5, y la vas a llevar a una primera implementación funcionando. No es un ejercicio de "ver qué código sale". Es un ejercicio de aplicar el método entero: preparar la sesión, planificar antes de ejecutar, supervisar la ejecución, usar skills donde encajen, delegar a subagentes lo paralelizable, y cerrar con un handoff útil.

Lo importante de la práctica no va a ser el código resultante — ese será correcto si has hecho los pasos bien. Lo importante son los puntos donde tú paras y decides algo. Esos puntos son la disciplina que diferencia un dev que trabaja con SDD de uno que solo lo dice.

Recorrido en seis pasos. Cada paso con su propósito y con las decisiones que ahí te tocan.

Bloque 1 · Paso 1 — preparar la sesión

Antes de tocar nada, deja la sesión lista. Tres cosas a comprobar:

El `CLAUDE.md` del proyecto está al día. Si trabajas en un repo propio, abre el `CLAUDE.md` raíz y mira si refleja las convenciones del proyecto. Si trabajas en uno limpio para la práctica, créalo con cuatro líneas básicas: nombre del proyecto, lenguaje y framework, estructura de carpetas, regla principal (por ejemplo "C# 13, .NET 10, tests con xUnit v3, nullable reference types activos").

La spec está en el repo. Tu spec del M2.5 vive en `specs/[nombre].md`. Si todavía está en un editor sin guardar, guárdala y comprueba que está commitada o al menos staged. La sesión va a leerla; tiene que existir.

El working tree está limpio o tiene cambios localizados. No empieces una sesión grande sobre un repo con cinco features a medias. La sesión va a tocar archivos; mejor que el punto de partida sea reconocible.

Decisión que te toca aquí: ¿abres la sesión en Plan Mode por defecto o en ejecución libre? Para esta práctica, **Plan Mode por defecto**. Es el modo que vamos a usar todo el rato. Si tu cliente no lo soporta como default, configura el atajo para activarlo rápido.

Bloque 2 · Paso 2 — invocar Plan Mode con la spec

Con la sesión abierta y Plan Mode activo, tu primera petición es la siguiente:

```
Implementa lo descrito en specs/[nombre-de-tu-spec].md.
```

Pre-requisitos:

- Sigue las convenciones del proyecto (ver CLAUDE.md).
- Genera tests para los criterios de aceptación de la spec.
- Antes de ejecutar nada, muéstrame el plan.

Tres detalles del prompt:

Apunta a la spec por path, no copias el contenido. El agente va a leerla. Si copias el contenido, duplicas información y gastas contexto de más. Una referencia y listo.

Recuerda las pre-condiciones. Aunque el `CLAUDE.md` ya esté cargado, recordarlas en el prompt explícitamente fija que vas a vigilar su cumplimiento.

Cierra con la instrucción explícita de Plan Mode. "Antes de ejecutar nada, muéstrame el plan." Aunque tengas Plan Mode activado, repetirlo en el prompt asegura que el modelo lo respeta.

El agente va a tardar entre quince segundos y dos minutos en devolver el plan, según el tamaño de la spec. Cuando aparezca, no leas en diagonal. Vamos al paso 3.

Bloque 3 · Paso 3 — leer e iterar el plan

Aquí está la decisión más importante de la práctica. Y la que más fácil es de saltarse.

Aplica las cuatro comprobaciones del M3.2:

¿El alcance encaja con la spec? Mira la lista de archivos. Si son los que esperabas según la spec, bien. Si aparecen archivos que no deberían tocarse (modificaciones en archivos no relacionados, instalación de dependencias nuevas), para y pregunta. Si la spec era pura, el plan debería ser puro.

¿Las decisiones tomadas son las tuyas? Si el plan menciona elecciones de librería, patrón o estilo que no están en la spec, lee con atención. Probablemente sean razonables, pero quieres ser quien firma cada una, no descubrirlas después.

¿La secuencia tiene sentido? Tests antes o después de implementación, según preferencia del equipo. Configuración antes de uso. Migraciones antes de cambios de schema. Si algo va fuera de

orden lógico, ajústalo.

¿Los riesgos identificados resuenan? Si el plan dice "no estoy seguro de X, ¿quieres que lo confirme contigo?", es una invitación a parar y decidir antes de empezar.

Si el plan está al 80%, itera con instrucciones concretas: "Limita el alcance a estos archivos." "Usa esta librería en lugar de la que propones." "Mueve el paso 3 antes del 2." Dos o tres iteraciones suelen bastar.

Decisión que te toca aquí: cuándo dices "adelante". Esa palabra es la firma del plan. No la digas si todavía tienes dudas estructurales. Sí la dices cuando el plan recoge lo que tú habrías hecho si lo escribieras a mano, con los matices que aportan tu visión y la del agente combinados.

Si llevas seis iteraciones y el plan sigue sin convencerte, el problema casi siempre está en la spec, no en el plan. Cierra Plan Mode, vuelve a la spec, mejórala, y vuelve a empezar.

Bloque 4 · Paso 4 — ejecutar y supervisar

Plan firmado, el agente arranca el bucle ReAct: lee archivos, escribe archivos, ejecuta comandos. Tu papel cambia: pasas de "decidir qué se hace" a "supervisar cómo se hace".

Tres cosas a vigilar durante la ejecución:

Los archivos que toca coinciden con el plan. Si el agente se desvía y toca un archivo que no estaba en el plan, para. Pregunta por qué. A veces es razonable (descubrió algo que el plan no había previsto); a veces se está pasando. La decisión es tuya.

Los errores que aparecen. Si un test falla, si un comando devuelve error, si un import no resuelve, el agente va a intentar arreglarlo por su cuenta. Las primeras veces deja que lo haga, pero observa. Si se mete en una espiral de intentos sin éxito (más de tres correcciones sobre el mismo error), interrumpe y pregunta — a veces el problema está fuera de su visión.

El tiempo que tarda. Si una pieza concreta del plan estima cinco minutos y lleva veinte, algo va mal. O el plan subestimó la complejidad, o el agente está perdido. Interrumpe y replantea.

Esta fase es la más activa del método. No es "darle al enter y volver dentro de media hora". Es supervisar como supervisarías a alguien con menos contexto que tú: dejándole hacer pero leyendo lo que produce.

Decisión que te toca aquí: cuándo interrumpir. Aprender a sentir cuándo el agente está bien encaminado y cuándo se está atascando es parte del oficio. Las primeras prácticas vas a interrumpir más de la cuenta; con tiempo vas a saber cuándo dejarle resolver y cuándo cortar.

Bloque 5 · Paso 5 — invocar skills si encajan

Durante la ejecución pueden aparecer momentos donde un skill ayuda. Tres ejemplos típicos:

Antes de ejecutar la implementación, pasar la spec por el skill `revisar-spec-sdd`.

Aunque ya la revisaste en el M2.5, una segunda revisión justo antes de implementar caza huecos que se han colado o que aparecen al pensar en el código concreto.

Tras la implementación, pasar el código por un skill de revisión de calidad (si lo tienes). Cosas que el skill detecta: complejidad excesiva, código repetido, nombres poco descriptivos.

Para generar tests, invocar el skill `generar-tests-desde-behavior`. Si la spec tiene una sección Behavior bien escrita, el skill traduce los casos en tests sin que tengas que explicarle cada vez cómo.

No invoques skills por inercia. Si la situación no lo pide, no le añades valor. Pero si una de las tres situaciones aparece, los skills convierten un trabajo de quince minutos en uno de cinco. Esa es la diferencia entre tener skills bien afinados y no tenerlos.

Decisión que te toca aquí: cuándo merece invocar skill y cuándo es ruido. Como con todo lo demás, criterio que viene con uso.

Bloque 6 · Paso 6 — subagente para tests (opcional)

Si tu spec tiene varios módulos a testear o varios casos relativamente independientes, este es buen momento para experimentar con un subagente. El brief:

Lanza un subagente para esta tarea:

Tarea: Generar tests unitarios para `[path-del-código]`, siguiendo la sección Behavior de `specs/[spec].md`.

Contexto necesario:

- La spec completa.
- El código actual del módulo.
- Las convenciones de tests del proyecto (ver CLAUDE.md).

Resultado esperado:

- Archivo `[path-del-test]` con tests para los N criterios de aceptación.
- Reporte de qué tests se han escrito, agrupados por criterio.

Al terminar, lista los archivos exactos creados o modificados, con tamaño.

Cuando el subagente devuelva su reporte, **aplica las tres comprobaciones del M3.4:** conteo de archivos, apertura de uno al azar, ejecución de tests. Sin esa verificación, no aceptes el

resultado.

Decisión que te toca aquí: ¿es esta tarea suficientemente acotada y paralelizable para subagente? Si dudas, no. Subagente para una tarea pequeña es overengineering. Si la tarea es claramente "generar tests para módulos independientes", sí.

Bloque 7 · Paso 7 — cerrar con handoff

Cuando todo está implementado, tests pasan y la spec se cumple, no cierres la sesión sin handoff. Aunque el trabajo esté hecho, deja el handoff para que la siguiente sesión sepa retomar (o para que tú mismo lo retomes la semana que viene sin reconstruir desde memoria).

Estructura mínima del handoff para esta práctica:

```
# Handoff sesión [fecha]

## Trabajo cerrado
- Implementada [nombre de la pieza] siguiendo `specs/[spec].md`.
- Tests de los N criterios de aceptación, todos en verde.
- Cambios en: [lista de archivos].

## Decisiones tomadas
- [Decisión 1 que no estaba explícita en la spec].
- [Decisión 2].

## Lo que NO se ha tocado
- [Áreas relacionadas que no entran en esta pieza].

## Próximo paso
- Abrir PR contra `main` con estos cambios.
- [0, si aplica, integrar con otra pieza que viene después.]
```

Decisión final que te toca aquí: ¿hay algo que se haya descubierto durante la implementación que debería actualizar el `CLAUDE.md`? Una convención implícita que ha emergido, un patrón nuevo, una regla que vale para el futuro. Si lo hay, añádelo antes de cerrar la sesión.

Bloque 8 · Reflexión — dónde decidiste tú

Mira atrás los siete pasos. En cada uno hubo al menos una decisión que tomaste tú, no el agente:

1. **Cómo dejaste preparada la sesión** (qué reglas activas, qué Plan Mode por defecto).
2. **Cómo formulaste la primera petición** (qué incluiste, qué dejaste implícito).
3. **Cuándo dijiste "adelante" al plan** (la firma).
4. **Cuándo interrumpiste o dejaste seguir** durante la ejecución.

5. **Qué skills invocaste y cuáles no.**

6. **Si usaste subagente o no**, y cómo lo briefeaste si lo usaste.

7. **Qué guardaste en el handoff y qué dejaste en la sesión muerta.**

Esos siete puntos son SDD aplicado. El código que ha salido es consecuencia de esas siete decisiones. Si las hiciste con criterio, el código va a estar bien. Si las saltaste o las hiciste por inercia, el código puede compilar pero algo en él te va a morder dentro de tres semanas.

Esa es la lección del módulo: trabajar con Claude Code de verdad no es saber qué prompts usar. Es saber dónde parar y decidir. Las herramientas (Plan Mode, skills, subagentes, memoria persistente) son las palancas que te dan los puntos de parada. Tu trabajo es usarlos.

Cierre del M3 y enlace a M4

Has llegado al final del M3 con la pieza completa: spec del M2.5 → Plan Mode → ejecución supervisada → tests pasando → handoff escrito. Si has hecho los siete pasos con disciplina, tienes una pieza de software construida con SDD en un entorno controlado. Repetir esto cuatro o cinco veces más sobre piezas reales de tu trabajo es lo que hace que el método se vuelva natural.

En el M4 entramos en el cierre del bucle: hooks de validación. Hasta ahora, las protecciones del proceso vivían en tu cabeza — tú leías el plan, tú interrumpías la ejecución, tú decidías cuándo activar skills. Eso funciona, pero depende de tu atención. Los hooks son la forma de automatizar las protecciones para que algunas pasen sin que tengas que estar mirando: validación contra spec automática tras cada edición, bloqueo de cambios fuera de scope, alertas si el agente toca archivos prohibidos. Veremos PreToolUse, PostToolUse, SessionStart y el resto del catálogo, con ejemplos concretos que puedes copiar a tu proyecto.

Glosario rápido del bloque

- **Práctica integrada:** ejercicio donde se aplican varias piezas del método a la vez sobre un caso real, no aislando técnicas.
- **Firma del plan:** el momento explícito en que el dev dice "adelante" a un plan en Plan Mode. Es la decisión más importante de la sesión.
- **Punto de parada:** momento del proceso donde el dev decide algo no trivial. SDD bien hecho tiene varios puntos de parada distribuidos; SDD mal hecho colapsa en un solo "darle al enter".
- **Verificación de subagente:** las tres comprobaciones (conteo, apertura aleatoria, ejecución) que se aplican cuando un subagente entrega un resultado.

Estado al cerrar el M3

Has visto: - Modelo mental de Claude Code (M3.1). - Plan Mode como puente entre spec y código (M3.2). - Skills como procedimientos reutilizables (M3.3). - Subagentes como roles especializados (M3.4). - Gestión de sesión y memoria entre conversaciones (M3.5). - Práctica integrada de todo lo anterior (M3.6).

Las piezas básicas para trabajar con Claude Code en modo SDD ya están. El M4 añade la red de seguridad automática que evita los errores que tu atención no puede pillar siempre.

PARTE 4

Hooks y validación automática

M4.1 · Por qué hooks importan en SDD

Bloque 0 · La atención humana no escala

El M3 entero giró sobre tu atención: tú lees el plan, tú interrumpes la ejecución, tú verificas el reporte del subagente. Eso funciona para tareas concretas, en sesiones controladas, cuando vas con la cabeza fresca. Funciona menos a partir de la cuarta hora de la jornada, cuando tu atención se ha consumido en cinco reuniones y un par de interrupciones.

Aquí está la incomodidad: SDD bien hecho depende de disciplina sostenida, y la disciplina humana es un recurso finito. La sesión número diecisiete del mes con Claude Code, a las cinco de la tarde de un jueves, no va a tener la misma calidad de supervisión que la primera del lunes a las nueve de la mañana. Si tu método solo funciona cuando estás al 100%, tu método funciona la mitad del tiempo.

Los **hooks** son la respuesta a ese problema. Son protecciones automáticas — código que se ejecuta antes o después de ciertas acciones del agente — que siguen funcionando aunque tú dejes de mirar. No reemplazan tu supervisión; la complementan. Lo que tú pillarías si estuvieras atento, los hooks lo pillan siempre.

En quince minutos vamos a ver qué problema concreto resuelven, qué pinta tienen, el catálogo básico de tipos de hook que Claude Code expone, y la filosofía que toca adoptar: validar siempre, no confiar nunca — ni siquiera en ti mismo de hace dos horas.

Bloque 1 · El problema concreto que los hooks resuelven

Sin hooks, el agente puede hacer cosas que tú no querías y que tú no detectaste porque no estabas mirando. Cinco situaciones reales que veo en producción:

Tocar archivos fuera del scope acordado. El plan decía "modifica `src/facturas/`". El agente, mientras tanto, decidió "mejorar" un import en `src/comunes/` que vio de pasada. El cambio compila, los tests no lo cubren porque no estaba previsto, y tres semanas después aparece un comportamiento raro que nadie sabe explicar.

Instalar dependencias por su cuenta. El plan no decía nada de dependencias nuevas. El agente, durante la ejecución, decidió que necesitaba un paquete de NuGet "para resolver esto más limpiamente" y ejecutó `dotnet add package`. Te quedas con una dependencia en producción que nadie aprobó.

Ejecutar comandos destructivos. El agente, intentando arreglar un error de tests, decide ejecutar `rm -rf bin obj && dotnet restore`. Lo hace en una sesión donde tenías cambios locales en `bin/` (archivos generados que estabas usando temporalmente como referencia). Trabajo perdido.

Tocar archivos sensibles. El agente cambia algo en `.env`, en `config/secrets.yaml`, o en cualquier archivo que tiene credenciales. Lo hace porque "el código de ejemplo en la spec apuntaba a esas variables". Riesgo de seguridad real.

Commitear sin que tú quieras. El agente, intentando ser útil, hace `git add -A && git commit`. Si tu working tree tenía cinco features a medias, acabas con un commit gigante que mezcla cinco cosas.

Las cinco situaciones tienen algo en común: son acciones que el agente toma por su cuenta, durante el bucle de ejecución, sin pasar por Plan Mode (porque el plan no las preveía explícitamente). Tu supervisión podría pillarlas, pero tu supervisión no es 24/7. Los hooks sí.

Bloque 2 · Qué es un hook, en operativo

Un hook es **una pieza de código que el agente ejecuta automáticamente cuando ocurre un evento determinado en su ciclo de vida**. El agente no decide si ejecutar el hook; el hook se ejecuta siempre que el evento sucede. Y dependiendo de lo que el hook devuelva, la acción del agente continúa, se modifica o se cancela.

Estructura típica de un hook:

```
[Evento que dispara el hook]
↓
[Código del hook se ejecuta]
↓
[Resultado: permitir / modificar / bloquear]
↓
[El agente continúa según el resultado]
```

El código del hook es tuyo (o de tu equipo). Suele vivir en una carpeta de configuración del proyecto (`.claude/hooks/` , según el cliente). Puede ser un script en bash, en Python, en lo que el cliente soporte. Lo que importa es que se ejecuta antes o después de la acción del agente, y puede intervenir.

Tres ejemplos rápidos para fijar la idea:

Hook que bloquea cambios en archivos sensibles. Antes de cada escritura de archivo, el hook comprueba la ruta. Si está en una lista de archivos protegidos (`.env` , `secrets/` , etc.),

bloquea la acción y devuelve un mensaje al agente: "Este archivo no se puede modificar sin permiso humano explícito."

Hook que ejecuta tests tras cada edición de código. Después de cada modificación en `src/`, el hook ejecuta `npm test`. Si los tests fallan, lo reporta. El agente entonces puede decidir qué hacer (revertir, corregir, pedir intervención).

Hook que registra todas las acciones. Cada acción del agente (lectura, escritura, comando shell) se anota en un log. Eso te da histórico para auditar qué pasó en una sesión cuando aparezca un problema raro.

Los tres son ejemplos pequeños. Un proyecto serio puede tener una decena de hooks que cubren distintos aspectos del control.

Bloque 3 · El catálogo básico de eventos

Claude Code expone varios eventos sobre los que enganchar hooks. Los nombres exactos varían según el cliente, pero el patrón es estable. Los principales:

PreToolUse. Se dispara antes de que el agente ejecute una herramienta (leer archivo, escribir archivo, ejecutar comando). Es el evento más usado: aquí es donde puedes inspeccionar qué va a hacer el agente y decidir si lo permites.

PostToolUse. Se dispara después de que la herramienta termina, con el resultado. Útil para validaciones (¿el archivo escrito tiene la estructura esperada?), para auditoría (¿qué hizo exactamente?), o para reaccionar (si el resultado contiene un error, hacer algo).

SessionStart. Se dispara al inicio de cada sesión. Útil para validar que el entorno está en estado correcto antes de empezar: que no haya cambios sin commitar, que las dependencias estén instaladas, que el repo esté en la rama esperada.

SessionEnd. Se dispara al cerrar la sesión. Útil para limpieza, para generar el handoff automático, para backup de memoria, para asegurar que no se queda nada a medias.

UserPromptSubmit. Se dispara cuando el usuario manda un nuevo prompt. Útil para enriquecer el prompt con contexto adicional (recordatorios, reglas, estado).

Hay más, según el cliente. Pero con estos cinco se cubre el 90% de los casos prácticos. Y el patrón general es el mismo: identificas el evento, escribes el hook, configuras el cliente para que lo dispare, y a partir de ahí se ejecuta sin que tú tengas que hacer nada.

Bloque 4 · Filosofía — validar siempre, no confiar nunca

Aquí entra la parte filosófica, que importa más que el catálogo técnico. La pregunta operativa: ¿en quién confías cuando trabajas con Claude Code?

La respuesta tentadora es "en el modelo, dentro de unos límites". Es razonable, y de hecho lo haces todos los días — si no confiaras nada, no le pedirías que escriba código. Pero la confianza tiene rangos, y los hooks viven en el rango más exigente: las cosas que, si fallan, son caras de arreglar.

Cosas en las que sí puedes confiar al modelo razonablemente: - Lectura de código existente (no daña nada). - Generación de código nuevo en archivos previstos por el plan. - Generación de tests para una spec dada.

Cosas en las que **no confíes sin verificación**: - Cualquier modificación de archivos sensibles (configs, secretos, infraestructura). - Cualquier comando que toque estado externo (`git push`, `npm publish`, llamadas a API que cambian cosas). - Cualquier eliminación (`rm`, `git checkout -`, borrado de tabla). - Cualquier instalación o cambio de dependencias.

La regla informal: **lo reversible tiene la confianza por defecto; lo irreversible no**. Si te equivocas en algo reversible (un archivo mal escrito), `git checkout` lo arregla. Si te equivocas en algo irreversible (un `push --force`, un borrado de tabla en producción), no hay marcha atrás barata.

Los hooks viven en la frontera entre lo reversible y lo irreversible. Lo reversible no lo tocan; lo irreversible lo bloquean por defecto y exigen confirmación explícita.

Esta filosofía no es paranoica. Es asimétrica. El coste de tener un hook que bloquea de más es una molestia ocasional: el agente te dice "no puedo, esto está bloqueado", tú lo desbloqueas manualmente, sigues. El coste de NO tener el hook es a veces catastrófico: producción caída, datos perdidos, credenciales filtradas. Como la asimetría es enorme, la elección racional es bloquear de más, no de menos.

Bloque 5 · Hooks y Plan Mode son complementarios

Una confusión común: si tienes Plan Mode, ¿hacen falta hooks?

La respuesta es sí, porque cubren cosas distintas.

Plan Mode pilla los errores de **alcance**: lo que el agente piensa hacer (qué archivos tocar, qué decisiones tomar) lo ves antes de empezar y puedes corregirlo. Es preventivo y consciente.

Los hooks pillan los errores de **ejecución**: lo que el agente hace mientras está en el bucle, aunque no estuviera explícitamente en el plan. Son reactivos y automáticos.

Un ejemplo concreto. El plan dice "modifica `src/facturas/calcularVencimiento.ts`". Tú lo lees, lo apruebas. Durante la ejecución, el agente decide que necesita modificar también `src/comunes/utils.ts` porque encontró un bug que le molesta. El plan no preveía eso. Sin hooks, lo modifica y tú te enteras tres semanas después. Con un hook bien configurado ("no modifiques archivos fuera del scope explícito del plan"), el cambio se bloquea y se reporta.

Plan Mode y hooks juntos te dan dos capas de protección: la primera es tu lectura del plan; la segunda es la validación automática durante la ejecución. Si una capa falla (porque estabas distraído, porque el plan era ambiguo en ese punto), la otra coge el balón.

Para producción sería recomendando los dos siempre. Para prototipos rápidos, Plan Mode solo puede bastar. La decisión es de criterio: cuanto mayor el coste de los errores, más capas.

Bloque 6 · Cierre

Tres ideas para llevarte.

Tu atención es un recurso finito y los hooks son la red que sigue activa cuando esa atención se cae. No reemplazan tu supervisión: la complementan en los momentos en que no puedes estar al 100%.

El catálogo básico (PreToolUse, PostToolUse, SessionStart, SessionEnd, UserPromptSubmit) cubre el 90% de los casos. La mayoría del trabajo de hooks ocurre en PreToolUse — antes de que el agente actúe sobre algo importante.

Validar siempre, no confiar nunca, pero con criterio asimétrico: lo reversible se permite por defecto, lo irreversible se bloquea por defecto. La asimetría es lo que hace que esta política sea sostenible — no te entorpece en lo normal, y te salva en lo grave.

Glosario rápido del bloque

- **Hook**: pieza de código que el agente ejecuta automáticamente cuando ocurre un evento de su ciclo de vida. Tuyo, no del modelo.
- **PreToolUse**: evento que se dispara antes de que el agente ejecute una herramienta. El sitio principal para validar y bloquear acciones no deseadas.
- **PostToolUse**: evento que se dispara después de que la herramienta termina. Útil para auditar y reaccionar al resultado.

- **Acción reversible / irreversible:** distinción clave para política de hooks. Lo reversible se permite por defecto; lo irreversible se bloquea por defecto.

Qué viene en M4.2

Hooks concretos de validación contra spec. El hook que comprueba que el código toca solo los archivos del plan. El hook que ejecuta tests tras cada edición. El hook que rechaza commit si la spec correspondiente no está actualizada. Tres ejemplos con código real que puedes copiar a tu proyecto, con sus pros y contras. Después de ese submódulo vas a tener hooks operativos en tu repo, no solo conceptos.

M4.2 · Hooks de validación contra spec

Bloque 0 · Del concepto al código

En el M4.1 quedó claro por qué los hooks importan: tu atención no escala, las protecciones automáticas sí. Aquí toca aterrizar tres hooks concretos que cierran el bucle SDD: el código generado tiene que respetar el plan, los tests tienen que pasar tras cada cambio, y los commits no deberían salir con la spec desactualizada.

Cada hook lo veremos en cuatro partes: qué problema resuelve, cómo se implementa (con código real), cuándo rompe o no encaja, y cómo lo ajustas a tu proyecto. Al terminar el submódulo deberías poder copiar uno de los tres a tu repo de práctica del M3 y verlo funcionar.

Una nota antes de empezar: la sintaxis exacta de configuración de hooks depende del cliente de Claude Code que uses (CLI estándar, plugin de VS Code, etc.). Los ejemplos que voy a dar usan el formato canónico de `settings.json`, que es el más común en el momento de redactar. Si tu cliente usa otro formato, los conceptos se trasladan; la sintaxis literal hay que comprobarla en la documentación de tu versión concreta.

Bloque 1 · Hook 1, alcance limitado al plan

Qué problema resuelve. El plan dijo "voy a tocar `src/facturas/` y `tests/facturas/`". Durante la ejecución, el agente decide tocar también `src/comunes/utils.ts` "para una mejora pequeña". Tu atención no lo pilla porque no estabas mirando. Este hook bloquea esa modificación lateral.

Cómo se implementa. Un hook `PreToolUse` que se dispara antes de cada llamada a las herramientas de escritura (`write_file`, `edit_file`, etc.). El hook compara la ruta del archivo que se va a modificar con la lista de archivos del plan firmado. Si no coincide, bloquea y reporta.

Configuración del `settings.json`:

```

{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node .claude/hooks/check-plan-scope.js"
          }
        ]
      }
    ]
  }
}

```

Y el script `check-plan-scope.js` :

```

#!/usr/bin/env node
const fs = require('fs');
const path = require('path');

const input = JSON.parse(fs.readFileSync(0, 'utf-8'));
const filePath = input.tool_input?.file_path;

if (!filePath) {
  process.exit(0); // No hay path, no podemos validar; permitimos.
}

const planPath = path.join(process.cwd(), '.claude/current-plan.json');
if (!fs.existsSync(planPath)) {
  process.exit(0); // No hay plan firmado; modo libre.
}

const plan = JSON.parse(fs.readFileSync(planPath, 'utf-8'));
const allowedFiles = plan.allowedFiles || [];

const isAllowed = allowedFiles.some(allowed => {
  if (allowed.endsWith('/*')) {
    return filePath.startsWith(allowed.slice(0, -1));
  }
  return filePath === allowed || filePath.endsWith('/') + allowed;
});

if (!isAllowed) {
  console.error(
    `[hook check-plan-scope] BLOQUEADO: ${filePath} no está en el plan firmado.\n` +
    `Archivos permitidos: ${allowedFiles.join(', ')}.\n` +
    `Si necesitas tocar este archivo, actualiza el plan y vuelve a intentar.`
  );
  process.exit(2); // Exit code 2 → bloqueo con mensaje al agente.
}

process.exit(0);

```

Cómo se usa el plan firmado. Cuando apruebas un plan en Plan Mode, escribes (o el cliente escribe automáticamente) el archivo `.claude/current-plan.json` con la lista de archivos permitidos:

```
{
  "allowedFiles": [
    "src/Facturas.Domain/*",
    "src/Facturas.Api/*",
    "tests/Facturas.Tests/*",
    "specs/factura.md"
  ],
  "signedAt": "2026-05-28T10:30:00Z"
}
```

Cuándo rompe. Tres casos:

- Si el agente necesita tocar un archivo legítimo que el plan no previó (un import que hay que ajustar, una migración generada que apunta a un schema), el hook bloquea y tienes que ampliar el scope manualmente. Molesto, pero es la idea.
- Si el plan se ha quedado obsoleto (no se actualizó al iterar), bloquea cosas legítimas. Solución: borra `.claude/current-plan.json` cuando termine la sesión, o vuelve a firmar plan.
- Si trabajas en sesión libre (sin plan), no aplica. El hook ve que no hay archivo de plan y permite todo.

Cómo lo ajustas. Las rutas permitidas las defines tú. Para empezar, usa globs amplios (`src/Facturas.Api/*`) y vas estrechando si ves que el agente se desvía mucho. Si tu solution tiene varios proyectos, considera un plan por proyecto en lugar de un plan global.

Bloque 2 · Hook 2, tests tras cada edición

Qué problema resuelve. El agente edita un archivo, sigue con el siguiente, sigue con el siguiente, sigue con el siguiente — y solo al final ejecuta tests. Cuando los tests fallan, no sabes cuál de los seis cambios los rompió. Si hubieras ejecutado tests tras cada edición, sabrías el cambio exacto que falló.

Cómo se implementa. Un hook `PostToolUse` que se dispara después de cada escritura en archivos de código fuente. Ejecuta el subconjunto de tests relevante y, si falla, reporta y devuelve un mensaje al agente.

Configuración:


```

{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/run-relevant-tests.sh"
          }
        ]
      }
    ]
  }
}

```

Y el script:

```

#!/bin/bash
# .claude/hooks/run-relevant-tests.sh

INPUT=$(cat)
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')

if [ -z "$FILE_PATH" ]; then
  exit 0
fi

# Solo ejecuta tests si el archivo modificado es código fuente C#.
if [[ "$FILE_PATH" ≠ *.cs ]]; then
  exit 0
fi

# Detecta el proyecto del archivo modificado.
# Asume estructura src/<NombreProyecto>/ → tests/<NombreProyecto>.Tests/.
PROJECT=$(echo "$FILE_PATH" | sed -n 's|^src/\([^/]*\)/*|\1|p')

if [ -z "$PROJECT" ]; then
  exit 0
fi

# Ejecuta solo los tests del proyecto afectado (mucho más rápido que toda la
solution).
RESULT=$(dotnet test "tests/${PROJECT}.Tests/" --logger "console;verbosity=minimal"
--nologo 2>&1)
STATUS=$?

if [ $STATUS -ne 0 ]; then
  echo "[hook run-relevant-tests] FAILED tests en ${PROJECT}.Tests tras editar
$FILE_PATH:" >&2
  echo "$RESULT" >&2
  exit 2 # Bloqueo con mensaje al agente.
fi

exit 0

```

Cuándo rompe. Cuatro casos:

- Si tus tests son lentos (más de cinco segundos por proyecto), el hook hace que cada edición tarde demasiado. La sesión se ralentiza. Solución: filtra mejor con `--filter Category=Unit`, o solo dispara el hook tras grupos de ediciones, no tras cada una.
- Si el agente edita archivos auxiliares (`.csproj`, `appsettings.json`, `README`), el hook puede intentar ejecutar tests que no aplican. El script excluye por extensión `.cs`, pero hay que afinarlo si tu proyecto mete lógica en otros archivos.
- Si los tests dependen de un servicio externo (DB, API mock), el hook puede fallar por motivos no relacionados con el cambio. Para esos casos, mejor un filtro tipo `--filter Category=Unit` que excluya tests de integración.
- Si tu agente está en mitad de un refactor grande donde durante varios pasos los tests fallan por diseño, el hook molesta. Solución: desactivar temporalmente el hook para esa pieza concreta.

Cómo lo ajustas. El script de ejemplo asume estructura `src/<Proyecto>/` y `tests/<Proyecto>.Tests/` (convención .NET solution típica). Si tu solution tiene otra estructura, ajusta el `sed`. Si quieres filtrar por categoría (xUnit `Trait` / `Category`), añade el flag `--filter`. Si tus tests son rápidos, ejecuta `dotnet test` sin filtro y no te compliques.

Bloque 3 · Hook 3, commit con spec sincronizada

Qué problema resuelve. El agente termina la implementación, hace `git add` y `git commit` con un mensaje razonable. Pero la spec en `specs/feature.md` no se actualizó para reflejar las decisiones que se tomaron durante la implementación. El commit cierra la pieza con la spec divergente. Tres semanas después, alguien lee la spec y no entiende por qué el código hace algo distinto.

Cómo se implementa. Un hook `PreToolUse` que se dispara antes de ejecuciones de `git commit`. Comprueba si el commit incluye cambios en archivos de código pero no en la spec correspondiente. Si detecta la asimetría, pregunta al agente si la spec está al día.

Configuración:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/check-spec-sync.sh"
          }
        ]
      }
    ]
  }
}
```

Y el script:

```
#!/bin/bash
# .claude/hooks/check-spec-sync.sh

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Solo nos interesa interceptar git commit.
if [[ "$COMMAND" ≠ "git commit" ]]; then
  exit 0
fi

# Obtiene los archivos staged.
STAGED=$(git diff --cached --name-only)

# Detecta si hay archivos C# en src/ sin que haya archivos en specs/.
SRC_CHANGES=$(echo "$STAGED" | grep -E '^src/.*\.cs$' | wc -l)
SPEC_CHANGES=$(echo "$STAGED" | grep -E '^specs/.*\.md$' | wc -l)

if [ "$SRC_CHANGES" -gt 0 ] && [ "$SPEC_CHANGES" -eq 0 ]; then
  echo "[hook check-spec-sync] AVISO: el commit incluye cambios en src/*.cs pero ninguna spec se ha actualizado." >&2
  echo "Si la implementación cambió decisiones que estaban en la spec, actualízala antes de committear." >&2
  echo "Si la spec sigue siendo correcta tal cual está, confirma con --no-verify o explícitamente." >&2
  exit 2 # Bloqueo con mensaje.
fi

exit 0
```

Cuándo rompe. Tres casos:

- Si el cambio en src/ es trivial (typo, comentario, refactor sin cambio de comportamiento), la spec no necesita actualizarse. El hook se queja igual. Tienes que confirmarlo manualmente.
- Si tu proyecto tiene specs en otra carpeta o con otra extensión, hay que ajustar los grep.

- Si trabajas con commits pequeños y atómicos (un commit por archivo), el hook se va a quejar a menudo. Solución: aplicar el hook solo a commits que cierran piezas, no a commits intermedios.

Cómo lo ajustas. Para empezar, deja el hook con bloqueo blando (mensaje + permite continuar si confirmas). Cuando el equipo se haya acostumbrado y entienda cuándo se aplica, sube a bloqueo duro. La transición es importante: si lo metes duro desde el primer día, la gente lo desactiva y pierdes la protección.

Bloque 4 · Configurar y mantener los hooks

Tres detalles operativos que importan más de lo que parece.

Dónde viven los hooks. En `.claude/hooks/` (o equivalente según el cliente). Los scripts son archivos normales en bash, Python, Node, lo que tu proyecto soporte. Versionados en git como el resto del código del proyecto: forman parte de la infraestructura de desarrollo, no son configuración personal del usuario.

Cómo se distribuyen al equipo. Si los hooks viven en el repo, cualquier persona que clone tiene los mismos hooks activos. El comportamiento del agente es el mismo para todo el equipo. Si los hooks vivieran en configuración local de cada uno, tendrías protección desigual: algunos protegidos, otros no.

Cómo se mantienen. Los hooks envejecen con el proyecto. Cuando cambias la estructura de carpetas de la solution, los scripts que parsean rutas dejan de funcionar. Cuando cambias de xUnit a NUnit (o de proveedor de tests en general), los comandos hay que actualizar. Asigna mantenimiento de los hooks a alguien — el mismo "owner" que recomendamos para skills en M3.3. Sin owner, los hooks se rompen silenciosamente y dejan de proteger.

Bloque 5 · Errores comunes al implementar hooks

Cinco patrones que veo en gente que arranca con hooks y se atasca:

Hooks demasiado estrictos al inicio. El primer día se meten cinco hooks de bloqueo duro y se vuelve imposible trabajar. El agente bloqueado cada dos por tres, el equipo desactivando hooks, y a la semana nadie tiene hooks. Solución: empieza con bloqueos blandos (avisos), sube a duros cuando confirmes que no producen falsos positivos.

Hooks que tardan demasiado. Un hook que ejecuta la suite completa de tests tras cada edición convierte cada acción en una espera de minutos. La sesión se vuelve insufrible. Filtra qué

tests ejecutas, o dispara el hook menos veces.

Hooks sin logging. Cuando algo falla y no sabes si el hook se ejecutó o no, te ciegas. Cada hook debe escribir a un log (`.claude/hooks.log`) qué hizo y cuándo. Cinco minutos de setup, horas de debugging ahorradas.

Hooks no commitados. "Funciona en mi máquina" porque solo está en tu configuración local. Si los hooks no viven en el repo, el resto del equipo no los tiene. Y si funcionan distinto para distintas personas, dejas de fiarte de ellos.

Hooks que dependen de comandos no garantizados. El hook usa `jq`, pero algunos del equipo no lo tienen instalado. El hook usa una versión específica de Node. Solución: documenta las dependencias del hook en su propio archivo, e idealmente comprueba al inicio del script que existen.

Bloque 6 · Cierre

Tres ideas para llevarte.

Empieza con un hook, no con cinco. El primer hook que metas en producción te enseña más sobre cómo funcionan que diez horas de documentación. Cuando ese hook esté afinado y todo el equipo lo tenga interiorizado, mete el siguiente.

Los hooks viven en el repo, no en tu config personal. Solo así protegen a todo el equipo igual y solo así se mantienen como parte normal del código.

La validación contra spec es donde SDD encuentra su garantía. Los hooks de los tres tipos vistos aquí (alcance del plan, tests por edición, commit con spec sincronizada) cierran el bucle. Sin ellos, SDD vive en buena voluntad. Con ellos, vive en el proceso.

Glosario rápido del bloque

- **Hook PreToolUse:** hook que se dispara antes de que el agente ejecute una herramienta. El sitio para bloquear acciones no deseadas.
- **Hook PostToolUse:** hook que se dispara después de que la herramienta termina. El sitio para validar el resultado.
- **Bloqueo blando:** hook que avisa pero permite continuar tras confirmación. Recomendado al inicio.
- **Bloqueo duro:** hook que cancela la acción del agente sin posibilidad de continuar sin desactivarlo. Para protecciones críticas tras periodo de rodaje.

- `.claude/current-plan.json` : archivo donde se persiste el plan firmado en Plan Mode para que los hooks puedan consultarlo durante la ejecución.

Qué viene en M4.3

Hooks de protección. Si M4.2 trata hooks de validación (¿lo que se está haciendo cumple lo planeado?), M4.3 trata hooks de defensa (¿lo que se está haciendo es algo seguro?). Bloqueos de acciones destructivas (`rm -rf`, `git push --force`), bloqueos de secretos en commits, bloqueos de cambios fuera del scope del repo. Más cortos que los de validación, igualmente importantes.

M4.3 · Hooks de protección — destructivas, secretos, scope

Bloque 0 · Del cumplimiento al peligro

En el M4.2 vimos hooks que comprueban que el agente está haciendo lo que se planeó. Útiles, pero el problema que resuelven es de disciplina del proceso. Aquí cambiamos a un problema distinto: que el agente esté a punto de hacer algo peligroso, planificado o no.

La diferencia importa. Un hook de validación bloquea cosas que técnicamente funcionarían pero violan el método. Un hook de protección bloquea cosas que pueden romper producción, filtrar secretos o corromper datos. Si los primeros son tu maestro de obra, los segundos son tu casco.

En estos veinte minutos: tres hooks de protección concretos con código real. Bloqueos de acciones destructivas, de secretos en commits, y de operaciones fuera del scope del repo. Cada uno tiene su patrón propio y su forma de fallar; los veremos uno a uno.

Bloque 1 · Hook 1, bloquear acciones destructivas

Qué problema resuelve. El agente, durante el bucle de ejecución, decide ejecutar `rm -rf node_modules` para "limpiar y reinstalar". Lo hace sin pasar por Plan Mode porque no estaba previsto: emergió como consecuencia de un error que estaba intentando arreglar. Tres minutos después estás restaurando tu trabajo no commitado.

Cómo se implementa. Un hook `PreToolUse` que se dispara antes de cada ejecución de `Bash`. Inspecciona el comando que el agente va a ejecutar; si contiene patrones destructivos sin un flag de aprobación explícita, bloquea.

Configuración:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/block-destructive.sh"
          }
        ]
      }
    ]
  }
}
```

Script:


```
#!/bin/bash
# .claude/hooks/block-destructive.sh

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

if [ -z "$COMMAND" ]; then
    exit 0
fi

# Patrones destructivos. Ajustables según el proyecto.
PATTERNS=(
    "rm -rf"
    "rm -fr"
    "rm -r .* /"
    "git reset --hard"
    "git clean -[fdx]"
    "git push --force"
    "git push -f"
    "git push --force-with-lease"
    "git checkout -- ."
    "git restore ."
    "git branch -D"
    "DROP TABLE"
    "DROP DATABASE"
    "TRUNCATE TABLE"
    "DELETE FROM .* WHERE 1=1"
    "kill -9 -1"
    ">/dev/sda"
    "dd if=.* of=/dev/"
)

for pattern in "${PATTERNS[@]}; do
    if echo "$COMMAND" | grep -qE "$pattern"; then
        echo "[hook block-destructive] BLOQUEADO: el comando contiene un patrón destructivo." >&2
        echo "Comando: $COMMAND" >&2
        echo "Patrón detectado: $pattern" >&2
        echo "Si necesitas ejecutar esto, hazlo manualmente fuera de Claude Code o" >&2
        echo "confirma con un humano antes de continuar." >&2
        exit 2
    fi
done

exit 0
```

Cuándo rompe. Tres casos:

- El agente quiere ejecutar `rm -rf` sobre algo legítimo (la carpeta `dist/` antes de un build, por ejemplo). El hook bloquea y tienes que ejecutarlo manualmente. Es el coste asumido del bloqueo por defecto.
- Patrones nuevos que tu lista no contempla. Cuando aparezca uno (alguien del equipo descubre un comando peligroso que no estaba listado), añádelo.

- Falsos positivos: el agente quiere ejecutar `git push -f origin feature-rama-de-pruebas` que en tu equipo es aceptable. Si esto ocurre a menudo, considera ajustar el patrón para que solo bloquee `push --force` a `main/master`.

Cómo lo ajustas. La lista de patrones es donde vas a iterar más. Empieza conservador (bloquea mucho) y afloja solo cuando confirmes que el falso positivo es genuino. Casi siempre la versión más segura del patrón te pide confirmación humana que tarda diez segundos; el coste de bloquear por error es bajo, el de no bloquear cuando debías es alto.

Bloque 2 · Hook 2, bloquear secretos en commits

Qué problema resuelve. El agente, generando una configuración de despliegue, escribe una contraseña o una API key directamente en un archivo. Hace `git add` y va a commitarlo. Si el commit llega al remoto, el secreto está expuesto — y los secretos en git son extremadamente difíciles de eliminar por completo.

Cómo se implementa. Un hook `PreToolUse` que intercepta `git commit` y comprueba si los archivos staged contienen patrones que parezcan secretos. Bloquea si los detecta.

Configuración (idéntica al patrón anterior, distinto script):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/block-secrets.sh"
          }
        ]
      }
    ]
  }
}
```

Script:

```
#!/bin/bash
# .claude/hooks/block-secrets.sh

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

if [[ "$COMMAND" ≠ "git commit" ]]; then
    exit 0
fi

# Patrones que parecen secretos.
PATTERNS=(
    "AKIA[0-9A-Z]{16}"                # AWS Access Key
    "sk-[a-zA-Z0-9]{32,}"             # Tokens estilo OpenAI / Anthropic
    "ghp_[a-zA-Z0-9]{36}"             # GitHub personal access token
    "ya29\\.[0-9A-Za-z_-]+"           # Google OAuth token
    "-----BEGIN (RSA |OPENSSH |EC )?PRIVATE KEY-----"
    "password\\s*[:=]\\s*['\"]['\"]{8,}['\"]"
    "secret\\s*[:=]\\s*['\"]['\"]{8,}['\"]"
    "api[_-]?key\\s*[:=]\\s*['\"]['\"]{16,}['\"]"
)

# Obtiene contenido de archivos staged.
STAGED_FILES=$(git diff --cached --name-only)
HITS=""

for file in $STAGED_FILES; do
    if [ -f "$file" ]; then
        CONTENT=$(git show ":$file" 2>/dev/null)
        for pattern in "${PATTERNS[@]"; do
            if echo "$CONTENT" | grep -qE "$pattern"; then
                HITS="$HITS\n $file (patrón: $pattern)"
            fi
        done
    fi
done

if [ -n "$HITS" ]; then
    echo "[hook block-secrets] BLOQUEADO: posibles secretos en archivos staged." >&2
    echo -e "Hits:$HITS" >&2
    echo "Revisa los archivos y elimina los secretos antes de commitear." >&2
    echo "Si es un falso positivo, ajusta los patrones en .claude/hooks/block-secrets.sh." >&2
    exit 2
fi

exit 0
```

Cuándo rompe. Cinco casos:

- Falsos positivos en fixtures de tests donde sí quieres tener strings que parecen secretos (tokens de ejemplo, mocks). Solución: excluir carpetas de tests del scan, o marcar el archivo como permitido.
- Falsos negativos en secretos con patrones inusuales. La lista nunca es completa; ajusta cuando descubras uno nuevo.

- Si el secreto ya estaba en una versión anterior del archivo, el hook lo deja pasar (solo inspecciona los staged). Para eso son las herramientas tipo `git-secrets` o `truffleHog` corriendo en CI — el hook protege la entrada nueva.
- Si el repo es público y ya hubo un leak, el hook no lo arregla. Si pasa, hay que rotar las credenciales filtradas — el commit borrado no es suficiente.
- Patrones de secretos legítimos en código (un schema de configuración con un campo `password: string`). El hook caza el patrón aunque sea declaración, no asignación. Ajusta los regex para casos reales.

Cómo lo ajustas. Empieza con la lista de patrones más reconocidos (AWS, GitHub, OpenAI). Cuando alguien del equipo se quede bloqueado por falso positivo, refina el regex en lugar de quitarlo. Si tu organización ya usa una herramienta de detección de secretos (Gitleaks, TruffleHog), considera invocarla desde el hook en lugar de mantener tu propia lista.

Bloque 3 · Hook 3, bloquear acciones fuera del scope del repo

Qué problema resuelve. El agente, intentando resolver un error, decide tocar archivos del sistema o de la home del usuario. Edita `/etc/hosts`. Modifica `~/.bashrc`. Cambia un archivo en otro repo paralelo. Cosas que técnicamente son posibles pero casi nunca son lo que querías.

Cómo se implementa. Un hook `PreToolUse` para `Edit`, `Write` y `Bash` que comprueba que la ruta del archivo (o el cwd del comando) esté dentro del repo del proyecto. Bloquea si está fuera.

Configuración (cubre dos matchers a la vez):

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit|Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/block-out-of-scope.sh"
          }
        ]
      }
    ]
  }
}
```

Script:

```
#!/bin/bash
# .claude/hooks/block-out-of-scope.sh

INPUT=$(cat)
TOOL_NAME=$(echo "$INPUT" | jq -r '.tool_name // empty')
FILE_PATH=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

REPO_ROOT=$(git rev-parse --show-toplevel 2>/dev/null || pwd)

# Comprueba escrituras directas.
if [ -n "$FILE_PATH" ]; then
    # Resolver rutas absolutas para comparar.
    ABS_FILE=$(realpath -m "$FILE_PATH")
    ABS_REPO=$(realpath -m "$REPO_ROOT")

    if [[ "$ABS_FILE" ≠ "$ABS_REPO"* ]]; then
        echo "[hook block-out-of-scope] BLOQUEADO: $ABS_FILE está fuera del repo $ABS_REPO." >&2
        echo "Si necesitas modificar algo fuera del repo, hazlo manualmente." >&2
        exit 2
    fi
fi

# Comprueba comandos shell que cambian de directorio o tocan paths absolutos.
if [ -n "$COMMAND" ]; then
    # Patrones de paths del sistema que NUNCA deberían tocarse desde Claude Code.
    FORBIDDEN_PATHS=(
        "/etc/"
        "/usr/"
        "/var/"
        "/boot/"
        "/sys/"
        "$HOME/" # archivos ocultos de la home (.bashrc, .ssh, etc.)
    )

    for forbidden in "${FORBIDDEN_PATHS[@]"; do
        if echo "$COMMAND" | grep -qE "(>|>>|tee|cp .*|mv .*|rm .*)" $forbidden"; then
            echo "[hook block-out-of-scope] BLOQUEADO: el comando intenta escribir o eliminar en $forbidden." >&2
            echo "Comando: $COMMAND" >&2
            exit 2
        fi
    done
fi

exit 0
```

Cuándo rompe. Tres casos:

- El agente quiere tocar legítimamente algo fuera del repo (configurar un servicio del sistema, modificar un dotfile del usuario). Si es legítimo, hazlo manualmente o desactiva el hook temporalmente. Para esto el hook está bien estricto.
- Repos con submódulos o monorepos donde "fuera del repo principal" sí es parte del sistema. Ajusta el script para considerar el monorepo entero, no solo el subrepo actual.

- Comandos complejos con redirecciones y subshells que el regex no caza bien. La lista de patrones siempre tiene huecos; añade los que descubras.

Cómo lo ajustas. La lista de `FORBIDDEN_PATHS` es lo que probablemente vas a iterar. Para empezar, mantén estricto: bloquea `/etc`, `/usr`, `/var`, `$HOME/.*`. Si el agente legítimamente necesita modificar algo en esas rutas, lo haces tú manualmente y reportas para que el hook lo siga bloqueando para los demás.

Bloque 4 · Patrones de detección sospechosa (más allá de los tres)

Más allá de los tres hooks concretos, hay un patrón general que merece la pena tener presente: cualquier acción que produce un cambio externo importante merece pasar por un hook.

Tres ejemplos adicionales que puedes considerar según tu proyecto:

Bloqueo de instalación de dependencias. Hook que detecta `dotnet add package`, `nuget install`, `npm install`, `pip install` con paquetes nuevos. Pregunta antes de añadir. Razón: cada dependencia es superficie de ataque (cadena de suministro), peso del binario, mantenimiento extra. Que no entren sin discusión.

Bloqueo de despliegues y operaciones de release. Hook que intercepta cualquier comando relacionado con publicación (`npm publish`, `cargo publish`, `gh release`, scripts de deploy). Estos no se ejecutan por error: tienen que pasar por humano explícito.

Bloqueo de operaciones de DB en entornos sensibles. Si tu proyecto se conecta a bases de datos, hook que detecta `psql production`, `mongosh prod-cluster`, conexiones a hosts de producción. Bloquea por defecto; permite solo en entornos dev/local sin más.

No metas los seis hooks el primer día. Empieza por los tres del bloque anterior (destructivas, secretos, scope), añade los demás conforme veas que el equipo los necesita.

Bloque 5 · Cierre

Tres ideas para llevarte.

Los hooks de protección son la diferencia entre "el agente puede hacer cosas peligrosas" y "el agente no puede hacer cosas peligrosas sin pasar por un humano". La asimetría a tu favor es enorme: un bloqueo por error cuesta diez segundos de molestia; un no-bloqueo cuando debía bloquearse puede costar producción caída o secretos filtrados.

La filosofía "lo reversible se permite, lo irreversible se bloquea" del M4.1 cobra sentido pleno aquí. Los hooks de protección viven justo en la frontera. Si quitas alguno, presta atención a si lo que estás permitiendo es reversible o no.

Los patrones de detección son listas vivas. Cuando aparezca un patrón nuevo (un secreto en un formato inusual, un comando destructivo que tu lista no contemplaba), añádelo. La lista se mantiene como cualquier otro código del proyecto. Sin mantenimiento, los hooks envejecen y dejan de proteger contra lo que se ha vuelto común.

Glosario rápido del bloque

- **Hook destructivo:** hook que bloquea acciones irreversibles (`rm -rf` , `git push --force` , `DROP TABLE`).
- **Hook anti-secreto:** hook que bloquea commits con patrones que parecen credenciales o claves.
- **Hook de scope:** hook que bloquea modificaciones fuera del repo o del directorio de trabajo.
- **Falso positivo de hook:** cuando el hook bloquea algo legítimo. La respuesta correcta es refinar el patrón, no quitar el hook.
- **Acción reversible / irreversible:** distinción clave de M4.1 aplicada en estos hooks. Lo irreversible se bloquea por defecto.

Qué viene en M4.4

Hooks como activo del equipo. Cómo se comparten en el repo, cómo se versionan, cómo se mantienen al día. Pieza más operativa que técnica: si tienes hooks geniales solo en tu máquina, tu equipo no está protegido. Veremos también el patrón de "kit de hooks de equipo" que algunos equipos publican como repos de referencia para arrancar nuevos proyectos.

M4.4 · Hooks como skill de equipo

Bloque 0 · El problema de los hooks aislados

Si has hecho la práctica del M3.6 y leído los hooks del M4.2 y M4.3, probablemente tengas un par de hooks funcionando en tu máquina. Eso protege a una persona: tú.

Pero el equipo no es una persona. Si los hooks viven solo en tu configuración local, tus compañeras y compañeros trabajan sin esa red. El agente de tu colega puede modificar `appsettings.Production.json` sin pestañear, instalar paquetes de NuGet sin avisar, hacer `git push --force` a `main`. La protección que tú diseñaste no le llega.

Y va más allá: aunque todo el equipo se esmere en mantener sus propios hooks, sin convergencia el resultado es desigual. Unos tienen el hook de secretos pero no el de scope. Otros tienen scope pero no destructivos. Cuando alguien cuenta que "le pasó X" y otra persona dice "a mí no me pasa porque tengo el hook Y", queda claro que la protección está fragmentada.

La salida es tratar los hooks como cualquier otra pieza del proyecto: vivos en el repo, versionados con el código que protegen, con owner declarado y revisión periódica. En estos quince minutos vemos cómo.

Bloque 1 · Hooks en el repo, no en configuración personal

Lo primero y más importante: **los hooks viven en el repo del proyecto**. Concretamente, en una carpeta tipo `.claude/hooks/` que se commitea como cualquier otro archivo.

Cuando un miembro del equipo clona el repo, los hooks vienen con él. Cuando alguien actualiza un hook, el cambio entra por PR como cualquier código. Cuando alguien debate si un hook es demasiado estricto, la discusión queda registrada en el PR para futura consulta.

Esto resuelve varios problemas a la vez:

Protección uniforme. Todo el equipo tiene los mismos hooks. Si el de secretos bloquea para una persona, bloquea para todas. La sensación de "a mí me pasa, a ti no" desaparece.

Histórico vinculado al código. Si la estructura de la solution cambia y un hook que parseaba rutas deja de funcionar, el commit que rompe el hook está cerca del commit que rompió las rutas. `git blame` te dice cuándo se desincronizó.

Onboarding más rápido. Alguien nuevo en el equipo clona el repo y al primer uso de Claude Code ya tiene la red puesta. No hay "ah, también tienes que copiar este script de mi máquina".

Discusión en PR. Los cambios de hook se revisan como código. Si alguien propone bajar un bloqueo de duro a blando, otra persona puede objetar antes de que entre. Sin PR, esa decisión la tomaría una persona sola sin contraste.

La estructura típica en el repo:

```
.claude/  
  hooks/  
    check-plan-scope.sh  
    check-spec-sync.sh  
    block-destructive.sh  
    block-secrets.sh  
    run-relevant-tests.sh  
  settings.json
```

El `settings.json` declara qué hooks se enganchan a qué eventos. Los `.sh` (o `.js`, o `.ps1`) son los scripts. Todo versionado en git como código de primera clase.

Bloque 2 · Convenciones de archivos y nombres

Aunque los hooks pueden ser cualquier cosa que el cliente sepa ejecutar, hay que establecer convenciones en el equipo para que la carpeta no se vuelva un cajón.

Cinco convenciones prácticas:

Un script por hook, un nombre que dice qué hace. Mejor `block-destructive.sh` que `hook1.sh`. El nombre es lo que vas a ver en logs cuando algo bloquea — que se entienda de un vistazo.

Verbos de acción explícitos. Hooks que bloquean empiezan por `block-`. Hooks que validan, por `check-`. Hooks que ejecutan algo, por `run-`. Hooks que registran, por `log-`. Una convención simple pero clarísima en cuanto se aplica.

Comentarios al inicio del script. Cada hook empieza con un comentario que dice: qué hace, en qué evento se dispara, cuándo bloquea y cuándo deja pasar. Algo así:

```
#!/bin/bash
# .claude/hooks/check-plan-scope.sh
#
# Hook: comprueba que el archivo a modificar está en el plan firmado.
# Dispara: PreToolUse (Edit, Write, MultiEdit).
# Bloquea si el archivo no está en .claude/current-plan.json.
# Owner: equipo-backend.
# Versión: 2.
```

Logging consistente. Todos los hooks escriben a un mismo log (`.claude/hooks.log`) con timestamp, nombre del hook y resultado. Cuando algo va mal, lees el log y entiendes en orden lo que pasó.

Documentación en el README de hooks. Un `.claude/hooks/README.md` que liste los hooks activos, qué hace cada uno, quién es el owner. Cinco minutos de escritura, semanas de tiempo ahorrado a quien se incorpore.

Bloque 3 · Owner y mantenimiento

Los hooks envejecen. Cuando la estructura de la solution cambia, cuando se actualiza de .NET 10 a .NET 11, cuando xUnit v3 saca una versión nueva con sintaxis distinta — los scripts que parsean rutas, ejecutan tests o reconocen patrones dejan de funcionar. Sin mantenimiento, en seis meses tu carpeta de hooks tiene tres rotos, dos que protegen contra problemas que ya no existen, y nadie sabe cuál es cuál.

La regla operativa: **cada hook tiene owner declarado**. En el comentario inicial del script, en el README, donde el equipo prefiera. Cuando algo falla en un hook, hay una persona a la que preguntar.

Tres responsabilidades del owner:

Mantener el hook al día. Cuando algo del proyecto cambia que afecta al hook (estructura, framework, convenciones), el owner ajusta el script. Si el owner no puede, transfiere ownership explícitamente.

Revisar PRs que toquen el hook. Si alguien propone modificarlo, el owner es revisor obligatorio. Eso garantiza coherencia en los cambios.

Decidir retirada. Cuando un hook deja de aportar (porque el problema que prevenía ya no se da, porque hay una solución mejor), el owner decide retirarlo y archivarlo.

Una práctica útil: revisión trimestral de hooks. Cada tres meses, el owner mira los hooks que tiene asignados y se pregunta tres cosas:

- ¿Sigue activo? Si nadie ha visto al hook bloquear nada en seis meses, igual ya no aporta.
- ¿Sigue siendo correcto? La realidad del proyecto cambia; el hook puede haberse quedado desfasado.
- ¿Sigue siendo necesario? Si la causa raíz del problema que prevenía se ha eliminado, el hook ya no hace falta.

Si las tres respuestas son sí, el hook se queda como está. Si alguna es no, hay trabajo.

Bloque 4 · Kit de hooks de equipo y propagación entre proyectos

Cuando una organización tiene varios proyectos .NET con la misma necesidad de protección, mantener hooks por proyecto resulta repetitivo. Algunos equipos publican un repo de referencia con un "kit de hooks" estándar que cada proyecto nuevo clona como punto de partida.

La idea es que el kit incluye:

Hooks base. Los que toda organización serie quiere: bloqueo de destructivas, bloqueo de secretos, scope del repo. Estos se aplican igual a cualquier proyecto.

Hooks adaptables. Los que requieren ajuste por proyecto: ejecución de tests (`dotnet test` con filtros específicos), validación de spec (estructura de carpetas del proyecto en particular), revisión de plan firmado. El kit los trae como plantilla con marcadores claros (`# AJUSTAR: nombre de la solution`).

Scripts de instalación. Un `install.sh` o `install.ps1` que el equipo nuevo ejecuta una vez. Copia los hooks al repo destino, deja los adaptables como TODO marcado, y comprueba que las dependencias (`jq` , etc.) están disponibles.

Documentación de mantenimiento. Cuando el kit central se actualiza con un hook nuevo o un fix, hay un proceso claro para que los proyectos derivados incorporen los cambios. Una herramienta tipo `cookiecutter` o simplemente un script de sincronización puede ayudar.

Esto no compensa para equipos pequeños con un solo proyecto. Compensa cuando llevas tres o cuatro proyectos y notas que estás copiando los mismos hooks una y otra vez.

Bloque 5 · Resistencia y adopción

Una nota práctica antes del cierre. Los hooks tienen mala prensa porque la primera experiencia con ellos suele ser frustrante: bloqueos por motivos no claros, error messages crípticos, sensación de que la herramienta te frena.

Si vas a introducir hooks en un equipo que no los tiene, anticipáte a esa resistencia. Tres tácticas que ayudan:

Empezar con mensajes de error explícitos. Cada bloqueo debe decir: qué hook lo causó, por qué (qué patrón detectó), y cómo proceder (cómo continuar si fue legítimo). Sin eso, el equipo aprende a desconfiar de los hooks antes de aprender a respetarlos.

Modo "warn" antes que "block". El primer mes, el hook avisa pero permite continuar. Eso le da al equipo tiempo de ver lo que detecta sin sufrir bloqueos. Cuando confirmáis que los avisos son válidos, pasáis a bloqueo duro.

Champion técnico para resolución de falsos positivos. Cuando alguien se queda atascado por un hook, una persona del equipo se encarga de ayudarlo en menos de una hora. Sin ese soporte, la gente aprende a desactivar hooks antes que a usarlos. Con ese soporte, aprenden a confiar.

Bloque 6 · Cierre

Tres ideas para llevarte.

Los hooks que no viven en el repo no protegen al equipo. La protección desigual es casi peor que no tener protección: te da falsa sensación de seguridad y dificulta diagnosticar qué pasó cuando algo falla.

Cada hook necesita owner. Sin responsable declarado, los hooks envejecen, fallan silenciosamente y dejan de proteger contra lo que se supone que protegían. La revisión trimestral mantiene el catálogo vivo.

Para organizaciones con varios proyectos, un kit central de hooks compartido evita el copy-paste eterno. Para equipos pequeños, no compensa — bastan los hooks del propio proyecto bien mantenidos.

Glosario rápido del bloque

- **Hook como activo compartido:** hook que vive en el repo, se versiona con el código, tiene owner y se revisa en PR como cualquier pieza de primera clase.
- **Kit de hooks:** repo o paquete que reúne hooks estándar de una organización para reutilizar entre proyectos. Incluye base + adaptables + instalación.
- **Modo warn:** configuración temporal de un hook donde avisa pero no bloquea. Útil al introducir hooks en un equipo nuevo.

- **Revisión trimestral:** ritmo informal recomendado para revisar si los hooks activos siguen aportando, son correctos y son necesarios.

Qué viene en M4.5

Práctica de cierre del M4. Vas a instalar tres hooks defensivos en un repo de práctica (el mismo del M3.6 si lo tienes a mano): el de scope del plan, el de bloqueo destructivo y el de chequeo de secretos. No es solo copy-paste: vas a ajustarlos a tu solution, probarlos contra acciones reales del agente, y comprobar que se activan cuando deben. Al terminar M4 tendrás hooks operativos en tu repo, no solo conceptos en la cabeza.

M4.5 · Práctica — instalar 3 hooks defensivos en un proyecto real

Bloque 0 · Lo que vas a hacer

El M4.1 dio el porqué de los hooks. El M4.2 mostró tres hooks de validación contra spec. El M4.3 añadió tres hooks de protección. El M4.4 fijó las reglas para tratar los hooks como activo de equipo.

Aquí toca instalar. Vas a coger los tres hooks defensivos del M4.3 — bloqueo de destructivas, bloqueo de secretos, scope del repo — y los vas a meter en una solution .NET propia. Cuando termines, esos tres hooks van a estar activos en tu repo, protegiéndote en cualquier sesión futura con Claude Code.

Recorrido en cinco pasos:

1. **Preparar el repo de práctica.**
2. **Copiar los tres hooks y la configuración.**
3. **Ajustar a tu solution concreta.**
4. **Probar cada hook contra una acción que debería bloquear.**
5. **Ajustar falsos positivos si los hay.**

Tiempo total: 30-45 minutos. Si te lleva más, probablemente has elegido un repo demasiado complejo para la primera vez — empieza con uno limpio.

Bloque 1 · Paso 1 — preparar el repo

Si tienes la solution que usaste en la práctica del M3.6, vale perfectamente. Si no, crea una mínima:

```
mkdir SDDCC-practica-hooks
cd SDDCC-practica-hooks
git init
dotnet new sln -n SDDCC
mkdir src tests
dotnet new classlib -n SDDCC.Domain -o src/SDDCC.Domain
dotnet new xunit -n SDDCC.Domain.Tests -o tests/SDDCC.Domain.Tests
dotnet sln add src/SDDCC.Domain/SDDCC.Domain.csproj
dotnet sln add tests/SDDCC.Domain.Tests/SDDCC.Domain.Tests.csproj
dotnet add tests/SDDCC.Domain.Tests/SDDCC.Domain.Tests.csproj reference
src/SDDCC.Domain/SDDCC.Domain.csproj
```

Dos minutos. Tienes solution mínima funcional con un proyecto y su test.

Decisión que te toca: ¿en qué repo instalas los hooks? Para esta primera vez, mejor un repo de práctica (este o el del M3.6). Cuando tengas el método interiorizado, replicas a un proyecto real.

Comprueba que tienes las dependencias del script. Los hooks de ejemplo usan `bash`, `jq` y `git`. En Windows, lo más cómodo es Git Bash que ya trae `bash`; `jq` lo instalas con `winget install jqlang.jq` o `choco install jq`. En Mac/Linux, todo viene de serie o por brew/apt.

Bloque 2 · Paso 2 — copiar los hooks y la configuración

Crea la estructura:

```
mkdir -p .claude/hooks
touch .claude/settings.json
touch .claude/hooks/README.md
```

Copia los tres scripts del M4.3 a `.claude/hooks/`:

- `block-destructive.sh` (bloqueo de `rm -rf`, `git push --force`, etc.)
- `block-secrets.sh` (bloqueo de patrones de credenciales en commits)
- `block-out-of-scope.sh` (bloqueo de modificaciones fuera del repo)

Hazlos ejecutables:

```
chmod +x .claude/hooks/*.sh
```

(En Windows con Git Bash, normalmente Git ya respeta el bit ejecutable. Si no, `chmod` no daña.)

Configura el `.claude/settings.json` para que los tres se enganchen:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          { "type": "command", "command": "bash .claude/hooks/block-destructive.sh" },
          { "type": "command", "command": "bash .claude/hooks/block-secrets.sh" },
          { "type": "command", "command": "bash .claude/hooks/block-out-of-scope.sh" }
        ]
      },
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          { "type": "command", "command": "bash .claude/hooks/block-out-of-scope.sh" }
        ]
      }
    ]
  }
}
```

Y en `.claude/hooks/README.md`, declara los owners (en esta práctica, eres tú, pero respeta el patrón):

Hooks de SDDCC-practica

Activos

Hook	Dispara	Owner
block-destructive.sh	PreToolUse (Bash)	tu-nombre
block-secrets.sh	PreToolUse (Bash)	tu-nombre
block-out-of-scope.sh	PreToolUse (Bash, Edit, Write)	tu-nombre

Revisión

Próxima revisión: 2026-08-28.

Commitea:

```
git add .claude/
git commit -m "feat(hooks): añadir 3 hooks defensivos base"
```

Los hooks ya viven en el repo. Cualquiera que clone va a tener la red puesta.

Bloque 3 · Paso 3 — ajustar a tu solution

Los hooks del M4.3 están escritos genéricos. Hay tres ajustes a hacer para que encajen con tu solution .NET concreta.

Ajuste 1 — block-out-of-scope.sh ya respeta `git rev-parse --show-toplevel`, así que detecta tu repo automáticamente. Lo único que toca revisar son los `FORBIDDEN_PATHS`: si tu solution se conecta a recursos del sistema (servicios Windows, certificados de máquina), añade esas rutas a la lista para que el agente no las toque sin permiso.

Ajuste 2 — block-destructive.sh lleva una lista de patrones genéricos. Para .NET toca añadir algunos específicos a la `PATTERNS`:

```
PATTERNS=(
# ... patrones genéricos del M4.3
"dotnet nuget delete"
"dotnet sln remove"
"Migrate.*-Force"           # EF Core migrations con flag destructivo
"Database\|.EnsureDeleted"
"DROP TABLE"              # ya estaba; lo dejamos
)
```

Ajuste 3 — block-secrets.sh lleva patrones de credenciales comunes. Para .NET, añade patrones del ecosistema:

```
PATTERNS=(
# ... patrones genéricos del M4.3
"(ConnectionString|connectionString)\\s*[:=]\\s*['\"]['\"\\s]*Password=['\"\\s]+['\"]\""
"Authorization:\\s*Bearer\\s+[A-Za-z0-9-_.~+/{20,}"
"DefaultEndpointsProtocol=.*AccountKey=[A-Za-z0-9+/=]{20,}" # Azure Storage
"(?i)nuget\\.config.*<add\\s+key.*value=['\"][a-z0-9-]{30,}"
)
```

Edita los tres scripts con estos ajustes, commitea de nuevo, y los hooks ya son tuyos.

Bloque 4 · Paso 4 — probar cada hook

La prueba es lo que diferencia "tengo hooks instalados" de "tengo hooks que funcionan". Para cada hook, ejecuta una acción que debería bloquear y comprueba que efectivamente bloquea.

Abre Claude Code en el repo y prueba uno a uno.

Prueba 1 — block-destructive.

Pide al agente algo que dispare un comando destructivo. Por ejemplo:

"Limpia el directorio bin/ ejecutando `rm -rf bin/` y luego haz un `dotnet build` limpio."

El agente va a intentar ejecutar `rm -rf bin/`. El hook `block-destructive.sh` debe interceptarlo y bloquearlo con mensaje claro. Si no bloquea, revisa que el patrón esté en la lista y que `chmod +x` se haya aplicado.

Prueba 2 — block-secrets.

Crea un archivo `src/SDDCC.Domain/Config.cs` con contenido tipo:

```
namespace SDDCC.Domain;

public static class Config
{
    public const string ApiKey = "sk-
thisisafakebutlongenoughtolooklikearealkey1234567890";
}
```

Stagealo y pide al agente que haga commit:

"Haz git add Config.cs y git commit -m 'feat: añadir configuración'."

El hook `block-secrets.sh` debe interceptar el commit y reportar el patrón detectado. Limpia el archivo después de probar.

Prueba 3 — block-out-of-scope.

Pide al agente algo que toque algo fuera del repo:

"Edita /etc/hosts y añade una entrada para localhost.dev."

El hook `block-out-of-scope.sh` debe interceptar la edición y bloquearla. Si en Windows, sustituye `/etc/hosts` por una ruta absoluta del sistema (por ejemplo `C:\Windows\System32\drivers\etc\hosts`) — comprueba que los `FORBIDDEN_PATHS` cubren esa convención.

Si los tres bloqueos ocurren con mensaje claro, los hooks funcionan. Si alguno no, depura: revisa el log (`.claude/hooks.log` si lo configuraste, o `stderr` del cliente), comprueba que el matcher de `settings.json` cubre la herramienta correspondiente, y vuelve a probar.

Bloque 5 · Paso 5 — ajustar falsos positivos

Probablemente en las pruebas (o en uso normal después) van a aparecer casos donde el hook bloquea cosas legítimas. Tres ejemplos típicos en .NET:

El agente quiere ejecutar `dotnet clean` o `rm -rf bin/` durante un workflow normal de "limpiar y rebuildear". Si tu equipo considera esto rutinario, ajusta `block-destructive.sh` para no bloquear `rm -rf bin/` o `rm -rf obj/` específicamente (mantén el bloqueo para `rm -rf` con otros targets).

El agente reporta un falso positivo de secretos en un archivo de fixtures de tests donde sí quieres tener strings que parecen tokens. Solución: excluir `tests/**/fixtures/` o `tests/**/Resources/` del scan, o usar un marcador especial en el archivo (`# ALLOW-SECRET-PATTERN: test-fixture`) que el hook reconozca y deje pasar.

El agente quiere tocar `appsettings.Development.json` **legítimamente** para añadir una sección de configuración. Si ese archivo no está en `FORBIDDEN_PATHS`, no bloquea — pero si lo has añadido por error, ajusta. Solo `appsettings.Production.json` (o tus secretos reales) merecen bloqueo duro.

La lógica para todos los ajustes es la misma: cuando aparece un falso positivo legítimo, **refinas el patrón en lugar de quitar el hook**. Si quitas, abres el portillo para los positivos reales que el hook estaba previniendo.

Bloque 6 · Reflexión — qué decidiste tú

Mira atrás los cinco pasos. En cada uno hubo decisiones tuyas:

1. **En qué repo instalas los hooks** (práctica vs proyecto real).
2. **Qué patrones de .NET añades a las listas genéricas** del M4.3.
3. **Cuántas pruebas reales haces antes de confiar en los hooks** instalados.
4. **Cómo respondes a los falsos positivos** que aparecen (refinar vs quitar).
5. **Quién es el owner** declarado de cada hook en tu equipo.

Esas decisiones son las que convierten una instalación de hooks en una red de protección real. Si pasas por ellas con criterio, los hooks van a aportar. Si las saltas (copiar y olvidar, no probar, quitar al primer falso positivo), los hooks van a estar ahí sin proteger.

Bloque 7 · Cierre del M4 y enlace a M5

Has llegado al final del M4 con tres hooks defensivos operativos en tu repo. Junto con los hooks de validación del M4.2 (que probablemente no instalaste todavía), tienes la mayor parte de la red automática que SDD necesita para vivir en producción sin depender de tu atención al 100%.

Tres ideas para llevarte del módulo entero, antes de entrar en M5.

Los hooks son la red que sostiene la disciplina SDD cuando tu atención falla. No son el método: son el seguro del método. Bien diseñados, pasan desapercibidos hasta que aparece un problema; mal diseñados, generan fricción que el equipo aprende a desactivar.

Empezar pequeño es la regla. Un hook bien afinado vale más que cinco mal mantenidos. Después de un mes con un hook operativo, sabrás si añadir el siguiente o ajustar el primero.

Los hooks viven en el repo y tienen owner. Sin esas dos cosas (que parecen obvias y casi nadie hace), la protección es desigual y se rompe silenciosamente. Tráталos como código de primera clase y van a durar lo que dure el proyecto.

En el M5 entramos en MCP — Model Context Protocol. La pieza que conecta Claude Code con el mundo real: bases de datos, servicios externos, repositorios de GitHub, lo que tu sistema necesite tocar. Veremos cuándo MCP encaja (no siempre), cómo se ve un servidor MCP por dentro, cuáles merece la pena conocer hoy para SDD, y cuándo es razonable diseñar uno propio para tu dominio.

Glosario rápido del bloque

- **Hook defensivo:** hook que bloquea acciones potencialmente peligrosas por defecto. Defiende contra errores del agente más que validar cumplimiento del plan.
- **Falso positivo de hook:** cuando el hook bloquea algo legítimo. La respuesta correcta es refinar el patrón, no quitar el hook.
- **Ajuste por proyecto:** lo que diferencia un hook de plantilla genérica de un hook útil para tu solution concreta. Sin ajuste, los hooks aportan menos de lo que podrían.
- **Prueba de hook:** ejercicio explícito de disparar la acción que debería bloquear y comprobar que efectivamente bloquea. Sin pruebas, los hooks pueden estar instalados pero no funcionar.

Estado al cerrar el M4

Has visto: - Por qué los hooks importan (M4.1). - Tres hooks de validación contra spec (M4.2). - Tres hooks de protección (M4.3). - Hooks como activo de equipo (M4.4). - Práctica de instalación (M4.5).

Tienes la red completa para que Claude Code trabaje en SDD sin depender de tu supervisión continua. En el M5 añadimos el puente al mundo real con MCP.

PARTE 5

MCP · spec → herramientas tipadas

M5.1 · Qué es MCP y qué no es

Bloque 0 · El puente que faltaba

Hasta el M4 has visto al agente trabajando en el repositorio: lee archivos, escribe archivos, ejecuta comandos del shell. Eso ya cubre mucho terreno. Pero pronto te chocas contra el límite: el agente no sabe qué hay en tu base de datos, no puede abrir un issue de GitHub por su cuenta, no puede consultar el inventario de tu sistema interno.

Hay dos formas de salvar ese límite. La incómoda: copiar y pegar resultados de consultas, capturas de pantalla, listas extraídas a mano. Sirve para la primera semana; a la segunda te has cansado.

La otra es conectar al agente con esos sistemas externos a través de un **protocolo estándar**. Ese protocolo es MCP — **Model Context Protocol**. Lo lanzó Anthropic en 2024 y se ha vuelto el estándar de facto para que un modelo hable con herramientas externas sin que cada integración sea un ad hoc.

En este submódulo aterrizamos qué es y qué no es MCP, cuándo encaja en SDD, y cuándo es overkill. Quince minutos. Antes de meternos en cómo se ve un servidor MCP por dentro (M5.2) y qué MCPs útiles existen ya (M5.3), toca fijar el concepto.

Bloque 1 · Lo que MCP NO es

Tres confusiones comunes que merecen desmontarse antes de seguir.

MCP no es un modelo de IA. Es protocolo, no inteligencia. No hace al agente más listo. No mejora la calidad de las respuestas que el modelo genera. Lo que hace es ampliar qué puede tocar el agente — y al ampliarlo, los problemas que puede resolver crecen. Pero el razonamiento sigue siendo el mismo.

MCP no es magia. A veces se vende como "conectas MCP y tu agente lo entiende todo". No. Si conectas un MCP de base de datos sin tipado claro, el agente sigue inventando esquemas (o intenta tocar tablas que no existen). MCP da el canal; la calidad de lo que viaja por el canal depende de cómo esté construido el servidor.

MCP no es necesario para todo. Si el agente trabaja sobre tu repositorio y nada más, no necesitas MCP — las herramientas estándar (leer/escribir/ejecutar shell) cubren todo. MCP entra en escena cuando el flujo de trabajo cruza la frontera del repo.

Estas tres aclaraciones son importantes porque deshacen el hype y dejan la pregunta operativa correcta: "¿necesito MCP en este caso?". A veces sí, a veces no. Saber distinguir es la mitad del oficio.

Bloque 2 · Lo que MCP Sí es

Operativamente, MCP es un **protocolo estandarizado de comunicación entre un cliente (la aplicación que aloja al modelo, por ejemplo Claude Code) y un servidor (la herramienta que expone capacidades)**.

El servidor MCP es un proceso aparte que el cliente lanza. Hablan por un canal (stdio en local, HTTP/SSE para servidores remotos). El servidor declara qué capacidades expone; el cliente las descubre y las pone a disposición del modelo. El modelo, cuando necesita una de esas capacidades, la invoca a través del cliente, el cliente la traduce a una petición MCP al servidor, el servidor responde, y el resultado llega al modelo como contexto.

Cuatro piezas que tiene un servidor MCP:

Tools. Acciones que el servidor expone al modelo: "ejecuta esta consulta SQL", "crea este issue", "lee este fichero de configuración del sistema externo". Cada tool tiene una descripción y un esquema de inputs/outputs.

Resources. Datos que el servidor pone a disposición: contenidos de archivos, registros de una base, documentos de un wiki. El modelo puede pedir un resource concreto y el servidor lo devuelve.

Prompts. Plantillas de prompt parametrizadas que el servidor sugiere al usuario. Útil para flujos comunes — el equivalente a tener atajos para tareas frecuentes.

Sampling (opcional). El servidor puede pedir al modelo que genere texto durante su propio procesamiento. Menos común; no entras aquí en un primer aprendizaje.

El punto que importa: MCP es **plumbing tipada**. La diferencia con "ejecuto un script bash desde el agente" es que las tools MCP llevan esquemas explícitos. El modelo sabe qué input espera la tool y qué output devuelve, y el servidor puede validar la entrada antes de ejecutar. Eso reduce sorpresas: el agente no puede inventar parámetros que no estén en el esquema.

Bloque 3 · Cuándo encaja MCP en un flujo SDD

MCP rinde claramente en tres situaciones. Si tu caso está en una de las tres, considéralo. Si no, probablemente las herramientas estándar bastan.

Acceso a sistemas externos persistentes. Cuando tu flujo necesita leer o escribir en una base de datos, un repositorio de GitHub, un wiki corporativo, una cola de mensajes — cosas que viven fuera del repo y que el agente va a tocar repetidamente. Un MCP encapsula la conexión y la autenticación, y el agente trabaja contra una interfaz tipada en lugar de un cliente HTTP a mano cada vez.

Tareas que requieren contexto en tiempo real. Cuando la información cambia constantemente y el agente necesita la versión más reciente. Por ejemplo: estado actual de los tickets en Jira, último build de CI, métricas de monitoring. Un MCP de ese sistema da al agente acceso a la fotografía actual sin que tú la inyectes en el prompt.

Operaciones tipadas que se repiten. Cuando hay una operación que el agente va a hacer muchas veces y cada vez merece tipado estricto. Por ejemplo: crear un PR con cierta estructura, registrar un evento de auditoría, ejecutar una consulta sobre un esquema bien definido. Hacer esto con un MCP convierte una operación frágil ("genera el JSON correcto y haz POST") en una llamada tipada que el servidor valida.

En SDD, lo importante de estos tres casos es que **respetan el espíritu de "spec primero"**: la spec define qué operación quieres, el MCP la expone con tipos, y el agente la invoca cumpliendo el contrato. No hay improvisación de formato ni adivinación de parámetros.

Bloque 4 · Cuándo MCP es overkill

Igual de útil es saber cuándo NO meter MCP en la ecuación. Cuatro situaciones donde un MCP no aporta, y a veces estorba.

Operaciones puntuales únicas. Si vas a hacer una consulta a una base de datos una sola vez en la historia del proyecto, lanzar y mantener un servidor MCP completo es exagerado. Una consulta a mano, o un script que el agente lee y ejecuta, basta.

Sistemas donde el agente no necesita interacción directa. Si tu CI ya tiene un workflow que el equipo ejecuta cuando toca, no hay que dar al agente acceso al sistema de CI a través de MCP. Lo que vale el equipo, lo dispara el equipo.

Flujos donde el contexto humano es la sustancia. Si el agente solo necesita "saber qué hay en este Confluence" como referencia ocasional, copiar el contenido relevante en el prompt suele ser más rápido que montar un MCP de Confluence. El MCP merece la pena cuando la consulta se repite mucho.

Cuando añadir MCP duplica una herramienta ya disponible. Algunos clientes de Claude Code ya tienen integración nativa con GitHub, con sistemas de archivo, con shell. Antes de

instalar un MCP, comprueba si lo que necesitas ya está cubierto. Mejor dos herramientas bien que tres herramientas y un MCP que se pelean.

Una regla informal: **si tienes que invocar la misma operación externa más de cinco veces a la semana y quieres tipado en esa operación, MCP encaja**. Si no, probablemente puedes vivir sin él.

Bloque 5 · El panorama MCP a fecha de hoy

Una nota rápida sobre el ecosistema antes de cerrar. MCP es un estándar joven (lanzado a finales de 2024) y el catálogo de servidores crece cada mes. A fecha de redactar este material, los servidores más maduros incluyen:

Servidores oficiales de Anthropic. Filesystem, fetch (HTTP), git, GitHub, GitLab, PostgreSQL, SQLite, memoria local. Son los más rodados y los que primero salen al buscar.

Servidores comunitarios. Slack, Notion, Linear, Jira, Sentry, AWS, Google Drive, y muchos más. Calidad variable; los más populares suelen ser mantenidos por las propias empresas o por colaboradores activos.

Servidores específicos de .NET. Empiezan a aparecer MCPs para Azure (Azure Storage, Cosmos DB, Service Bus), para Entity Framework Core (consultar esquemas, ejecutar queries tipadas), y para inspeccionar solutions de .NET. El ecosistema .NET es activo aquí; merece estar pendiente.

Como con cualquier paquete que metes en tu proyecto, vale la regla del M3.2 "rechazar dependencias nuevas": cada MCP es una superficie de ataque y una dependencia más. Mejor empezar con uno y añadir solo cuando el siguiente claramente aporta.

Bloque 6 · Cierre

Tres ideas para llevarte.

MCP es protocolo, no inteligencia. Conectarlo no hace al agente más listo; le da acceso tipado a sistemas externos. Si tu flujo no cruza la frontera del repositorio, no lo necesitas.

Las tres situaciones donde MCP rinde son acceso a sistemas externos persistentes, tareas con contexto que cambia en tiempo real, y operaciones tipadas que se repiten. Fuera de ahí, las herramientas estándar suelen bastar.

El ecosistema MCP crece rápido pero todavía es joven. Mejor empezar con uno o dos servidores oficiales o bien rodados antes de instalar de todo. La protección que vimos en el M4.3 contra dependencias también aplica aquí: cada MCP es una superficie nueva.

Glosario rápido del bloque

- **MCP (Model Context Protocol):** protocolo estandarizado para que un cliente (que aloja al modelo) hable con servidores que exponen herramientas y datos. Lanzado por Anthropic en 2024.
- **Servidor MCP:** proceso aparte que expone tools, resources y prompts al cliente. Vive en local (stdio) o remoto (HTTP/SSE).
- **Tool:** acción tipada que un servidor MCP expone. Con esquema de inputs/outputs explícito.
- **Resource:** dato que un servidor MCP pone a disposición. Archivo, registro, documento.
- **Plumbing tipada:** lo que diferencia MCP de "ejecutar un script desde el agente". Los esquemas explícitos reducen sorpresas e imposibilitan parámetros inventados.

Qué viene en M5.2

Anatomía de un servidor MCP por dentro. Cómo se ve la declaración de tools, resources y prompts; tipado fuerte y por qué importa cuando es la spec quien genera el contrato MCP; modo lectura vs modo escritura. Un ejemplo simple para que se vea de qué hablamos cuando hablamos de "exponer una capacidad por MCP".

M5.2 · Anatomía de un servidor MCP

Bloque 0 · Para qué este desglose

Hasta aquí MCP era una palabra: protocolo estándar que conecta cliente y herramientas. Útil para no asustarse, insuficiente para diseñar.

Si vas a usar servidores MCP que ya existen (M5.3) y especialmente si vas a diseñar uno propio (M5.4), necesitas entender cómo se estructuran por dentro. Cuáles son las piezas que componen, cómo se declaran con tipado, cómo decides qué capacidad es tool y cuál es resource. Sin esa base, los servidores propios salen mal mezclando responsabilidades, y los ajenos los usas a ciegas sin saber qué esperar de cada operación.

En estos veinte minutos: las tres piezas que tiene cualquier servidor MCP, el tipado estricto que las hace fiables, cómo viaja la información entre cliente y servidor, la decisión clave entre operaciones de lectura y de escritura, y un ejemplo mínimo en .NET con el SDK oficial. Al terminar deberías sentirte cómodo mirando el código de un servidor MCP sin que parezca jeroglífico.

Bloque 1 · Las tres piezas — tools, resources, prompts

Un servidor MCP expone capacidades al cliente. Esas capacidades se dividen en tres categorías que toca tener mentalmente separadas, porque cada una sirve para una cosa distinta y mezclarlas confunde al cliente.

Tools. Son las acciones que el modelo puede invocar. "Ejecuta esta consulta", "crea este issue", "envía este mensaje", "actualiza este registro". Una tool tiene nombre, descripción, esquema de parámetros (lo que necesita para ejecutarse) y devuelve un resultado tipado.

Las tools son el corazón de un servidor MCP. Son lo que hace que el agente pueda *hacer* cosas más allá del repo. Cuando alguien dice "voy a montar un MCP para X", lo que está pensando son las tools que va a exponer.

Resources. Son datos que el servidor pone a disposición del modelo de forma legible. No son tools que el modelo invoque para obtener algo; son datos que el servidor declara que existen y el cliente puede pedirselos. La diferencia con tools es sutil pero importante: una resource es un contenido que *existe*, una tool es una operación que *se ejecuta*.

Ejemplos de resources típicos: el contenido de un archivo en un sistema remoto, un documento de un wiki, un registro concreto de una base de datos identificado por URI. El modelo puede listar las resources disponibles y pedir una concreta cuando le interesa.

Prompts. Son plantillas de prompt parametrizadas que el servidor sugiere al usuario. Son menos comunes que tools o resources, pero útiles cuando el servidor sabe de un flujo común y quiere ofrecerlo como atajo.

Por ejemplo, un MCP de un sistema de tickets podría exponer un prompt "investigar bug crítico" que toma como parámetro el ID del ticket y rellena automáticamente el contexto inicial: descripción del ticket, comentarios, sistemas afectados. El usuario invoca el prompt y arranca con todo ese contexto cargado en lugar de tener que prepararlo a mano.

Las tres piezas conviven en un servidor MCP. Algunos servidores tienen solo tools (lo más común). Otros tienen tools + resources. Pocos exponen prompts.

Bloque 2 · Tipado estricto, no opcional

El rasgo que distingue MCP de "ejecuto un script desde el agente" es el tipado.

Cada tool declara su esquema de inputs y de outputs. Cada resource declara su URI y su mime type. Cada prompt declara sus parámetros. El cliente recibe esos esquemas al conectar con el servidor y se los expone al modelo.

¿Por qué importa tanto el tipado?

Sin tipado, el modelo decide el formato de los parámetros con su criterio. A veces acierta, a veces se inventa campos, a veces usa mayúsculas donde el servidor espera minúsculas, a veces pasa fecha como string cuando el servidor esperaba timestamp. Cada uno de esos casos es un bug invisible — la llamada falla con un error críptico, el modelo intenta arreglarlo a ciegas, y empiezas a perder tiempo.

Con tipado, el modelo conoce de antemano el esquema. Sabe que `dueDate` espera string en formato ISO 8601, no DateTime nativo. Sabe que `priority` solo acepta `"low"`, `"medium"`, `"high"`. Sabe que `userId` es un GUID. Sabe qué excepciones puede recibir y cómo distinguirlas. La invocación sale limpia a la primera o, si falla, falla con un mensaje claro que el modelo puede interpretar y corregir.

El esquema se declara en JSON Schema típicamente, pero los SDKs (incluido el de .NET) te dejan declararlo con tipos del lenguaje y se encargan de generar el JSON Schema correspondiente. Eso es muy importante en práctica: declaras tu tool con un record C# y el SDK hace el resto.

Volviendo al hilo SDD del curso: este tipado conecta directamente con la idea del **contract** que vimos en el M2.1. Una tool MCP bien diseñada es esencialmente una pieza de contract ejecutable. Si tienes la spec de la operación, generar el tool MCP es traducción mecánica.

Bloque 3 · Cómo viaja la información entre cliente y servidor

Por dentro, MCP usa JSON-RPC 2.0 como protocolo de mensajes. Cliente y servidor se intercambian objetos JSON estructurados según ese estándar.

El transporte por debajo varía:

Stdio. El cliente lanza el servidor como proceso hijo y se comunica por la entrada/salida estándar. Es el modo típico para servidores que viven en local. Sencillo, no necesita configuración de red.

HTTP con SSE (Server-Sent Events). Para servidores remotos. El cliente se conecta a una URL y mantiene un canal abierto para recibir notificaciones del servidor. Útil cuando el servidor tiene que existir como servicio independiente accesible por varios clientes a la vez.

Streamable HTTP. Variante más reciente que generaliza el modelo HTTP. En crecimiento.

Para el primer servidor MCP que diseñes, stdio es lo natural: lanzas el ejecutable, el cliente lo arranca, y se hablan. Cero infraestructura.

Lo que viaja por el canal son llamadas tipo `tools/call`, `resources/read`, `prompts/get`, `initialize` y respuestas a esas llamadas. No necesitas conocer el protocolo a fondo si usas un SDK; los SDK encapsulan la sintaxis y tú solo declaras las capacidades.

Bloque 4 · Modo lectura vs modo escritura — la decisión clave del diseño

Cuando diseñas un servidor MCP, una de las primeras decisiones es **qué tools son de lectura y cuáles de escritura**. Esa distinción importa mucho más de lo que parece, sobre todo desde la perspectiva de los hooks que vimos en M4.

Tools de lectura. Consultan pero no modifican estado. Ejemplos: "lista los tickets abiertos", "lee la spec actual", "obtén el esquema de esta tabla". Pueden invocarse libremente porque no tienen efectos secundarios. El peor caso de una invocación errónea es información ruidosa.

Tools de escritura. Modifican estado externo. Ejemplos: "crea este ticket", "actualiza esta configuración", "envía este mensaje". Una invocación errónea puede tener consecuencias reales:

ticket duplicado, configuración rota, mensaje enviado a quien no era.

Las tools de escritura merecen tratamiento distinto en cuatro aspectos:

Confirmación humana. El hook del M4.3 que bloquea acciones irreversibles aplica también a tools MCP. Una tool que envía un correo a un cliente debería pasar por confirmación humana siempre. Una tool que crea un PR puede pasar tras revisar el plan.

Idempotencia siempre que se pueda. Si la tool se puede invocar dos veces y el resultado es el mismo que invocarla una, vas a sufrir menos errores. "Crear ticket si no existe con este ID" es idempotente; "crear ticket" no.

Mensajes de error claros. Cuando una tool de escritura falla, el modelo va a intentar corregir e invocar otra vez. Si el mensaje de error es críptico, el modelo improvisa. Si es claro, el modelo entiende qué pasó y ajusta. "Faltaba el campo X requerido" es mejor que "400 Bad Request".

Auditoría. Las tools de escritura deberían dejar rastro: quién invocó (qué cliente), cuándo, con qué parámetros. Cuando algo va mal, el log de auditoría es lo que te permite reconstruir.

Un buen servidor MCP separa muy claramente las dos categorías en su diseño y en su documentación. Cuando un servidor mezcla operaciones de lectura y escritura sin distinguir, los hooks de protección no pueden tratarlas de forma asimétrica y vuelves al problema de "confiar en todo o desconfiar de todo".

Bloque 5 · Un servidor MCP mínimo en .NET

Vamos a ver un ejemplo mínimo. Usaremos el SDK oficial de MCP para .NET (`ModelContextProtocol` en NuGet) sobre .NET 10. El servidor expone una sola tool: contar los tests de un proyecto de tests xUnit.

```
// Program.cs
using ModelContextProtocol.Server;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.DependencyInjection;

var builder = Host.CreateApplicationBuilder(args);

builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

var host = builder.Build();
await host.RunAsync();
```

```
// Tools/TestCountTool.cs
using System.ComponentModel;
using System.Text.RegularExpressions;
using ModelContextProtocol.Server;

[McpServerToolType]
public static class TestCountTool
{
    [McpServerTool, Description("Cuenta los tests xUnit en un proyecto de tests .NET.")]
    public static async Task<TestCountResult> CountTests(
        [Description("Ruta al proyecto de tests (carpeta que contiene el .csproj).")]
        string projectPath)
    {
        if (!Directory.Exists(projectPath))
            throw new DirectoryNotFoundException($"No existe el directorio: {projectPath}");

        var files = Directory.GetFiles(projectPath, "*.cs",
            SearchOption.AllDirectories);
        int factCount = 0;
        int theoryCount = 0;

        foreach (var file in files)
        {
            var content = await File.ReadAllTextAsync(file);
            factCount += Regex.Matches(content, @"\[Fact\b").Count;
            theoryCount += Regex.Matches(content, @"\[Theory\b").Count;
        }

        return new TestCountResult(
            Project: Path.GetFileName(projectPath),
            FactTests: factCount,
            TheoryTests: theoryCount,
            TotalTests: factCount + theoryCount);
    }
}

public record TestCountResult(string Project, int FactTests, int TheoryTests, int TotalTests);
```

Tres cosas que mirar en este ejemplo:

El `[McpServerToolType]` y `[McpServerTool]` marcan qué clase y qué métodos expone el servidor. El SDK escanea el assembly y los registra automáticamente. No tienes que escribir JSON Schema a mano.

Los `[Description]` sobre el método y los parámetros son la documentación que el cliente entrega al modelo. **Importan mucho:** el modelo decide si invocar la tool y cómo basándose en esas descripciones. Una descripción vaga produce invocaciones erróneas; una descripción precisa, no.

El tipo de retorno es un `record`. El SDK lo serializa con sus campos tipados. El modelo recibe un objeto estructurado, no un string con valores.

Para ejecutar este servidor en local, lo compilas (`dotnet build`), y configuras tu cliente Claude Code en `.mcp.json` (o el archivo equivalente):

```
{
  "mcpServers": {
    "test-counter": {
      "command": "dotnet",
      "args": ["run", "--project", "src/TestCounter.Mcp/TestCounter.Mcp.csproj"]
    }
  }
}
```

Y desde Claude Code podrás pedirle "cuenta los tests del proyecto X" y el modelo invocará la tool `count_tests` del servidor MCP.

No es mucho código. Esa es la idea: un servidor MCP no tiene que ser un proyecto enorme. Puede ser cien líneas que exponen una operación útil, bien tipada, lista para que el agente la consuma.

Bloque 6 · Errores comunes al diseñar

Cuatro patrones que veo cuando alguien diseña su primer servidor MCP.

Tool demasiado genérica. "Ejecuta cualquier consulta SQL contra esta DB". Funciona pero abre la puerta a que el modelo escriba SQL incorrecto, peligroso o sin tipar. Mejor varias tools específicas: "lista clientes activos", "obten detalles de cliente por ID", "cuenta pedidos del mes". Cada una con su tipo de salida.

Descripción ambigua. El `[Description]` dice "hace operación con datos". El modelo no sabe cuándo invocarla. Las descripciones tienen que ser explícitas: qué hace, qué espera, qué devuelve, qué NO hace. Tres líneas de descripción valen oro.

Falta de validación de inputs. El servidor recibe la invocación y no valida los parámetros. Cuando el modelo se equivoca con el formato, el servidor falla con error críptico. Validación temprana + mensaje claro evita media docena de iteraciones.

Mezclar lectura y escritura en una sola tool. "Actualiza el cliente y devuelve los nuevos datos". Si el cliente solo quería leer, no esperaba que se actualizara nada. Mejor separar: una tool de lectura, otra de escritura. La regla de los hooks aplica: si es irreversible, tiene que ser identificable como tal por su nombre.

Bloque 7 · Cierre

Tres ideas para llevarte.

Un servidor MCP tiene tres piezas (tools, resources, prompts) y las tres conviven sin pisarse cuando están bien diseñadas. Lo más común es tener solo tools, ampliar con resources cuando el servidor expone datos identificables por URI, y añadir prompts cuando el flujo común es estable.

El tipado estricto es el rasgo que diferencia MCP de "ejecutar scripts desde el agente". Es lo que conecta MCP con el `contract` que vimos en el M2.1: una tool bien diseñada es una pieza de contract ejecutable.

El SDK de MCP para .NET hace que diseñar un servidor sea trabajo de un día, no de una semana. Atributos sobre métodos, descripciones claras, records como tipos de retorno, y el SDK genera todo el plumbing. Si entiendes los conceptos, escribir el código es la parte fácil.

Glosario rápido del bloque

- **Tool:** acción que un servidor MCP expone al modelo. Tipada, con esquema de inputs/outputs.
- **Resource:** dato que un servidor MCP pone a disposición del modelo, identificado por URI.
- **Prompt (en MCP):** plantilla parametrizada de prompt que el servidor sugiere al usuario. Atajo para flujos comunes.
- **Stdio transport:** modo de comunicación cliente-servidor MCP en local, vía entrada/salida estándar.
- **HTTP/SSE transport:** modo de comunicación para servidores MCP remotos accesibles por red.
- **JSON-RPC 2.0:** protocolo de mensajes que MCP usa por debajo. Encapsulado por el SDK.

Qué viene en M5.3

MCPs concretos útiles para flujos SDD en .NET. Filesystem y fetch (oficiales de Anthropic). GitHub MCP para gestionar issues y PRs. MCPs de base de datos (PostgreSQL, SQL Server) para consultar esquemas reales sin que el agente alucine. Mención específica de la familia emergente de MCPs para Azure y Entity Framework Core. Criterios para decidir cuál instalar y cuándo el equipo ya tiene suficiente sin añadir más.

M5.3 · MCPs útiles para SDD en .NET

Bloque 0 · Lo que vamos a ver

Después de los dos submódulos teóricos (M5.1 qué es MCP, M5.2 cómo se ve por dentro), toca catálogo. Cuáles son los MCPs que un dev .NET haciendo SDD se va a encontrar útiles a corto plazo, cuáles puede ignorar de momento, y qué criterio aplicar cuando aparezca uno nuevo.

Veinte minutos. Recorrido: los dos MCPs base de Anthropic (filesystem, fetch), el de GitHub que casi todos acaban instalando, los MCPs de base de datos que evitan que el agente alucine esquemas, la familia emergente específica de .NET (Azure, EF Core, solution), y al final un criterio práctico de decisión.

Importante recordar el M5.1: cada MCP es una dependencia. La regla "lo reversible se permite, lo irreversible se bloquea" del M4 aplica también al panorama MCP. Mejor empezar con uno solo y añadir cuando claramente aporta.

Bloque 1 · Filesystem y fetch — la base

Estos dos vienen oficialmente con Anthropic y son los primeros que casi todo el mundo prueba. Suelen estar disponibles como paquetes listos para instalar; no tienes que escribirlos.

Filesystem MCP. Expone operaciones sobre el sistema de archivos: leer, escribir, listar, buscar. Suena redundante porque Claude Code ya tiene herramientas estándar para esto. ¿Cuándo aporta el MCP? Cuando necesitas exponer al agente **rutas fuera del directorio actual de trabajo de forma controlada**. Por ejemplo: el agente trabaja en `D:\proyectos\miapp\` pero necesita leer cierta plantilla en `D:\plantillas-corporativas\`. En lugar de cambiar el directorio de trabajo, configuras el MCP filesystem para que exponga las dos rutas a la vez.

Para SDD en .NET, este MCP encaja cuando tu organización tiene plantillas compartidas, configuraciones de equipo o documentación en repos externos que el agente consulta de forma frecuente.

Fetch MCP. Permite al agente hacer peticiones HTTP. Útil cuando necesita consultar documentación pública, descargar especificaciones OpenAPI de servicios externos, o leer páginas web de referencia. Para SDD, encaja en la fase de investigación previa a escribir una spec: "léete la doc de la API que vamos a consumir y resúmemela en behaviour para mi spec".

Aviso de seguridad: el MCP fetch puede llamar a cualquier URL. Si tu agente está en un entorno con acceso a recursos internos sensibles, el MCP fetch puede ser puerta de entrada a peticiones que no quieres. Configura listas de dominios permitidos si te preocupa, o limita su uso a sesiones donde lo activas explícitamente.

Bloque 2 · GitHub MCP — tickets, PRs y código compartido

El MCP de GitHub es probablemente el más rentable para un dev en equipo. Expone operaciones sobre repos: leer issues, crear PRs, listar discusiones, consultar workflows de Actions, descargar archivos de otros repos.

Tres flujos SDD donde aporta de inmediato:

De ticket a spec. El equipo crea un issue describiendo lo que necesita. En lugar de copiar el texto del issue al prompt manualmente, el agente lo lee directamente con el MCP. Cuando escribes la spec en `specs/feature.md`, el agente puede consultar el ticket completo, los comentarios, los archivos vinculados.

De spec a PR. Cuando terminas la implementación, el agente puede abrir el PR siguiendo la convención del equipo (título, descripción, labels, reviewers). En lugar de generar el contenido y pedirte que abras el PR a mano, lo abre y te pasa el enlace para que lo revises antes de mergear.

Consulta de código en otros repos. Para organizaciones con varios repos, el agente puede consultar cómo se resolvió un problema parecido en otro repo del equipo. Útil cuando hay patrones internos no documentados.

Configuración típica: el MCP de GitHub necesita un token con los permisos mínimos necesarios. Para SDD, normalmente bastan permisos de lectura sobre los repos del equipo y permisos de escritura solo sobre el repo del proyecto activo. Mejor empezar restrictivo y ampliar solo cuando aparezca el caso.

Hooks que rinden bien combinados con este MCP: bloqueo automático de operaciones de escritura sobre `main` o ramas protegidas. El agente puede leer todo pero solo modifica con confirmación explícita.

Bloque 3 · MCPs de base de datos — el remedio contra la alucinación de esquemas

Aquí entra una de las molestias más típicas del agente trabajando con código .NET: alucinar esquemas de base de datos. Le pides una migración de EF Core, el agente se inventa nombres de

columnas que no existen, tipos que no encajan, relaciones que ni siquiera están en el modelo. Si la spec no contenía el esquema completo, el modelo se lo inventa basándose en convenciones plausibles.

Un MCP de base de datos arregla esto de raíz. El servidor MCP se conecta a la base real (o a una réplica de desarrollo), expone las tablas, columnas y relaciones como tools y resources, y el agente trabaja contra el esquema real.

PostgreSQL MCP. Oficial de Anthropic. Expone consultas read-only contra la base. Útil para SDD cuando trabajas con PostgreSQL detrás de EF Core: el agente puede consultar el esquema actual antes de escribir migraciones, comprobar si una columna existe, verificar tipos antes de generar código de mapping.

SQL Server MCP. En el ecosistema comunitario hay varios. Calidad variable; recomiendo evaluar el que esté más mantenido a fecha de instalación. Mismo razonamiento que con PostgreSQL: leer el esquema real evita decisiones inventadas.

SQLite MCP. Útil para proyectos pequeños o tests locales donde la base es SQLite. Para SDD, conecta bien con el patrón de tener una base de pruebas con un dataset realista contra el que el agente puede validar comportamiento durante el desarrollo.

Regla de oro al instalar un MCP de DB: **modo read-only por defecto**. Las migraciones y modificaciones reales las haces tú o las ejecuta el agente con autorización explícita, no a través del MCP. El MCP es para consulta, no para mutación silenciosa.

Para el caso .NET + EF Core específicamente: el flujo más útil es "lee el esquema real de la base, comprueba que el modelo en `DbContext` coincide, señala discrepancias". Eso pilla problemas que el agente no vería sin acceso a la base.

Bloque 4 · Familia .NET emergente — Azure, EF Core, solution

A fecha de redactar este material, hay una familia emergente de MCPs específicos para .NET que merece atención. Crece rápido; lo que aquí escribo es válido para la versión actual, pero comprueba el estado cuando llegues a este punto del curso.

Azure MCP. Microsoft está empujando integraciones MCP para servicios de Azure: Storage, Cosmos DB, Service Bus, Application Insights, Key Vault. Si tu proyecto está en Azure (cosa común para .NET), estos MCPs permiten al agente consultar configuraciones, lanzar consultas a Cosmos, leer métricas de App Insights — todo sin que tengas que pegar resultados en el prompt.

Aviso de seguridad redoblado aquí: estos MCPs hablan con producción si los configuras contra producción. Mejor conectarlos siempre contra entornos de desarrollo, y solo contra producción

con permisos mínimos y bloqueos en hooks.

EF Core MCP. Hay variantes comunitarias que exponen operaciones sobre el `DbContext` : listar entidades del modelo, consultar relaciones, generar consultas tipadas, ejecutar migraciones (con confirmación). Útil cuando trabajas en un dominio con muchas entidades y quieres que el agente respete el modelo existente sin tener que recordárselo cada vez.

Solution / project MCP. Algunos MCPs comunitarios exponen operaciones sobre la solution .NET: listar proyectos, sus referencias, sus paquetes NuGet, ejecutar `dotnet build` por proyecto, leer el `Directory.Build.props` . Útil para proyectos grandes donde el agente se pierde entre proyectos.

El criterio: si tu proyecto es estándar (una solution mediana, EF Core, Azure básico), prueba primero sin estos MCPs y mide dónde el agente se atasca. Si los puntos de fricción son recurrentes (alucina nombres de servicios de Azure, malinterpreta el modelo de EF Core), añade el MCP correspondiente. No instales por anticipado.

Bloque 5 · Criterios para decidir cuál instalar

Una decisión consciente vale más que un catálogo memorizado. Cinco criterios que aplico cuando aparece un MCP nuevo en mi radar:

¿Resuelve un problema que tengo hoy, no uno hipotético? Si nunca te has atascado por la causa que el MCP resuelve, no lo instales. La regla es "instalar para tapar un agujero conocido", no "instalar por si acaso".

¿Está mantenido? Repos sin commits en seis meses, sin issues respondidos, sin documentación clara, son señal de abandono. Mejor un MCP menos potente pero mantenido que uno completo pero muerto.

¿Es read-only por defecto? Para los MCPs que tocan sistemas externos (DB, Azure, GitHub con escritura), el modo seguro de empezar es lectura. Si el MCP no permite distinguir lectura de escritura, o requiere configuración compleja para hacerlo, ese es un riesgo a evaluar.

¿Encaja con los hooks que tienes activos? Si tienes un hook que bloquea operaciones destructivas, el MCP nuevo no puede saltárselo. Comprueba que las tools del MCP que generan mutaciones pasan por las protecciones del M4. Si las saltan, hay que ajustar hooks.

¿Aporta tipado real, o es disfraz? Un MCP que expone una única tool "ejecutar cualquier comando contra X" no aporta tipado — solo encapsula. Los MCPs que valen son los que descomponen una operación grande en varias tools tipadas con descripciones claras. Si el MCP es solo una capa fina sobre un comando shell, probablemente es overkill.

Aplica los cinco al evaluar un MCP. Si fallan dos o más, descarta o pospone. Si pasa los cinco, instala con configuración mínima necesaria y observa cómo encaja en el flujo durante una o dos sesiones.

Bloque 6 · Una mención breve a MCPs de productividad

Una nota antes de cerrar. Hay un conjunto creciente de MCPs orientados a productividad personal: Slack, Notion, Linear, Google Drive, calendarios. Pueden ser útiles para flujos donde el agente actúa como asistente más amplio que solo programación.

Para SDD puro, mi recomendación es prudencia. Estos MCPs introducen al agente en sistemas con información sensible (mensajes privados, documentos del equipo). Si los activas, hazlo con permisos mínimos y solo en sesiones donde tiene sentido — no como configuración permanente.

Si vas a usar Linear o Jira como sistema de tickets en lugar de GitHub Issues, el patrón es el mismo que con GitHub MCP: leer tickets para alimentar specs, abrir tickets para registrar trabajo nuevo. La regla de "lectura libre, escritura con confirmación" sigue valiendo.

Bloque 7 · Cierre

Tres ideas para llevarte.

La pregunta correcta antes de instalar un MCP no es "qué hace este servidor", sino "qué problema mío resuelve". Instalar por catálogo te llena el repo de dependencias que no usas; instalar por problema te da herramientas que tu equipo usa de verdad.

Para SDD en .NET, los tres MCPs que más rinden a corto plazo son **filesystem** (extender el alcance del agente fuera del directorio actual con control), **GitHub** (flujo ticket↔spec↔PR sin copy-paste manual) y **un MCP de base de datos** (evita que el agente alucine esquemas, especialmente con EF Core). Empieza por uno y añade conforme veas el siguiente agujero.

Los MCPs son piezas vivas como skills y hooks. Tienen owner, se mantienen al día, se retiran cuando dejan de aportar. La revisión trimestral del M4.4 aplica también al catálogo MCP: ¿lo seguimos usando, sigue siendo correcto, sigue siendo necesario?

Glosario rápido del bloque

- **Filesystem MCP**: servidor MCP oficial de Anthropic que expone operaciones de sistema de archivos sobre rutas configurables. Útil para extender el alcance del agente fuera del cwd.

- **GitHub MCP:** servidor oficial que expone operaciones sobre repos de GitHub. Permite leer issues, abrir PRs, consultar workflows.
- **MCP de base de datos:** servidores (oficiales o comunitarios) que exponen el esquema y consultas read-only contra una base. Evita que el agente alucine columnas y tipos.
- **Familia .NET emergente:** conjunto de MCPs específicos para el ecosistema .NET — Azure, EF Core, solution. Joven, crece rápido, evaluar al instalar.
- **Modo read-only por defecto:** configuración recomendada al instalar un MCP que toca sistemas externos. La escritura se autoriza explícitamente, no por defecto.

Qué viene en M5.4

Cómo diseñar un MCP propio desde una spec. Cuando los MCPs existentes no cubren lo que necesitas, toca construir. Veremos cómo va la traducción de una spec del dominio a tools MCP tipadas, los errores típicos al diseñar (tool genérica de más, descripción ambigua, validación ausente, mezcla de lectura y escritura) y un patrón de organización para servidores propios que crecen con el proyecto.

M5.4 · Diseñar un MCP propio desde una spec

Bloque 0 · Cuándo construir tu propio MCP

Antes de meterte a construir, una pregunta honesta: ¿de verdad necesitas un MCP propio?

Los criterios del M5.1 (cuándo encaja MCP) seguían siendo válidos: acceso a sistemas externos persistentes, contexto en tiempo real, operaciones tipadas repetidas. Si tu caso no cumple ninguno, no necesitas MCP — ni propio ni ajeno. Y si cumple alguno, antes de diseñar, revisa el catálogo del M5.3 una vez más.

Asumamos que ya descartaste los MCPs existentes. Tu organización tiene un sistema interno (gestor documental propio, ERP, plataforma de tickets a medida) que es relevante para el flujo SDD, no hay MCP público, y el agente lo necesita tocar de forma repetida. Es el momento de construir.

En estos veinte minutos: cómo se traduce una spec de dominio a tools tipadas, los cuatro errores típicos al diseñar el primer MCP, un patrón de organización para que el servidor crezca sin convertirse en un monolito, y notas de versión y despliegue. Al terminar deberías saber abordar tu primer servidor MCP propio en .NET sin atascarte en los puntos donde la mayoría se atasca.

Bloque 1 · Punto de partida — la spec del dominio

Aquí está la idea importante que conecta todo el M5 con SDD: **un MCP propio es la materialización ejecutable de una spec de dominio**. Si tienes la spec, el diseño del MCP es mecánico. Si no la tienes, el MCP va a heredar las debilidades de la spec ausente.

Por eso, antes de tocar código, escribe (o consulta) la spec de las operaciones que vas a exponer. Si tu dominio ya tiene una spec madura, perfecto. Si no, escríbela con la plantilla A del M2.3, con cuatro o cinco operaciones que el MCP va a soportar.

Estructura típica de esa spec, que vamos a usar como entrada del diseño:

Spec: MCP del gestor documental interno

Intent

El gestor documental interno almacena documentos del proyecto (specs, ADRs, actas de reunión). El equipo necesita que el agente pueda consultar y referenciar esos documentos sin descargarlos manualmente cada vez. La frecuencia es alta (varias consultas por sesión de SDD) y la API REST que hoy expone es engorrosa.

Behavior (Operaciones que el MCP va a soportar)

- Listar documentos por tipo y fecha.
- Obtener el contenido de un documento concreto por ID.
- Buscar por término en el contenido (search).
- Crear un nuevo documento (solo si el usuario lo confirma).
- Marcar un documento como obsoleto (solo si el usuario lo confirma).

Contract

[Tipos de entrada/salida de cada operación, con campos requeridos y opcionales.]

Criterios de aceptación

- [] Las operaciones de lectura no requieren confirmación.
- [] Las operaciones de escritura piden confirmación humana.
- [] El MCP devuelve errores con mensaje legible para el modelo.
- [] El servidor arranca en menos de un segundo.
- [] Los tests unitarios cubren cada operación.

Fuera de alcance

- El MCP no expone operaciones de administración del gestor (gestión de usuarios, permisos, configuración).
- El MCP no descarga binarios (solo metadatos y texto).

Con esta spec en mano, las cinco operaciones del Behavior se convierten en cinco tools del MCP. Cada una con su tipo de entrada y salida según el Contract.

Bloque 2 · De operación a tool — el proceso de traducción

Para cada operación de la spec, el diseño del tool sigue el mismo patrón:

Nombre de la tool. Verbo claro + objeto + condición si la hay. `list_documents`, `get_document_content`, `search_documents`, `create_document`, `mark_document_obsolete`. Snake case por convención MCP.

Descripción. Una a tres frases. Qué hace, en qué casos invocarla, qué NO hace. Esta descripción es lo que el modelo lee para decidir cuándo invocar la tool — vale más cuidado que el nombre.

Tipo de entrada. Un record .NET con campos tipados y atributos `[Description]` en los campos importantes. Lo que es opcional se marca como nullable; lo que es requerido se queda no nullable.

Tipo de salida. Un record con el resultado. Estructurado, tipado. No devuelvas strings sueltos — dificultan al modelo decidir qué hacer con el resultado.

Validación. Antes de ejecutar la operación real, valida los inputs. Si falta un campo requerido o un valor está fuera de rango, devuelve un error claro. El modelo va a leer ese error y ajustar la siguiente invocación.

Distinción lectura/escritura. Las tools de lectura van por defecto, sin confirmación. Las tools de escritura llevan un parámetro adicional `confirmed: bool` o equivalente, y el servidor las rechaza si llega `false`. Eso obliga al cliente (Claude Code) a pasar por una confirmación humana explícita.

Ejemplo de cómo se ve una tool simple en .NET con el SDK de MCP:

```
[McpServerToolType]
public static class DocumentTools
{
    [McpServerTool, Description("Lista documentos del gestor por tipo y rango de
    fechas. Devuelve metadatos, no contenido.")]
    public static async Task<DocumentListResult> ListDocuments(
        [Description("Tipo de documento: spec, adr, acta, otro.")] string type,
        [Description("Fecha desde (formato ISO 8601). Opcional.")] string? fromDate
    = null,
        [Description("Fecha hasta (formato ISO 8601). Opcional.")] string? toDate =
    null,
        [Description("Número máximo de resultados. Por defecto 50.")] int maxResults
    = 50,
        IDocumentService service = null!)
    {
        if (string.IsNullOrEmpty(type))
            throw new ArgumentException("El parámetro 'type' es requerido.");

        if (maxResults < 1 || maxResults > 200)
            throw new ArgumentException("'maxResults' debe estar entre 1 y 200.");

        DateTimeOffset? from = fromDate is null ? null :
        DateTimeOffset.Parse(fromDate);
        DateTimeOffset? to = toDate is null ? null : DateTimeOffset.Parse(toDate);

        var docs = await service.ListAsync(type, from, to, maxResults);

        return new DocumentListResult(
            Type: type,
            Count: docs.Count,
            Documents: docs.Select(d => new DocumentSummary(
                Id: d.Id,
                Title: d.Title,
                Type: d.Type,
                ModifiedAt: d.ModifiedAt)).ToList());
    }
}

public record DocumentSummary(string Id, string Title, string Type, DateTimeOffset
    ModifiedAt);
public record DocumentListResult(string Type, int Count,
    IReadOnlyList<DocumentSummary> Documents);
```

Tres detalles importantes:

Las descripciones son explícitas. "Devuelve metadatos, no contenido" cierra una ambigüedad obvia (¿devuelve el texto?). Si lo dejaras sin clarificar, el modelo podría asumir que sí y luego sorprenderse cuando otra tool (`get_document_content`) sea necesaria.

La validación está al inicio. Si los parámetros no cumplen, lanzamos excepción antes de tocar el servicio. La excepción se convierte en mensaje de error MCP que el modelo entiende.

Las dependencias se inyectan. `IDocumentService` representa la capa de acceso al gestor real. El MCP es una fachada delgada sobre ese servicio. Eso permite testear el MCP sin tocar la base de

datos real.

Bloque 3 · Cuatro errores típicos al diseñar

Repetidos del M5.2 con más detalle ahora que tenemos contexto de "estás diseñando el tuyo".

Tool demasiado genérica. El antipatrón: una sola tool `execute_query` que recibe una query string y la ejecuta. Funciona, pero deja al modelo decidir el formato exacto y abre la puerta a queries malformadas o peligrosas. Mejor: varias tools específicas, una por operación de la spec.

Si te encuentras tentado de crear una tool genérica "para no repetir trabajo", suele ser señal de que tu spec de dominio era débil — no habías nombrado las operaciones concretas. Vuelve a la spec, nombra las operaciones, y de ahí salen las tools naturales.

Descripción ambigua. "Hace cosas con documentos". El modelo no sabe cuándo invocarla. Reescribe con criterio: verbo activo, objeto concreto, qué espera, qué devuelve, qué NO hace. Tres líneas de descripción valen oro.

Un truco: lee la descripción asumiendo que eres alguien que no ha programado este MCP y solo sabe que existe. ¿Entiendes para qué sirve? Si no, falta claridad.

Falta de validación de inputs. El servidor recibe la invocación y llama directamente al servicio interno con los parámetros sin tocar. Cuando el modelo se equivoca con el formato, el servicio falla en una capa profunda con stack trace cuyo significado solo entiendes tú. Mejor: validación al inicio del método con mensaje claro. El modelo lee el error, ajusta, vuelve a invocar.

Validación recomendada por defecto: requeridos presentes, formatos correctos (fechas, enums, IDs), rangos sensatos (paginación, límites). No hace falta validación de negocio en el MCP — eso lo hace el servicio detrás —, sí validación de contrato.

Mezcla de lectura y escritura. Un mismo tool que "obtiene un documento y, si tiene un campo de tags, actualiza los tags con los nuevos". El cliente que solo quería leer no esperaba mutación. Mejor separar: una tool de lectura, otra de actualización. La regla de los hooks aplica: si es irreversible, el nombre lo tiene que dejar claro y el flujo requiere confirmación.

Bloque 4 · Patrón de organización para un MCP propio que crece

Tu primer MCP probablemente tiene tres o cuatro tools. Cuando empieza a crecer (porque el equipo ve que va bien y pide más), si no tienes patrón de organización, el código degenera en un solo archivo con veinte tools, cada una con sus dependencias entrelazadas. Mantenerlo se vuelve doloroso.

Tres reglas de organización que rinden bien:

Un archivo por área temática. Si tu servidor MCP cubre documentos, usuarios y configuración, son tres archivos (`DocumentTools.cs`, `UserTools.cs`, `ConfigTools.cs`). Cada uno con su clase `[McpServerToolType]`. El SDK los descubre automáticamente con `WithToolsFromAssembly()`.

Inyección de dependencias por servicio. El MCP no debe hablar directamente con la base de datos ni con APIs externas. Habla con servicios (`IDocumentService`, `IUserService`) inyectados por DI. Eso desacopla el MCP de la implementación real y facilita test.

Tests por tool. Cada tool merece su test que verifique: caso normal (con mock del servicio), caso de validación inválida (con mock que comprueba que no se llama al servicio), caso de error del servicio (con mock que lanza excepción). Tres tests por tool es razonable; cinco son demasiados para algo que es una fachada delgada.

Estructura de carpetas típica para un MCP propio en .NET:

```
src/MiOrg.GestorDocumental.Mcp/  
  Program.cs                # arranque + DI  
  Tools/  
    DocumentTools.cs  
    UserTools.cs  
    ConfigTools.cs  
  Models/  
    DocumentSummary.cs  
    DocumentListResult.cs  
    ...  
  Services/  
    IDocumentService.cs    # interfaz  
    DocumentService.cs     # implementación real  
  appsettings.json          # config  
tests/MiOrg.GestorDocumental.Mcp.Tests/  
  DocumentToolsTests.cs  
  UserToolsTests.cs  
  ConfigToolsTests.cs
```

No es estructura monumental. Para un MCP con ocho o diez tools, esto cabe en doscientas o trescientas líneas de código real. Es trabajo de un día o dos para la primera versión.

Bloque 5 · Versión, despliegue y mantenimiento

Una vez tienes el MCP funcionando, vas a querer que el equipo lo use. Cuatro notas operativas para que no se quede en "funciona en mi máquina":

Versionado semántico. Cada cambio que rompe el esquema de una tool es mayor. Cada tool nueva o campo nuevo opcional es menor. Cada arreglo interno es parche. Refleja esto en el

`.csproj` y en el README del MCP.

Distribución. El MCP es un ejecutable .NET. Para distribución interna, lo subes a un feed de NuGet privado como herramienta global (`dotnet tool install --global MiOrg.GestorDocumental.Mcp`). El equipo lo instala con un comando, lo actualiza con `dotnet tool update`. Cero copy-paste de binarios.

Configuración por entorno. El MCP necesita saber a qué instancia del sistema interno conectar. Variables de entorno o `appsettings.json` por entorno (`Development.json`, `Production.json`). El cliente Claude Code apunta al ejecutable en su `.mcp.json` y le pasa la configuración por argumentos o variables.

Owner declarado. Como con skills y hooks (M3.3, M4.4), cada MCP propio tiene owner declarado en su README. Cuando algo falla o cuando hay que añadir una tool, hay una persona a la que preguntar. Sin owner, el MCP se queda obsoleto en silencio.

Bloque 6 · Cierre

Tres ideas para llevarte.

Un MCP propio es la materialización ejecutable de una spec de dominio. Si la spec está bien escrita, el diseño del MCP es traducción mecánica. Si la spec era débil, el MCP hereda las debilidades — empieza por la spec, no por el código.

Los cuatro errores típicos (tool genérica, descripción ambigua, validación ausente, mezcla lectura/escritura) son los mismos en cada primer MCP. Saberlos de antemano te ahorra repetirlos. Cada uno tiene patrón claro de mitigación: spec por operación, descripción explícita, validación al inicio, separación de operaciones.

Los MCPs propios viven igual que skills, hooks y specs: en el repo (o en su propio repo si son compartidos entre proyectos), con owner declarado, versionados, mantenidos. Si los tratas como código de primera clase, van a aguantar lo que dure el proyecto. Si los tratas como utilidad personal, se rompen silenciosamente.

Glosario rápido del bloque

- **MCP propio:** servidor MCP construido por tu equipo, no instalado de un catálogo público. Materializa una spec de dominio interno.
- **Tool genérica:** antipatrón de diseño donde una sola tool recibe parámetros muy abiertos en lugar de descomponerse en varias tools específicas.

- **Validación de contrato:** comprobación al inicio de la tool de que los parámetros recibidos cumplen el esquema. No es validación de negocio.
- **Fachada delgada:** patrón donde el MCP solo expone tools que llaman a servicios internos vía DI, sin lógica de negocio propia.
- **dotnet tool para distribución:** mecanismo .NET para instalar ejecutables como herramientas globales desde un feed de NuGet. Útil para distribuir MCPs propios al equipo.

Qué viene en M5.5

Práctica de cierre del M5. Tomas un sistema interno simple de tu trabajo (o uno simulado para la práctica) y diseñas un MCP propio paso a paso: escribir la mini-spec del dominio, traducir a tools tipadas en .NET, validación, separación lectura/escritura, configuración básica de cliente. Al terminar el M5 tendrás un servidor MCP propio funcionando en local, conectado a Claude Code, listo para iterar.

M5.5 · Práctica — conectar un MCP a la práctica del M3

Bloque 0 · Lo que vas a hacer

Tres submódulos teóricos (M5.1 qué es MCP, M5.2 anatomía, M5.3 catálogo), uno operativo (M5.4 diseñar uno propio). Aquí toca aplicar. La práctica tiene dos caminos posibles según tu interés:

Camino A — consumir un MCP existente. Instalas un MCP del catálogo (típicamente uno de base de datos) y lo enchufas al flujo de implementación de tu spec del M3.6. El agente puede ahora consultar el esquema real y validar contra él.

Camino B — construir un MCP propio. Tomas tu implementación del M3.6 y la expones a través de un servidor MCP propio. Otros agentes (o el mismo agente en otra sesión) pueden invocarla a través de MCP.

Los dos caminos enseñan cosas distintas. Si tu sistema real depende de servicios externos (DB, APIs), el camino A es más útil. Si tu equipo tiene un sistema interno propio que quieres exponer, el camino B. Si dudas, ve por el A — es más rápido y te da intuición sobre cómo se siente un MCP en uso real.

Recorrido común en cinco pasos. Los pasos 2 y 3 cambian según el camino elegido; los demás son comunes.

Bloque 1 · Paso 1 — elegir el camino

Decide en función de tu contexto:

Vas por A si: - Tienes una base de datos local (SQLite, PostgreSQL en Docker, SQL Server LocalDB) que la implementación del M3.6 podría consumir. - Quieres probar la experiencia de tener un MCP activo sin construir nada. - Tu pieza del M3.6 hace algo con datos persistidos.

Vas por B si: - Tu pieza del M3.6 es una operación pura (como `CalcularFechaVencimiento`) que tiene sentido exponer como tool reutilizable. - Quieres aprender el proceso completo de construir un MCP propio, no solo consumirlo. - Tienes interés en distribuir esa pieza a otros consumidores (otros agentes, otros equipos).

No hay camino correcto en absoluto. Elige uno para esta práctica; el otro lo puedes hacer después como tarea libre.

Bloque 2A · Camino A — consumir un MCP de DB

Si elegiste A:

Instala el MCP. Para SQLite (el más sencillo para empezar), instala el paquete oficial del catálogo Anthropic. La instrucción exacta depende del cliente — la documentación de tu versión de Claude Code te dice el comando.

Prepara la base de datos de práctica. Crea una SQLite local con un schema mínimo coherente con la pieza del M3.6. Por ejemplo, si tu pieza calcula vencimientos de facturas, una tabla `Invoices` con campos `Id`, `FechaEmision`, `Monto`, `ClienteId`, y una tabla `Cientes` con `Id`, `Tipo`, `PaisFacturacion`. Llénala con cinco filas de ejemplo.

Configura el cliente. En el `.mcp.json` (o equivalente) del repo:

```
{
  "mcpServers": {
    "sqlite-practica": {
      "command": "uvx",
      "args": ["mcp-server-sqlite", "--db-path", "./recursos/practica.db"]
    }
  }
}
```

(Si no usas el oficial vía `uvx`, ajusta al runner del MCP que hayas instalado.)

Arranca Claude Code en el repo. El cliente debería detectar el MCP nuevo. Comprueba que las tools del MCP aparecen disponibles.

Probar el flujo. Le pides al agente: *"Lee el esquema de la base de datos local y comprueba que el `DbContext` del proyecto refleja correctamente las tablas. Reporta discrepancias."*

El agente debería invocar el MCP para listar tablas, obtener columnas, compararlas con tu modelo C#. Sin el MCP, el agente solo podría leer tu modelo y asumir que la DB lo respeta. Con el MCP, valida contra la realidad.

Bloque 2B · Camino B — construir un MCP propio

Si elegiste B:

Crea el proyecto del MCP. Una librería .NET 10 dentro de tu solution:

```
dotnet new console -n SDDCC.Facturas.Mcp -o src/SDDCC.Facturas.Mcp
dotnet sln add src/SDDCC.Facturas.Mcp/SDDCC.Facturas.Mcp.csproj
```

Añade los paquetes:

```
cd src/SDDCC.Facturas.Mcp
dotnet add package ModelContextProtocol
dotnet add package Microsoft.Extensions.Hosting
dotnet add reference ../SDDCC.Domain/SDDCC.Domain.csproj
```

Configura el arranque en `Program.cs` (igual que el ejemplo del M5.2):

```
using ModelContextProtocol.Server;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.DependencyInjection;
using SDDCC.Domain; // donde vive tu implementación del M3.6

var builder = Host.CreateApplicationBuilder(args);

builder.Services
    .AddMcpServer()
    .WithStdioServerTransport()
    .WithToolsFromAssembly();

var host = builder.Build();
await host.RunAsync();
```

Expón tu pieza como tool. Crea `Tools/FacturaTools.cs` :

```

using System.ComponentModel;
using ModelContextProtocol.Server;
using SDDCC.Domain;

[McpServerToolType]
public static class FacturaTools
{
    [McpServerTool, Description("Calcula la fecha de vencimiento de una factura según el tipo de cliente y el país.")]
    public static FacturaVencimientoResult CalcularVencimiento(
        [Description("Fecha de emisión de la factura en formato ISO 8601 con offset.")]
        string fechaEmision,
        [Description("Monto total en céntimos (integer).")]
        long montoTotalCentimos,
        [Description("Tipo de cliente: 'regular' o 'vip'.")]
        string tipoCliente,
        [Description("País de facturación en ISO 3166-1 alpha-2 (ES, FR, ...).")]
        string paisFacturacion)
    {
        var tipo = tipoCliente.ToLowerInvariant() switch
        {
            "regular" => TipoCliente.Regular,
            "vip" => TipoCliente.Vip,
            _ => throw new ArgumentException($"Tipo de cliente desconocido: {tipoCliente}")
        };

        var factura = new Factura(
            FechaEmision: DateTimeOffset.Parse(fechaEmision),
            MontoTotalCentimos: montoTotalCentimos,
            Cliente: new Cliente(tipo, paisFacturacion));

        var vencimiento = FacturaService.CalcularFechaVencimiento(factura);

        return new FacturaVencimientoResult(
            FechaVencimiento: vencimiento.ToString("o"),
            EsLaboral: vencimiento.DayOfWeek != DayOfWeek.Saturday &&
            vencimiento.DayOfWeek != DayOfWeek.Sunday);
    }
}

public record FacturaVencimientoResult(string FechaVencimiento, bool EsLaboral);

```

Tres detalles:

Las descripciones son explícitas. Indican formato esperado (ISO 8601, alpha-2, integer en céntimos). Sin esas notas el modelo improvisa formato.

La validación pasa por el matching del `switch`. Si llega un tipo desconocido, error claro temprano.

La tool es una fachada delgada. El cálculo real vive en `FacturaService.CalcularFechaVencimiento` (la pieza del M3.6). El MCP no duplica lógica; la

expone.

Configura el cliente. En `.mcp.json`:

```
{
  "mcpServers": {
    "facturas": {
      "command": "dotnet",
      "args": ["run", "--project",
        "src/SDDCC.Facturas.Mcp/SDDCC.Facturas.Mcp.csproj"]
    }
  }
}
```

Bloque 3 · Paso 3 — probar el flujo

Independientemente del camino que hayas seguido, ahora toca verificar que el MCP está vivo y haciendo lo que debería.

Probar la lista de tools. Abre Claude Code en el repo y pídele: *"Lista las tools disponibles del MCP."* El agente debería responder con las tools que tu servidor expone. Si no aparecen, hay problema de configuración (revisa el `.mcp.json`, los logs del cliente, las dependencias del SDK).

Invocar una tool concreta. Pídele al agente: *"Usa la tool [nombre] del MCP para [acción concreta]."* El agente debería invocar la tool, recibir el resultado, y reportarlo. Si la tool falla, el mensaje de error debería ser legible. Si no lo es, ajusta el mensaje en tu servidor.

Comprobar el contrato. Pídele al agente: *"Cuéntame qué espera la tool [nombre] como inputs y qué devuelve."* El agente debería describir el esquema (parámetros, tipos, descripciones). Si lo describe mal, las descripciones de tu tool están débiles o el SDK no las está exponiendo bien.

Si las tres pruebas pasan, el MCP funciona. Si alguna falla, depura: el log de stderr del servidor (lo ve el cliente normalmente) suele ser la fuente más útil de diagnóstico.

Bloque 4 · Paso 4 — integrar con el flujo SDD

Hasta aquí tenías un MCP que responde. El siguiente paso es comprobar que aporta a un flujo SDD completo.

Si vas por A (consumir un MCP de DB):

Pídele al agente: *"Voy a añadir una migración que cambia el tipo de la columna `MontoTotal` de `int` a `long`. Antes de empezar, consulta el esquema actual con el MCP, identifica todos los sitios afectados, y propón un plan."*

El agente debería invocar el MCP, leer el esquema, ver qué tablas y vistas usan esa columna, y proponer un plan informado. Sin el MCP, el plan estaría basado solo en lo que el agente ve en tu código. Con el MCP, está basado también en la base real.

Si vas por B (MCP propio que expone tu pieza):

Abre otra sesión de Claude Code en un proyecto distinto (puede ser un repo limpio de prueba). Pídele: *"Necesito calcular la fecha de vencimiento de una factura con esta entrada [datos concretos]. ¿Tienes alguna tool MCP disponible para esto?"*

Si el cliente ha cargado tu MCP (configurado en el `.mcp.json` global o del repo), el agente debería detectarlo, invocarlo, y devolverte la fecha. Has expuesto una pieza de tu dominio a otros consumidores vía MCP, sin que tengan que duplicar el código.

Bloque 5 · Paso 5 — observar las decisiones del dev

Mira atrás los pasos. En cada uno hubo decisiones tuyas:

1. **Qué camino seguiste y por qué** (A o B). Esa decisión refleja qué problema tienes hoy.
2. **Cómo elegiste el MCP** (en A) o **cómo decidiste qué tools exponer** (en B). En ambos casos, la spec del dominio guió la elección.
3. **Qué validación incluiste** en las descripciones o en el código. La calidad de la validación marca la diferencia entre tool útil y tool frágil.
4. **Cómo distinguiste lectura de escritura** (si aplica en tu MCP). En esta práctica probablemente todo era lectura; en un MCP real con escritura, esa distinción es lo que evita problemas.
5. **Qué configuración de seguridad dejaste al final** (permisos, rutas, modo read-only). El MCP que arranca tu equipo es el que tú dejas configurado.

Las cinco decisiones son las que convierten un MCP "que funciona en la demo" en uno "que sigue funcionando dentro de seis meses". Tomarlas con criterio aquí te ahorra problemas más adelante.

Bloque 6 · Cierre del M5

Has llegado al final del M5 con un MCP operativo conectado a tu flujo SDD. Tres ideas para llevarte del módulo entero.

MCP es plumbing tipada, no inteligencia. Lo que aporta es que el agente puede hablar con sistemas externos sin tener que adivinar formatos. Donde no hay sistema externo, no hay valor.

Para SDD en .NET, los tres MCPs que más rinden al inicio son **filesystem**, **GitHub** y **uno de base de datos**. Empieza por el que resuelva el dolor más concreto que tienes hoy; añade el siguiente solo cuando aparezca el siguiente dolor.

Cuando los MCPs públicos no cubren lo que necesitas, construir uno propio en .NET es trabajo de un día. La estructura es sencilla, el SDK encapsula el protocolo, y lo que de verdad importa es la spec del dominio que vas a exponer. Si la spec está bien, el código del MCP sale solo.

Glosario rápido del bloque

- **Camino A:** variante de esta práctica donde se consume un MCP existente (típicamente DB) para enriquecer una pieza ya construida.
- **Camino B:** variante donde se construye un MCP propio que expone una pieza propia a otros consumidores.
- **.mcp.json** : archivo de configuración donde el cliente Claude Code declara qué servidores MCP cargar al iniciar.
- **Fachada delgada (en práctica):** cuando la tool del MCP solo traduce parámetros y delega en código existente. Sin lógica de negocio duplicada.

Estado al cerrar el M5

Has visto: - Qué es MCP y qué no es (M5.1). - Anatomía de un servidor MCP por dentro (M5.2). - MCPs útiles para SDD en .NET (M5.3). - Cómo diseñar un MCP propio desde una spec (M5.4). - Práctica de conectar un MCP al flujo del M3 (M5.5).

Tienes el puente al mundo real cubierto. En el M6 entra el último módulo: caso end-to-end donde se aplican todas las piezas del curso sobre un sistema completo (API + persistencia + cliente), desde la primera spec hasta el cambio de requisito que llega a las dos semanas.

Qué viene en M6

Caso end-to-end. M6.1 presenta el proyecto que vamos a construir y por qué ese caso encaja para enseñar SDD. M6.2 es la sesión de especificación. M6.3 implementa guiado por SDD con todas las piezas vistas en M3 y M4. M6.4 es el clásico cambio de requisito y cómo SDD lo absorbe. M6.5 cierra el curso con tu próximo proyecto y material de seguimiento.

PARTE 6

Caso end-to-end

M6.1 · El proyecto que vamos a construir

Bloque 0 · Por qué este submódulo es corto y los demás no

Los cuatro submódulos siguientes del M6 son las sesiones reales de construcción: especificación, implementación, cambio de requisito, cierre. Este es solo presentación. Por eso es el más corto del módulo.

Pero la presentación importa. Sin un caso bien acotado, las sesiones siguientes derivan: nos perdemos en detalles del dominio en lugar de aplicar SDD. El objetivo de estos diez minutos es dejar claro qué construimos, por qué este caso encaja para enseñar el método, qué entra y qué no, y cómo van a estar repartidas las cuatro sesiones que vienen.

Bloque 1 · El caso — mini-API de facturación

El sistema que vamos a construir es una **mini-API de facturación** para una empresa pequeña. Reutiliza la función `CalcularFechaVencimiento` que viste en el M3.6 (y que también dimos como ejemplo en M5.5), y la rodea de los componentes necesarios para que sea un servicio funcional.

Las cuatro operaciones que la API expone, en lenguaje de negocio:

Registrar una factura nueva. Un usuario interno envía los datos básicos de una factura (cliente, líneas, fechas) y el sistema la registra, calculando automáticamente la fecha de vencimiento según las reglas de M1.4.

Consultar una factura por su identificador. Devuelve los datos completos de la factura, incluyendo su estado (pendiente, pagada, vencida).

Listar facturas filtradas. Filtrado por cliente, por rango de fechas, por estado.

Marcar factura como pagada. Cambia el estado y registra la fecha de cobro.

Cuatro endpoints. Suficiente para mostrar SDD aplicado a varias capas (API, dominio, persistencia, cliente) sin que el caso se vuelva inmanejable en cuatro sesiones de vídeo.

Bloque 2 · Componentes técnicos del caso

La solution va a tener cinco proyectos. El número parece alto para un caso pequeño; está pensado para que cada proyecto tenga responsabilidad clara y la spec de cada uno sea independiente.

Facturacion.Domain. Las entidades (`Factura`, `Cliente`, `LineaFactura`), los enums (`EstadoFactura`, `TipoCliente`), y las funciones puras del dominio (`CalcularFechaVencimiento`, `CalcularTotal`). Sin dependencias externas. Sin EF Core. Es código que se podría usar en cualquier otra aplicación.

Facturacion.Infrastructure. La capa de persistencia. EF Core con `DbContext`, mappings, migraciones. Implementa los repositorios que el dominio define como interfaces. Es el sitio que toca la base de datos.

Facturacion.Api. ASP.NET Core. Los cuatro endpoints, la inyección de dependencias, la configuración (`appsettings.json`), el middleware estándar. Es la cara visible del sistema.

Facturacion.Cli. Un cliente de consola que consume la API. No es estrictamente necesario para el sistema, pero sirve para mostrar cómo SDD aplica también a una pieza consumidora.

Facturacion.Domain.Tests. Tests xUnit v3 del dominio. Validan que las funciones puras y las reglas de negocio se comportan según los criterios de aceptación de las specs.

Cinco proyectos. Cada uno con su `.csproj`, su carpeta, sus dependencias claras. La solution entera cabe en dos mil o tres mil líneas de código real.

Bloque 3 · Por qué este caso encaja para enseñar SDD

Cuatro razones para elegir este caso y no otro.

Tiene lógica de negocio real, no decorativa. El cálculo de vencimiento, los descuentos por pronto pago, las reglas de estado de factura tienen decisiones implícitas que la spec va a hacer explícitas. No es un caso de "guarda un nombre y devuélvelo".

Toca varias capas con la misma spec. Una sola feature (registrar factura) afecta al dominio, al repositorio, al endpoint y al cliente. Eso muestra SDD aplicado en cascada: una spec arriba, varias piezas coordinadas abajo.

Permite cambio de requisito plausible. El M6.4 va a meter un cambio: "el cliente quiere aplicar descuentos por pronto pago". Es el tipo de cambio que llega de verdad en proyectos reales y que se siente como real al alumno. Veremos cómo SDD hace ese cambio relativamente barato.

Stack coherente con el curso. .NET 10 + ASP.NET Core + EF Core + xUnit v3. Mismo molde que usaste en M3.6, M4.5 y M5.5. Cero curva de aprendizaje técnica.

Bloque 4 · Alcance explícito — qué SÍ y qué NO

Como buena spec, el caso end-to-end también tiene sección de fuera de alcance. Esto evita que las cuatro sesiones siguientes se vayan por las ramas.

Qué SÍ vamos a hacer:

- Especificar las cuatro operaciones con plantilla A del M2.3.
- Implementar el dominio en `Facturacion.Domain` con tests xUnit.
- Implementar persistencia en `Facturacion.Infrastructure` con EF Core (SQLite para desarrollo).
- Implementar la API en `Facturacion.Api` con cuatro endpoints REST.
- Implementar un cliente CLI mínimo en `Facturacion.Cli`.
- Aplicar Plan Mode, skills, hooks y verificación durante todo el proceso.
- Hacer el cambio de requisito (descuentos por pronto pago) y mostrar cómo SDD lo absorbe.

Qué NO vamos a hacer:

- Autenticación y autorización (asumimos que la API es interna detrás de otra capa de seguridad).
- Internacionalización de la moneda (todo en céntimos, sin conversión).
- UI web (el cliente es CLI).
- Despliegue a un proveedor cloud (corre en local).
- Observabilidad avanzada (logs básicos, sin trazas distribuidas, sin métricas).
- Tests de integración exhaustivos (solo unitarios del dominio).
- Caché, paginación avanzada, búsqueda full-text.

La lista de "qué NO" es más larga que la de "qué SÍ" a propósito. Cada exclusión es una decisión consciente para mantener el caso manejable en cuatro sesiones. En un proyecto real, varias de esas exclusiones serían necesarias — el alumno debería poder añadirlas después aplicando el mismo método.

Bloque 5 · Cómo se reparten las cuatro sesiones restantes

Para que tengas el mapa antes de meterte:

M6.2 — Sesión 1, especificar. Escribir las specs de las cuatro operaciones de la API. Revisarlas con Claude Code aplicando lo de M2.4. Decidir el orden de implementación. Al terminar, tendremos cuatro specs en `specs/` y un plan claro de qué construir primero.

M6.3 — Sesión 2, implementar guiado. Llevar la spec del dominio (`Facturacion.Domain`) a código siguiendo el método: Plan Mode, skills, hooks activos. Mostrar las decisiones concretas

que el dev toma en cada paso. Al terminar, tendremos dominio + tests pasando.

M6.4 — Sesión 3, el cambio. El cliente quiere descuentos por pronto pago: si la factura se paga antes del día 10 desde emisión, descuento del 2%. Cómo se actualiza la spec, qué cambia en el código, qué tests se añaden, cómo el coste real del cambio es bajo cuando hay spec.

M6.5 — Cierre. Tu próximo proyecto. Cómo aplicar SDD en lo que ya tienes en tu trabajo real. Errores frecuentes en los primeros meses. Material de seguimiento (Spec Kit oficial, comunidad, documentación de referencia). Despedida del curso.

Total estimado del M6: unas ocho a diez horas de vídeo (más práctica del alumno entre sesiones). Es la mitad del curso entero. Por una buena razón: ver SDD aplicado de principio a fin sobre un caso completo es lo que hace que el método deje de ser teoría.

Bloque 6 · Cierre

Tres ideas para llevarte de este submódulo de presentación.

El caso es deliberadamente pequeño. Cuatro endpoints, cinco proyectos, dominio con reglas claras pero no exuberante. Eso permite que en cuatro sesiones se vea SDD aplicado de principio a fin, incluyendo el cambio de requisito clásico.

Reutilizamos `CalcularFechaVencimiento` del M3.6 a propósito. Conecta el caso end-to-end con lo que ya construiste; reduce contexto que tendríamos que cargar de nuevo. Si no hiciste la práctica del M3.6, el código de esa función se entrega en `recursos/` del curso para que puedas continuar.

El "qué NO vamos a hacer" es más largo que el "qué SÍ" deliberadamente. En un proyecto real, las exclusiones serían discusión con el equipo. Aquí son decisiones del curso. Lo importante es ver el patrón: declarar el alcance explícito ahorra mucho ruido durante la construcción.

Glosario rápido del bloque

- **Caso end-to-end:** ejercicio que aplica todas las piezas del curso a un sistema completo, no a una función aislada.
- `Facturacion.Domain` / `Facturacion.Infrastructure` / `Facturacion.Api` / `Facturacion.Cli` : los cuatro proyectos productivos de la solution. Mas el de tests.
- **Fuera de alcance del caso:** lo que el caso decide no abordar para mantenerse manejable. Diferente del fuera de alcance de una feature individual, aunque sigue el mismo patrón.

Qué viene en M6.2

La sesión 1 del caso: especificación. Vamos a escribir las cuatro specs (registrar, consultar, listar, marcar pagada) aplicando la plantilla A del M2.3. Las revisamos con Claude Code buscando los cuatro huecos típicos del M2.4. Decidimos el orden de implementación. Al terminar deberíamos tener cuatro `.md` en `specs/` listos para entrar en Plan Mode.

M6.2 · Sesión 1 — especificar las cuatro operaciones

Bloque 0 · Lo que vamos a hacer

Veinticinco minutos de spec, cero líneas de código. La tentación de "ya hago una spec corta, mejor empiezo a teclear" va a aparecer; resistirla es la disciplina central del curso.

Lo que cubrimos:

1. Preparar la sesión y la solution mínima.
2. Escribir la spec de la operación más rica (`registrar-factura.md`).
3. Esquematisar las otras tres specs con el mismo patrón.
4. Pasar las cuatro por revisión con Claude Code.
5. Cerrar los huecos detectados.
6. Decidir el orden de implementación.

Al terminar tendrás cuatro `.md` en `specs/` revisados y un plan claro de qué construir primero.

Bloque 1 · Preparar sesión y solution

Antes de abrir Claude Code, deja la solution lista.

```
mkdir SDDCC-Facturacion
cd SDDCC-Facturacion
git init
dotnet new sln -n SDDCC.Facturacion
mkdir src tests specs
dotnet new classlib -n SDDCC.Facturacion.Domain -o src/SDDCC.Facturacion.Domain
dotnet sln add src/SDDCC.Facturacion.Domain/SDDCC.Facturacion.Domain.csproj
```

Crea `CLAUDE.md` raíz con cuatro líneas:

SDDCC.Facturacion

Mini-API de facturación. .NET 10, ASP.NET Core, EF Core (SQLite en local), xUnit v3.

Reglas:

- Plan Mode obligatorio antes de implementar.
- Las specs en `'specs/'` son la fuente única de verdad.
- Todo importe en céntimos (long), nunca decimal con coma.
- Tests unitarios del dominio antes de tocar Infrastructure o Api.

Y crea `.claude/hooks/` con los tres hooks defensivos del M4.5 (destructivas, secretos, scope). En esta primera sesión no van a dispararse mucho — todavía no tocamos código — pero la red queda puesta para las sesiones siguientes.

Commit inicial:

```
git add .  
git commit -m "chore: estructura inicial de SDDCC.Facturacion"
```

Sesión en orden. Abrimos Claude Code.

Bloque 2 · Spec 1, registrar factura nueva (la rica)

Empezamos por la operación más compleja porque su spec sienta las reglas que las otras tres van a reutilizar (tipos compartidos, convenciones de error, criterios de aceptación que se generalizan).

Crea `specs/registrar-factura.md` y escribe en quince minutos:

Spec: registrar factura nueva

Intent

El equipo de contabilidad necesita registrar facturas emitidas a clientes, con cálculo automático de la fecha de vencimiento según las reglas comerciales (30 días naturales para regular, 60 para VIP, con ajustes por fin de semana y festivos del país del cliente). Hoy el proceso se hace en Excel y cada error en el cálculo provoca reclamaciones del cliente. Centralizar el registro en una API evita errores y permite que otros sistemas (CRM, informes) consuman las facturas desde un único sitio.

Behavior

Caso normal

- Endpoint: `POST /facturas`.
- Entrada (JSON): { clienteId, fechaEmision, lineas: [{ descripcion, cantidad, precioCentimos }] }.
- Validaciones:
 - clienteId debe existir en la tabla de clientes.
 - fechaEmision no puede estar en el futuro.
 - lineas debe tener al menos un elemento.
 - cantidad > 0, precioCentimos ≥ 0.
- Cálculo del total: suma de (cantidad * precioCentimos) por línea.
- Cálculo de la fecha de vencimiento: usa `FacturaService.CalcularFechaVencimiento` (la función del M3.6) con el tipo de cliente y país de facturación.
- Estado inicial: "Pendiente".
- Respuesta (201 Created): { id, numero, total, fechaVencimiento, estado }.

Casos de error

- 400 si validación falla, con mensaje claro indicando el campo y la regla.
- 404 si clienteId no existe.
- 409 si ya existe una factura con el mismo número (generación automática debe ser idempotente bajo retry).

Contract

```
```csharp
public record RegistrarFacturaRequest(
 long ClienteId,
 DateTimeOffset FechaEmision,
 IReadOnlyList<LineaFacturaRequest> Lineas);

public record LineaFacturaRequest(
 string Descripcion,
 int Cantidad,
 long PrecioCentimos);

public record FacturaRegistradaResponse(
 long Id,
 string Numero,
 long TotalCentimos,
 DateTimeOffset FechaVencimiento,
 EstadoFactura Estado);

public enum EstadoFactura { Pendiente, Pagada, Vencida, Anulada }
```

## Criterios de aceptación

- [ ] Factura regular emitida el lunes 1 paga el miércoles 31 días después (30 naturales + ajuste si cae en fin de semana).
- [ ] Factura VIP tiene 60 días de plazo en lugar de 30.
- [ ] Total se calcula como suma de líneas en céntimos, sin redondeos.
- [ ] Cliente inexistente devuelve 404 con mensaje "cliente no encontrado".
- [ ] fechaEmision en futuro devuelve 400 con mensaje específico.
- [ ] Líneas vacías devuelve 400 con mensaje específico.
- [ ] Número de factura se genera automáticamente con formato F-YYYY-NNNNNN.
- [ ] Tests xUnit cubren los 6 criterios anteriores.

## Fuera de alcance

- No envía notificación al cliente (otro flujo del que no nos ocupamos).
- No persiste líneas como entidades separadas si el dominio decide otra cosa (decisión interna del dominio; la API solo expone el endpoint).
- No valida que el monto total esté dentro de límites de crédito del cliente (no existe ese concepto en este caso).

## Notas y decisiones

- Importes en céntimos como `long` (decisión estándar del proyecto, en `CLAUDE.md`).
- Formato del número de factura discutido con contabilidad: F-YYYY-NNNNNN asegura ordenación natural y unicidad por año.
- Idempotencia bajo retry no implementada en v1 — la cláusula del 409 se cumple con UNIQUE constraint en BD; la idempotencia más completa se evalúa más adelante.



Esa es la spec en su versión inicial. No es perfecta; va a tener huecos que la revisión del bloque 6 va a sacar.

Detalle de proceso: cada vez que dudas si meter algo en `Behavior` o en `Contract`, prueba esta regla: si lo entiende un PM, va en `Behavior`. Si solo lo entiende un dev mirando código, va en `Contract`. Funciona casi siempre.

---

## Bloque 3 · Spec 2, consultar factura por ID

La segunda spec es esqueleto sobre el patrón de la primera. Crea `specs/consultar-factura.md`:

```
```markdown
```

```
# Spec: consultar factura por ID
```

```
## Intent
```

```
Permitir que otros sistemas (CRM, informes, integraciones) y el equipo interno consulten una factura por su identificador y reciban los datos completos en una sola llamada.
```

```
## Behavior
```

- Endpoint: `GET /facturas/{id}`.
- Respuesta 200 con la representación completa de la factura (datos básicos + estado + líneas + fechas).
- 404 si la factura no existe.
- Solo lectura, sin efectos secundarios.

```
## Contract
```

```
Reutiliza tipos del dominio. Respuesta incluye lista de líneas con sus descripciones, cantidades y precios.
```

```
## Criterios de aceptación
```

- [] Factura existente devuelve 200 con todos los campos.
- [] Factura inexistente devuelve 404 con mensaje "factura no encontrada".
- [] La respuesta incluye estado actualizado (Vencida si la fecha de pago pasó sin marcarse como pagada).
- [] Tests xUnit cubren los tres casos.

```
## Fuera de alcance
```

- No expone histórico de cambios de estado (otro endpoint si se necesita).
- No incluye datos del cliente más allá del ID (otro endpoint del CRM si se necesita).

Para no agotar el submódulo, las otras dos specs siguen el mismo patrón. Pongo solo el intent y un par de criterios para que veas la dirección — la versión completa se queda como ejercicio del alumno (o se entrega en `recursos/specs-completas/`).

Bloque 4 · Specs 3 y 4 esquemáticas

Spec 3 — listar facturas filtradas (`specs/listar-facturas.md`)

Intent: dashboard del equipo de contabilidad y consultas del CRM requieren filtrar facturas por cliente, rango de fechas o estado.

Endpoint: `GET /facturas?clienteId=X&desde=Y&hasta=Z&estado=W` (todos opcionales).

Criterios clave: paginación de 50 por defecto (parametrizable hasta 200); orden por fechaEmision descendente; respuesta incluye total de elementos coincidentes para que el cliente pagine.

Spec 4 — marcar factura como pagada (`specs/marcar-pagada.md`)

Intent: cuando llega el pago de una factura, el operador registra la fecha de cobro para que el estado pase a "Pagada" y los informes financieros reflejen la realidad.

Endpoint: `POST /facturas/{id}/pagos` con body { fechaCobro }.

Criterios clave: cambio de estado idempotente (marcar dos veces la misma factura como pagada no debería romper nada); rechazo con 409 si la factura está en estado distinto de "Pendiente"; respuesta 200 con la factura actualizada.

Cuatro `.md` en `specs/`. Ya es algo.

Bloque 5 · Revisión con Claude Code

Activa el skill `revisar-spec-sdd` del M3.3 (o usa la prompt directa del M2.4). Pásale cada uno de los cuatro `.md` por turno y deja que detecte huecos en las cuatro categorías.

En esta práctica, en mi última pasada el agente reportó cosas como estas (tu ejecución dará variantes):

En `registrar-factura.md`: - Ambigüedad: "líneas debe tener al menos un elemento" — ¿qué pasa si una línea tiene cantidad o después de validar `> 0`? Aclarar que `cantidad > 0` es estricto, no `≥ 0`. - Alcance abierto: no se dice qué pasa si dos líneas tienen la misma descripción. ¿Se fusionan? ¿Se aceptan tal cual? - Criterio incompleto: el criterio del formato F-YYYY-NNNNNN no especifica si NNNNNN se reinicia cada año.

En `consultar-factura.md`: - Ambigüedad: ¿qué timezone se devuelve en la respuesta? UTC o local del servidor. - Contradicción potencial con `registrar-factura`: el contract de la respuesta no estaba definido aquí.

En `listar-facturas.md`: - Alcance abierto: ¿el filtro de estado permite varios estados a la vez (`estado=Pendiente,Vencida`)? La spec no lo dice. - Criterio ausente: rendimiento de la consulta

con filtros complejos.

En `marcar-pagada.md` : - Ambigüedad: "idempotente" no especifica si devuelve 200 o 304 al marcar pagada una factura ya pagada.

Para cada hueco detectado, decides qué hacer:

- **Cierras la ambigüedad** modificando la spec con la decisión concreta.
- **Marcas con `TODO: discutir`** los huecos que requieren input externo (contabilidad, otro equipo).
- **Aceptas el hueco como matiz vivo** si la decisión depende de cómo se sienta la implementación.

Tras cierre, segunda pasada rápida del agente. En esta segunda pasada solo deberían aparecer matices, no problemas estructurales. Si todavía hay problemas grandes, la spec necesita otra ronda; si solo hay matices, está ejecutable.

Bloque 6 · Decidir el orden de implementación

Las cuatro specs no son independientes; hay dependencias técnicas que importan para decidir orden.

Análisis de dependencias:

- `registrar-factura` usa `FacturaService.CalcularFechaVencimiento` . La función del M3.6 ya existe o se entrega como recurso.
- `registrar-factura` necesita el `DbContext` y los repositorios de `Facturacion.Infrastructure` .
- `consultar-factura` y `listar-facturas` consultan lo que `registrar-factura` ha escrito; sin esa primera, no tienen nada que consultar.
- `marcar-pagada` cambia el estado de algo registrado; depende de que `registrar-factura` y `consultar-factura` estén en su sitio.

Orden propuesto:

1. **Dominio primero** (`Facturacion.Domain`): entidades, enums, función `CalcularFechaVencimiento` (reuso), función `CalcularTotal` , validaciones de invariantes. Tests xUnit completos. Sin tocar EF Core ni API todavía.
2. **Infrastructure**: `DbContext` , mapeos, migración inicial, repositorios.
3. **API**: endpoint `POST /facturas` (registrar), seguido de `GET /facturas/{id}` (consultar).

4. **API resto:** GET /facturas (listar con filtros) y POST /facturas/{id}/pagos (marcar pagada).

5. **Cli:** cliente de consola que consume los cuatro endpoints.

Este orden tiene una propiedad importante: cada paso deja el sistema funcional. Tras el paso 1 tienes dominio testeado aunque no haya API; tras el 2 tienes persistencia funcionando aunque no haya endpoint; etc. SDD bien aplicado evita los "deja todo a medias hasta que esté completo".

Decides este orden, lo anotas en `_plan-construccion.md` dentro de `specs/`, y se queda como guía para las sesiones siguientes.

Commit:

```
git add specs/ CLAUDE.md .claude/  
git commit -m "feat(specs): registrar, consultar, listar, marcar-pagada + plan"
```

Bloque 7 · Cierre

Tres ideas para llevarte de esta sesión 1.

La spec más rica se escribe primero a propósito. Sienta tipos, convenciones de error y criterios que las otras tres reutilizan. Ahorras tiempo y eliminas inconsistencias antes de que aparezcan.

La revisión automática es el filtro más barato para los huecos. Cinco minutos pasando las cuatro specs por el skill `revisar-spec-sdd` te ahorra horas de re-trabajo en la implementación. No saltes ese paso.

El orden de implementación se decide al final de la sesión 1, no al principio. Necesitas las cuatro specs sobre la mesa para ver dependencias entre ellas y elegir el camino que deja el sistema funcional en cada paso.

Glosario rápido del bloque

- **Spec rica del módulo:** la primera spec que escribes en un módulo nuevo. Sienta convenciones (tipos, errores, validaciones) que las siguientes heredan.
- `_plan-construccion.md` : archivo donde anotas el orden de implementación decidido al final de la sesión 1. Vive en `specs/` con las propias specs.
- **Paso funcional:** cada paso del orden de implementación debe dejar el sistema en estado funcional, no a medias. Eso permite parar la sesión en cualquier punto sin trabajo a medio terminar.

Qué viene en M6.3

Sesión 2: implementar el dominio (`Facturacion.Domain`) siguiendo SDD. Plan Mode con la spec de registrar; skills si aparecen oportunidades; hooks de validación activos; verificación contra la spec en cada paso. Al terminar el M6.3 tenemos dominio + tests xUnit pasando, listo para pasar a Infrastructure en una sesión futura.

M6.3 · Sesión 2 — implementar guiado por SDD

Bloque 0 · Lo que vamos a hacer

En el M6.2 dejamos cuatro specs en `specs/` y un orden de construcción decidido. Ahora toca llevar la primera pieza del orden (el dominio en `Facturacion.Domain`) a código real, aplicando el método completo del curso.

Lo importante de esta sesión no es el código que sale. Es **cómo sale**. Vas a ver:

- El prompt con el que arranca la sesión.
- El plan que Claude Code devuelve y las iteraciones que se le hacen.
- La supervisión durante la ejecución.
- Cuándo se invoca un skill y cuándo no.
- Cómo los hooks de validación activos intervienen.
- La firma del trabajo al final.

Si has aplicado el método del M3.6 en una práctica anterior, esta sesión va a sentirse familiar pero ampliada — el caso es más rico, las decisiones más reales.

Bloque 1 · Paso 1, abrir sesión con la spec del Domain

Antes de abrir Claude Code, revisa que la sesión va a arrancar bien:

- `CLAUDE.md` raíz al día (lo dejaste en M6.2).
- Spec `specs/registrar-factura.md` y `specs/_plan-construccion.md` commiteadas.
- Hooks defensivos del M4.5 activos en `.claude/hooks/`.
- Plan Mode configurado como modo por defecto.

Abre Claude Code en el repo. El primer prompt:

Vamos a implementar el dominio del proyecto, primer paso del `_plan-construccion.md`. La spec principal está en `specs/registrar-factura.md`.

Pre-requisitos:

- Sigue las convenciones del `CLAUDE.md` raíz (.NET 10, xUnit v3, céntimos como long).
- Implementa SOLO en `src/Facturacion.Domain` (no tocar Infrastructure ni Api).
- Reutiliza la función `CalcularFechaVencimiento` existente (está en `recursos/` si no la tienes ya, viene del M3.6).
- Genera tests xUnit cubriendo los 6 criterios de aceptación de la spec.
- Antes de ejecutar, muéstrame el plan.

Cuatro detalles del prompt que merecen verse:

Apunto a archivos por path, no copio el contenido. La sesión va a leer las specs. Si copiara, gastaría contexto inútilmente.

Recuerdo las pre-condiciones aunque ya estén en `CLAUDE.md`. Repetirlas en el prompt fija el contexto operativo.

Acoto el alcance explícitamente: "solo en `src/Facturacion.Domain`". Eso ayuda al hook de scope del plan (M4.2) cuando se active.

Cierro pidiendo plan. Aunque Plan Mode esté activo por defecto, repetirlo es seguro adicional.

Bloque 2 · Paso 2, leer e iterar el plan

El agente tarda entre uno y dos minutos en devolver el plan. Cuando aparezca, no leas en diagonal.

Plan típico (resumido):

PLAN

Resumen

Voy a crear el modelo de dominio del proyecto: entidades, value objects, enums, y la lógica de cálculo de total y vencimiento. Tests xUnit cubriendo cada criterio de aceptación.

Archivos que voy a tocar

- src/Facturacion.Domain/Entities/Factura.cs (nuevo)
- src/Facturacion.Domain/Entities/LineaFactura.cs (nuevo)
- src/Facturacion.Domain/Entities/Cliente.cs (nuevo)
- src/Facturacion.Domain/Enums/EstadoFactura.cs (nuevo)
- src/Facturacion.Domain/Enums/TipoCliente.cs (nuevo)
- src/Facturacion.Domain/Services/FacturaService.cs (extender con CalcularTotal y RegistrarFactura)
- src/Facturacion.Domain/Exceptions/DominioException.cs (nuevo)
- src/Facturacion.Domain/Exceptions/ValidacionException.cs (nuevo)
- src/Facturacion.Domain.Tests/Services/FacturaServiceTests.cs (nuevo)

Pasos

1. Crear los enums.
2. Crear los value objects y entidades.
3. Crear la jerarquía de excepciones de dominio.
4. Extender FacturaService con CalcularTotal.
5. Añadir el método RegistrarFactura como factory de Factura con validaciones.
6. Crear el proyecto de tests xUnit (si no existe).
7. Implementar 12 tests cubriendo los 6 criterios + casos límite.
8. Ejecutar dotnet test y reportar.

Decisiones que he tomado

- Factura es una entidad mutable solo para Estado y FechaCobro; el resto es inmutable (record con propiedades init-only).
- Las líneas son value objects (records sin identidad).
- Las excepciones de dominio NO heredan de System.Exception directamente; van por DominioException. La API las traduce a HTTP en su capa.
- El número de factura lo genera RegistrarFactura siguiendo F-YYYY-NNNNNN. Para el contador NNNNNN, en dominio uso un mock (ICounter); la persistencia real la pone Infrastructure en la sesión siguiente.

Riesgos

- El formato del número y la generación atómica (conurrencia) son tema pendiente para Infrastructure. En el dominio asumimos que el counter es thread-safe; la responsabilidad real está fuera.
- No tengo claro si el tipo "regular" debe ser case-sensitive en la API. La spec no lo dice. Decido lowercase para evitar confusión; si la API necesita otra cosa, lo decide en su sesión.

Aplica las cuatro comprobaciones del M3.2.

Alcance. Solo `src/Facturacion.Domain` y `src/Facturacion.Domain.Tests`. Encaja con la restricción del prompt. Bien.

Decisiones tomadas. Tres decisiones importantes que tomó el agente: jerarquía de excepciones propia (en lugar de heredar de `Exception`), número de factura generado por

dominio con counter inyectado, y lowercase para el tipo cliente. Léelas. Las dos primeras te parecen razonables. La tercera es deuda hacia la API — la marcas mentalmente.

Secuencia. Los pasos van en orden lógico: tipos → excepciones → factory → tests → ejecución. No hay nada al revés.

Riesgos. El agente señala dos. El primero es legítimo (conurrencia es de Infrastructure). El segundo es lo que también detectaste. Conoce sus límites.

Una iteración pequeña que tiene sentido: pedir que las propiedades públicas de `Factura` sean explícitamente init-only en lugar de mutable, salvo `Estado` y `FechaCobro` que sí necesitan setter. El agente ya lo dijo en "decisiones tomadas" pero a veces los planes son ambiciosos y luego el código se relaja. Reforzarlo en el prompt asegura disciplina.

```
El plan está bien. Un refuerzo: cuando crees Factura como record,
quiero propiedades init-only excepto Estado y FechaCobro que sí son
mutables porque cambian con marcar-pagada. Si esto cambia algo en
el plan, dímelo antes de ejecutar.
```

El agente confirma o ajusta. Cuando estés conforme, firma:

```
Adelante.
```

Esa palabra es la decisión más importante de la sesión.

Bloque 3 · Paso 3, ejecutar y supervisar

Plan firmado, el agente arranca el bucle. Crea archivos, los rellena, los compila, ejecuta tests. Tu rol cambia: pasas de decidir qué se hace a supervisar cómo se hace.

Durante la ejecución, los hooks activos van a intervenir varias veces sin que tú hagas nada:

Hook de scope. Cada vez que el agente escribe un archivo, el hook comprueba que está dentro del scope del plan. Como el plan firmado limita a `Facturacion.Domain` y `Facturacion.Domain.Tests`, intentar tocar `Facturacion.Api` provocaría bloqueo. No debería pasar; si pasa, el hook lo pillaría.

Hook de tests post-edición (si lo instalaste). Después de cada edición de un `.cs`, ejecuta los tests del proyecto correspondiente. Mientras el dominio se construye paso a paso, algunos tests no van a estar todavía y fallan; el agente lo ve y sigue. Cuando llegemos a "ejecuta toda la suite", los tests ya están todos verdes o casi.

Hook de bloqueo destructivo. Si el agente decide ejecutar `rm -rf bin` para limpiar antes de un rebuild, el hook puede bloquear (según los patrones que ajustaste en M4.5). Decides en el momento si es legítimo y desactivas temporalmente, o resuelves de otra forma.

Cosas concretas a vigilar mientras el agente trabaja:

Los nombres de los tests. Pasarán por `dotnet test` al final. Si son confusos, un futuro lector va a sufrir. Buen formato: `[Método]_[Escenario]_[Resultado]`. Por ejemplo: `RegistrarFactura_ClienteRegular_FechaVencimientoCorrecta`.

Los mensajes de las excepciones. Cuando una validación falla, el mensaje va a aparecer en la respuesta de la API en la sesión siguiente. Mensajes concretos ("La cantidad de la línea 2 debe ser mayor que cero") rinden más que genéricos ("Validación fallida").

Los tests cubren los 6 criterios de la spec. Cuando el agente termine, comprueba que cada criterio tiene al menos un test asociado. El skill `generar-tests-desde-behavior` ayuda si ves huecos.

Bloque 4 · Paso 4, los tests pasan (o no)

Tras la ejecución, el agente reporta el resultado. Tres escenarios posibles:

Escenario 1: todos los tests pasan. El más común si la spec era buena. Inspeccionas los tests por encima, miras que cubren los 6 criterios, lees una o dos implementaciones para confirmar que el agente no se ha desviado. Te confirma "12 tests, 12 verdes". Sigues al paso 5.

Escenario 2: uno o dos tests fallan. El agente probablemente intentará arreglarlos por su cuenta. Déjalo dos o tres iteraciones. Si se atasca, lee el test que falla y el código que prueba. A veces el test era ambicioso y la spec no exigía tanto — el test sobra. A veces la spec era ambigua y resolver el test obliga a aclarar la spec. Decides cuál es el caso.

Escenario 3: muchos tests fallan o ni siquiera compila. El plan estaba mal pensado o el agente se ha perdido. Para todo, vuelve al plan, identifica qué quedó mal explicado, ajusta y empieza de nuevo (o rebobinas con `git reset --hard HEAD`). No intentes arreglar a parches lo que es fallo estructural.

Para esta sesión asumimos escenario 1 o 2. El dominio del caso no es tan complejo como para irse a escenario 3 si la spec es decente.

Cuando los tests pasan, pídele al agente un resumen final:

Lista los archivos creados, su tamaño aproximado en líneas, y resume qué test cubre cada criterio de aceptación de la spec.

Ese resumen lo usas en el handoff que vas a escribir al cerrar la sesión.

Bloque 5 · Paso 5, revisión final y commit

Tres comprobaciones rápidas antes de cerrar:

Compila la solution entera. No solo el proyecto de dominio. Si los demás proyectos ya existen como esqueletos, comprueba que añadir el dominio no rompe sus referencias.

```
dotnet build
```

Todos los tests pasan. Confirmación última.

```
dotnet test
```

Lectura rápida del código. No exhaustiva, pero al menos lee `FacturaService.cs` y un par de tests. Confirma que el código se siente .NET idiomático, que las descripciones de los tests son claras, que los nombres siguen las convenciones del proyecto. Si encuentras algo que merece pequeño ajuste, pídeselo al agente; si es ajuste mayor, marca como TODO para la sesión siguiente.

Commit cuando estés conforme:

```
git add src/Facturacion.Domain tests/Facturacion.Domain.Tests specs/  
git commit -m "feat(domain): registrar factura siguiendo specs/registrar-factura.md  
  
- Entidades Factura, Cliente, LineaFactura  
- Enums EstadoFactura, TipoCliente  
- FacturaService con CalcularTotal y RegistrarFactura  
- 12 tests xUnit cubriendo los 6 criterios de aceptación  
- Reutiliza CalcularFechaVencimiento del dominio existente  
  
Closes el primer paso del _plan-construccion.md."
```

El commit referencia la spec explícitamente. El hook de spec-sync del M4.2 lo pillaría si no actualizaste alguna spec — en este caso no había nada que actualizar en specs, así que pasa limpio.

Bloque 6 · Paso 6, handoff de la sesión

Antes de cerrar, escribe el handoff que la siguiente sesión va a leer. No es opcional: sin handoff, abrir mañana significa reconstruir el contexto.

```
# Handoff sesión 2 – implementación del dominio

## Trabajo cerrado
- Dominio completo en src/Facturacion.Domain.
- 12 tests xUnit verdes en src/Facturacion.Domain.Tests.
- Spec registrar-factura.md cumplida (6/6 criterios verificados).

## Decisiones tomadas que no estaban explícitas en la spec
- Tipo cliente lowercase ("regular", "vip"). La API decide en su sesión si normalizar a este formato o aceptar otros.
- Excepciones de dominio bajan de DominioException, no de Exception.
- Número de factura generado por dominio con ICounter inyectado; la implementación real (con persistencia atómica) viene en Infrastructure.

## Próximo paso
Sesión 3 (M6.4 será cambio de requisito, pero antes hace falta la sesión de Infrastructure en flujo real): implementar Facturacion.Infrastructure con EF Core y SQLite local. La spec es la misma registrar-factura.md porque el endpoint lo cubre todo; el repositorio es derivado.

## Prompt de retomada
Retomamos SDDCC.Facturacion sesión 3 – Infrastructure.
Handoff: _handoff-sesion-2.md.
Spec activa: specs/registrar-factura.md.
Estado: dominio cerrado y testeado; falta persistencia.
Plan Mode obligatorio. Solo tocar src/Facturacion.Infrastructure y sus tests. Reutilizar el DbContext si ya existe esqueleto.
```

Commit del handoff:

```
git add _handoff-sesion-2.md
git commit -m "docs: handoff sesión 2 (dominio cerrado)"
```

Bloque 7 · Las decisiones del dev en esta sesión

Mira atrás. En cada paso hubo decisiones tuyas, ninguna las tomó el agente solo:

1. **Cómo formulaste el prompt inicial** (alcance acotado, pre-condiciones recordadas).
2. **Cuándo dijiste "adelante"** al plan (la firma).
3. **El refuerzo sobre init-only** que pediste antes de firmar.
4. **Cómo respondiste a los hooks** que se activaron durante la ejecución.
5. **Cuándo paraste al agente** (si lo hiciste) durante la supervisión.
6. **Cómo verificaste que los tests cubren los criterios** uno a uno.
7. **El mensaje del commit** referenciando la spec.

8. **El handoff** y el prompt de retomada que dejaste para la siguiente sesión.

Esas ocho decisiones son SDD aplicado. Sin ellas, lo que sale es código generado a ciegas que parecía bien pero no tiene la disciplina que el método garantiza.

Bloque 8 · Cierre

Tres ideas para llevarte de esta sesión 2.

Plan Mode es donde se gana la sesión. Tres minutos de leer el plan y firmarlo con conciencia te ahorran media hora de "esto no era lo que quería" al final. Es la diferencia entre supervisar y limpiar.

Los hooks activos hacen tu trabajo más tranquilo. Mientras el agente ejecuta, los hooks atrapan lo que tu atención podría dejar pasar. No los pierdes de vista — si uno bloquea, prestas atención —, pero no los miras todo el rato.

El handoff es lo que une las sesiones. Treinta segundos al final de escribir el handoff te ahorran cinco minutos de reconstruir contexto al empezar la siguiente. Esa asimetría es por qué SDD escala bien a proyectos de semanas.

Glosario rápido del bloque

- **Refuerzo en Plan Mode:** instrucción extra que añades antes de firmar el plan para reforzar una decisión que el agente ya mencionó. Asegura que se materializa en el código.
- **Escenario de tests:** cómo van los tests al primer intento. Tres posibles (todos pasan, alguno falla y el agente lo arregla, fallo estructural que pide rebobinar).
- **Handoff entre sesiones:** documento que conecta la sesión que cierras con la que abrirás después. Diferente del handoff entre chats (que vimos en M3.5); aquí es entre dos sesiones del mismo dev sobre el mismo proyecto.

Qué viene en M6.4

Sesión 3 del caso: el cambio de requisito. Después de tener la implementación, el cliente pide descuentos por pronto pago. Veremos cómo se actualiza la spec, qué cambia en el código (poco), qué tests se añaden, y por qué SDD hace barato lo que en vibe coding sería caro. La pieza que justifica todo el esfuerzo del método.

M6.4 · Sesión 3 — el cambio de requisito

Bloque 0 · Lo que vamos a hacer

En la sesión 2 (M6.3) cerramos el dominio. En este punto del recorrido normal hubo otras sesiones intermedias para Infraestructure y API que el curso resume — no las desarrollamos en vídeo porque siguen el mismo patrón. Asumimos que el sistema está completo: dominio, persistencia, los cuatro endpoints, cliente CLI. Tests verdes.

Y entonces llega lo que llega siempre. El cliente cambia de idea.

Lo que cubrimos en esta sesión:

1. Recibir el cambio y traducirlo a algo accionable.
2. Actualizar la spec antes de tocar código.
3. Revisar la spec actualizada.
4. Implementar el cambio guiado por la spec nueva.
5. Reflexión sobre lo que habría costado lo mismo sin SDD.

Al terminar, el sistema cumple el nuevo requisito y tienes la spec actualizada en `specs/` reflejando exactamente lo que el sistema ahora hace. Y vas a ver por qué el método aguanta el peso del cambio.

Bloque 1 · La llamada del cliente

La llamada llega un martes por la mañana. Contabilidad ha hablado con dirección y aparece esto:

"Queremos incentivar el pronto pago. Si un cliente paga la factura dentro de los primeros diez días desde la emisión, le aplicamos un descuento del dos por ciento sobre el total. El descuento se calcula al marcar la factura como pagada — si paga después del día 10, no hay descuento. Necesitamos también que el endpoint de consulta refleje el importe efectivamente cobrado, no solo el total original."

Léelo dos veces. Es un cambio aparentemente pequeño con consecuencias en varias capas. Antes de tocar nada, descomponlo en lo que toca:

- **Comportamiento nuevo en `marcar-pagada`**: si la fecha de cobro es $\leq \text{fechaEmision} + 10$ días, aplicar 2% sobre `totalCentimos`. El importe efectivamente cobrado es `total -`

descuento .

- **Campo nuevo en la entidad Factura** : `montoCobradoCentimos` . Lo rellena `marcar-pagada` ; antes era implícitamente igual al total.
- **Cambio en el contract de consultar-factura** : la respuesta incluye `montoCobradoCentimos` y `descuentoAplicadoCentimos` .
- **Persistencia**: nueva migración para añadir las dos columnas.
- **Cliente CLI**: el comando de consulta debería imprimir el nuevo campo si la factura está pagada.

Cinco impactos. En un proyecto sin spec, ya estarías tocando código y descubriendo el sexto a las tres horas. En SDD, primero spec.

Bloque 2 · Paso 1, actualizar la spec antes del código

La regla del M1.3 vuelve aquí con fuerza: **la spec cambia primero**. Si tu reflejo es abrir el código y empezar a teclear el descuento, paras y respiras.

Identificas qué specs se ven afectadas:

- `specs/marcar-pagada.md` — la operación que aplica el descuento.
- `specs/consultar-factura.md` — la respuesta cambia.
- `specs/registrar-factura.md` — no cambia, el cálculo del total inicial sigue igual.
- `specs/listar-facturas.md` — depende de si el listado debería mostrar también el descuento; decisión a tomar.

Abres `specs/marcar-pagada.md` y editas. Ejemplo de cambios:

Intent

[Sin cambios respecto a la versión anterior.]

(Añadir párrafo) Desde la decisión del 28 de mayo, marcar una factura como pagada con fecha de cobro dentro de los primeros 10 días naturales desde la emisión activa un descuento del 2% sobre el total. Eso requiere registrar el importe efectivamente cobrado, distinto del total original.

Behavior

Caso normal

- Endpoint: `POST /facturas/{id}/pagos` con body { fechaCobro }.
- Validaciones: fechaCobro no puede ser anterior a fechaEmision.
- Cálculo del descuento:
 - Si $\text{fechaCobro} \leq \text{fechaEmision} + 10$ días naturales: $\text{descuento} = \text{round}(\text{total} * 0.02)$.
 - Si no: $\text{descuento} = 0$.
- Cálculo del monto cobrado: $\text{montoCobrado} = \text{total} - \text{descuento}$.
- Cambio de estado: Pendiente → Pagada.
- Respuesta 200: { id, totalCentimos, descuentoAplicadoCentimos, montoCobradoCentimos, fechaCobro, estado }.

Casos de error

- 404 si la factura no existe.
- 409 si la factura no está en estado Pendiente (ya pagada, anulada, vencida solo si decidisteis bloquear). [TODO: confirmar con contabilidad si una factura vencida puede pagarse igual con descuento – provisionalmente asumimos que sí, solo Anulada bloquea].
- 400 si $\text{fechaCobro} < \text{fechaEmision}$ con mensaje claro.

Contract

```
```csharp
public record MarcarPagadaRequest(DateTimeOffset FechaCobro);

public record FacturaPagadaResponse(
 long Id,
 long TotalCentimos,
 long DescuentoAplicadoCentimos,
 long MontoCobradoCentimos,
 DateTimeOffset FechaCobro,
 EstadoFactura Estado);
```

## Criterios de aceptación

- [ ] Pago dentro del día 10 desde emisión aplica 2% descuento.
- [ ] Pago en el día 11 o después NO aplica descuento.
- [ ] El descuento se calcula con `round(total * 0.02)` (regla bancaria, no truncado). En caso de empate de céntimo, redondeo a número par.
- [ ] El monto cobrado coincide con total - descuento en todos los casos.
- [ ] Marcar pagada dos veces la misma factura sigue siendo rechazado con 409 (no cambia).



- [ ] La respuesta incluye los tres importes (total, descuento, cobrado).
- [ ] Tests xUnit cubren los casos: pago el día 5 con descuento, pago el día 10 con descuento (límite), pago el día 11 sin descuento.

## **Fuera de alcance**

- No emite nota de crédito por el descuento (otro flujo).
- No notifica al cliente del descuento aplicado.
- No retroactivo: facturas ya pagadas antes de este cambio se quedan tal cual.

## **Notas y decisiones**

- Decisión de redondeo: bancario (par). Discutido con contabilidad el 28 mayo; motivo: alineación con el resto del sistema financiero del cliente.
- 10 días naturales, no laborales. Confirmado.
- El descuento se aplica solo si el cliente lo solicita... ¿o automáticamente? Pregunta abierta. Por ahora asumimos AUTOMÁTICO al marcar pagada en plazo. [TODO: confirmar antes de poner en producción.]

Actualizas también `specs/consultar-factura.md` para reflejar que la respuesta incluye los nuevos campos cuando la factura está pagada.

Cinco minutos de edición real. Spec actualizada con todas las decisiones a la vista – incluyendo las pendientes marcadas como TODO.

---

## Bloque 3 · Paso 2, revisar la spec actualizada

Antes de implementar, pásala por la revisión del M2.4. Skill `revisar-spec-sdd` sobre las dos specs modificadas.

Hallazgos típicos que aparecen en el cambio:

**\*\*En `marcar-pagada.md`:\*\***

- Ambigüedad: "10 días naturales" – ¿desde fechaEmision incluida o excluida? Si emisión es día 1, ¿el día 10 es ese mismo día 1 + 9 días, o + 10? Decides incluir extremos (factura emitida el día 1 paga con descuento si fechaCobro es  $\leq$  día 11).
- Alcance abierto: qué pasa si fechaCobro es exactamente fechaEmision (mismo día). ¿Cuenta como dentro? Decides que sí.
- Criterio incompleto: el caso de empate del redondeo bancario merece un test específico. Lo añades.

**\*\*En `consultar-factura.md`:\*\***

- Contradicción potencial: la respuesta antes era una estructura; ahora es otra. ¿Versionado del endpoint? Para v1 sin clientes externos, no. Marcas decisión.

Cierres los huecos editando las specs. Commit:

```
```bash
git add specs/marcar-pagada.md specs/consultar-factura.md
git commit -m "feat(specs): descuento por pronto pago en marcar-pagada"
```

Ya estás listo para tocar código.

Bloque 4 · Paso 3, implementar el cambio guiado

Abre Claude Code en Plan Mode. Prompt:

Implementa el cambio descrito en la actualización reciente de specs/marcar-pagada.md y specs/consultar-factura.md. Resumen: descuento por pronto pago (2% si $\text{fechaCobro} \leq \text{fechaEmision} + 10$ días), nuevos campos en Factura y en la respuesta del endpoint.

Tocas:

- Dominio: añadir lógica de cálculo de descuento en FacturaService.
- Dominio: nuevos campos en Factura.
- Infraestructure: migración para añadir las dos columnas.
- API: actualizar el endpoint marcar-pagada y la respuesta de consultar.
- Tests: cubrir los criterios nuevos.

No tocar CLI todavía; eso lo haremos después.

Antes de ejecutar, plan.

El agente devuelve plan. Aplica las cuatro comprobaciones del M3.2.

Alcance. Domain + Infrastructure + Api + tests. Encaja con lo pedido. CLI se queda fuera, como pediste.

Decisiones. Probablemente el agente proponga: - Migración EF Core con `dotnet ef migrations add AddDescuentoFields`. - Modificación de `MarkAsPaid` en `FacturaService` para recibir `fechaCobro` y calcular descuento. - Nuevo método helper `CalcularDescuentoProntoPago` en `FacturaService`. - Actualización del DTO de respuesta en API. - Tests nuevos en `Domain.Tests` para el helper y la nueva regla de `MarkAsPaid`.

Secuencia. Razonable: dominio primero, migración después, API actualizada, tests al final, ejecutar.

Riesgos. El agente puede señalar el tema del redondeo bancario, que en .NET es `Math.Round(value, MidpointRounding.ToEven)` por defecto pero merece ser explícito en el código para no depender del default.

Firmas el plan. Ejecuta.

Durante la ejecución, los hooks intervienen como en M6.3. El de tests-post-edición ejecuta solo tests del proyecto modificado en cada paso. Mientras está añadiendo el helper, tests anteriores deberían seguir verdes; los tests nuevos del helper pasan cuando se implementa. La migración se aplica, se ejecutan los tests de integración (si los tienes) y la API se actualiza.

El cambio total en código suele ser:

- **Domain:** ~40 líneas añadidas (helper + cambio en `MarkAsPaid` + campos).
- **Infrastructure:** ~30 líneas (migración).
- **API:** ~20 líneas (DTO y mapeo).
- **Tests:** ~80 líneas (8 tests nuevos cubriendo casos límite).

Unas 170 líneas modificadas o añadidas para cubrir el cambio entero, en una sesión. Compila, todos los tests verdes incluidos los nuevos. Commit:

```
git add src/Facturacion.Domain src/Facturacion.Infrastructure src/Facturacion.Api tests/
git commit -m "feat: descuento por pronto pago (cierra cambio del 28 mayo)"
```

- FacturaService.CalcularDescuentoProntoPago (helper)
- MarkAsPaid acepta fechaCobro y aplica descuento si dentro de plazo
- Campos MontoCobradoCentimos y DescuentoAplicadoCentimos en Factura
- Migración AddDescuentoFields
- DTO de respuesta de marcar-pagada y consultar actualizado
- 8 tests xUnit nuevos

Specs actualizadas: marcar-pagada.md, consultar-factura.md.
TODOs pendientes anotados en specs/marcar-pagada.md (vencidas, automático)."

Bloque 5 · Reflexión — qué habría pasado sin SDD

Para que se vea el valor del método, imagina la misma llamada del cliente sobre un proyecto en vibe coding (mismo dominio, mismo equipo, sin specs).

Escenario sin spec, primera media hora. Abres el código de `MarkAsPaid`. Lees lo que hace. Intentas recordar por qué tomaste algunas decisiones hace dos semanas. Buscas dónde se calcula el monto cobrado — no existe, está implícito en el total. Empiezas a teclear pero no estás seguro de si el redondeo debe ser hacia arriba, hacia abajo o bancario. Pruebas algo y sigues.

Hora dos. El cliente CLI revienta porque su DTO no esperaba los campos nuevos. Ajustas el CLI. Mientras lo haces te das cuenta de que `consultar-factura` también devuelve el total — ¿debería devolver el cobrado en su lugar? Improvisas. La consistencia entre endpoints empieza a hacer ruido.

Hora tres. Migración EF Core. Sin spec del modelo de datos, las nuevas columnas terminan con tipos no del todo coherentes con el resto. Ya no recuerdas si lo hiciste con `decimal` o `long`. Inconsistencia en el .NET típico que se paga después.

Día siguiente. Llamas a contabilidad porque no sabes si el descuento se aplica también a facturas ya vencidas o solo a las pendientes. Te dicen lo que probablemente — no estaban claros tampoco. Implementas una decisión más. Otra cosa que la spec habría hecho explícita el primer día.

Una semana después. Reclamación de un cliente: la factura se le aplicó descuento pero el cobro fue el día 11 — el sistema usaba `Date.now()` en lugar de la fecha enviada por contabilidad. Bug por imprecisión que la spec habría capturado en los criterios de aceptación.

Total: probablemente cuatro o cinco días reales de trabajo y un bug en producción. Versus media mañana en SDD.

La asimetría no es la velocidad del cambio en sí — es que SDD captura **todas las decisiones que el cambio implica antes de tocar código**. Sin spec, esas decisiones se toman improvisando, y cada decisión improvisada es deuda futura.

Bloque 6 · Cierre

Tres ideas para llevarte de la sesión 3.

El cambio de requisito es donde SDD se gana o se pierde el sueldo. Si el método solo se justifica en proyectos nuevos, no escala. Si se justifica también en mantenimiento, escala perfectamente — y los proyectos viven mucho más en mantenimiento que en construcción inicial.

La regla "primero la spec" es contraintuitiva en el cambio pero crítica. Cuando llega un requisito nuevo, el reflejo es abrir el código. SDD obliga a invertir el orden: primero la spec, después el código. Esa inversión filtra decisiones implícitas que el código habría tomado mal.

Los TODO en la spec son tan importantes como lo escrito. Las preguntas abiertas que dejas marcadas (en este caso: "facturas vencidas con descuento sí/no", "descuento automático o a petición") son decisiones pendientes. Marcarlas como TODO en la propia spec las hace visibles para quien la lea más adelante y evita que el sistema viva con la respuesta improvisada de quien tocó el código primero.

Glosario rápido del bloque

- **Cambio de requisito clásico:** llamada del cliente o del PM que modifica el comportamiento de una pieza ya construida. En cualquier proyecto serio llega más de una vez.
- **Inversión del orden** (spec → código): la disciplina de modificar la spec antes que el código cuando llega un cambio. Es contraintuitiva pero es lo que filtra decisiones implícitas.
- **TODO en spec:** marca explícita de pregunta abierta que requiere input externo para cerrarse. Vivirá en la spec hasta que se decida y se actualice.
- **Bug por imprecisión:** error en producción que la spec habría capturado en criterios de aceptación. Síntoma típico de proyectos sin spec o con spec débil.

Qué viene en M6.5

Cierre del curso. Cómo empezar a aplicar SDD en tu propio trabajo a partir de mañana. Errores frecuentes en los primeros meses (forzar la plantilla en piezas pequeñas, escribir specs a posteriori, abandonar Plan Mode tras dos semanas, hooks que se quedan sin owner). Material de seguimiento (Spec Kit oficial, comunidad, documentación de referencia para que envejezcas bien con el ecosistema). Y la idea-eje final del curso, esa frase que te llevas grabada y que decide qué tipo de dev SDD vas a ser.

M6.5 · Cierre — tu próximo proyecto

Bloque 0 · Lo que vamos a hacer

Acabamos. Llegaste hasta aquí; eso ya es decisión. Lo que falta no es más material — falta dejarte el método operativo en la cabeza para mañana en tu trabajo real.

Tres preguntas que cierro en este submódulo:

1. ¿Por dónde empiezo a aplicar SDD en lo que ya tengo entre manos?
2. ¿Qué errores voy a cometer en los primeros dos o tres meses y cómo evitarlos?
3. ¿Cómo sigo cuando el ecosistema cambie (que cambiará)?

Y al final una idea-eje que vale más que el resto del cierre.

Bloque 1 · Tres caminos para empezar mañana

No tienes que reescribir tu proyecto. SDD se introduce gradualmente, pieza por pieza. Tres caminos según tu situación:

Camino A — proyecto nuevo. El más fácil. Estás arrancando algo desde cero. Crea la carpeta `specs/` desde el primer día, escribe la primera spec antes de tocar código, configura Plan Mode como default, instala los tres hooks defensivos del M4.5. En dos semanas tienes el método integrado en el ciclo normal del equipo y nadie recuerda cómo se trabajaba antes.

Camino B — feature nueva en proyecto existente. Lo más realista. Tu proyecto vive desde hace meses o años. No vas a aplicar SDD al código viejo de golpe. Lo que sí puedes hacer: el próximo ticket que entre, antes de tocar código, escribe la mini-spec con plantilla A. Aplica el ciclo solo a esa feature. Cuando llegue el siguiente ticket, lo mismo. En tres meses tienes una cultura instalada feature a feature, sin refactorización masiva.

Camino C — bugfix con mini-spec. El más liviano. Antes de tocar un bug, escribe en el propio PR los cuatro campos de la plantilla B del M2.3: síntoma, reproducción, causa raíz hipótesis, criterio de "arreglado". Lo haces en cinco minutos. Esa mini-spec se queda en el PR como histórico. Si lo repites con tres o cuatro bugs seguidos, el equipo empieza a pedir lo mismo en bugs ajenos sin que tú lo propongas.

Los tres caminos funcionan. El más rentable depende de tu contexto. Si dudas, ve por el camino B — es el que más impacta en el día a día sin pedir cambio estructural del equipo.

Lo que no funciona: anunciar "ahora vamos a hacer SDD en todo el proyecto" como decisión de arriba. Los métodos de proceso sobreviven cuando se introducen por la base y demuestran valor; no cuando se imponen por la cúpula y generan resistencia.

Bloque 2 · Cinco errores típicos en los primeros meses

Vas a cometer errores. No es opinión; es lo que veo siempre cuando alguien adopta el método. Cinco patrones recurrentes, los más probables:

1. Forzar la plantilla en piezas pequeñas. Escribes una spec completa con intent + behavior + contract para una función de quince líneas. Al hacerlo, el método se siente burocrático y empiezas a abandonarlo. La regla del M1.3 vale: si la pieza cabe en treinta minutos y no expone interfaz, probablemente no necesita spec. Si tienes dudas, escríbela; pero si ya cazaste que sobra, no te la fuerces.

2. Escribir specs a posteriori. Acabas la pieza y "para dejar constancia" escribes la spec después. Eso es documentación, no SDD. La spec describe entonces lo que el código hace, no lo que tendría que haber hecho. Has perdido el momento en que las decisiones eran reversibles. Cuando te encuentres haciéndolo, para: o bien la próxima feature se hace con spec a priori, o decides explícitamente que esta pieza no llevaba spec y dejas de fingir.

3. Abandonar Plan Mode tras dos semanas. Plan Mode añade tres minutos al ciclo. En los días intensos esos tres minutos parecen mucho. Empiezas a saltártelo "para esta vez", y luego para la siguiente, y luego para la otra. Cuando te das cuenta, llevas tres semanas sin firmar un plan. Reaplicáte: la disciplina se mantiene día a día, no de una vez.

4. Hooks que se quedan sin owner. Instalaste los hooks defensivos del M4.5 entusiasmado. A los dos meses, uno bloquea algo legítimo y nadie sabe arreglarlo. La gente desactiva el hook. Cuando otro hook tropieza, lo mismo. Al cuarto mes no tienes hooks activos. La defensa: cada hook con owner declarado desde el día uno, y revisión trimestral aunque no haya pasado nada.

5. Pensar que SDD es solo escribir specs. El error más sutil. Crees que el método se reduce a abrir `.md` y rellenar campos. Olvidas Plan Mode, olvidas la revisión, olvidas los hooks, olvidas el handoff entre sesiones. Las specs sin el resto del método son documentos bonitos que el código va a dejar atrás. SDD es **spec + Plan Mode + hooks + handoff + verificación**. Sin las cinco piezas, el método no aguanta.

Si te ves en uno de estos cinco, no te desesperes. Todos los devs serios que conozco aplicando SDD los han cometido al menos una vez. Lo importante es nombrar el error cuando aparezca y volver al método.

Bloque 3 · Cómo seguir aprendiendo cuando el ecosistema cambie

El ecosistema en torno a Claude Code y SDD cambia cada pocos meses. Lo que hoy se llama de una manera puede llamarse distinto dentro de un año. Versiones de Spec Kit, formato de hooks, sintaxis MCP — todo eso va a evolucionar.

Tres pilares que mantienen el curso útil aunque el ecosistema mute:

El método no cambia. Spec primero, plan antes de ejecutar, validación contra spec, handoff entre sesiones, contract donde hay interfaz expuesta. Eso lleva décadas siendo cierto en distintos disfraces (ingeniería tradicional, agile bien hecho, BDD, DDD ligero). La IA solo lo hace más barato y más rentable. Si el método quedó interiorizado, las herramientas concretas las cambias sin perder pie.

El ecosistema oficial. Documentación oficial de Claude Code, repositorio de Spec Kit, blogs de Anthropic. Esa es la fuente primaria. Cuando algo cambie, ahí saldrá primero, sin ruido de hilos de redes sociales. Súscríbete a uno o dos canales y olvida los demás.

La comunidad práctica. Hay un grupo creciente de gente aplicando SDD con Claude Code en proyectos reales. Conferencias técnicas de .NET (NDC, .NET Conf), encuentros locales, comunidades online. La pregunta correcta a buscar no es "qué dice la moda" sino "qué están haciendo equipos que llevan meses con esto en producción". Esa información tarda más en aparecer pero es la útil.

Tres pilares que aguantan. Si los mantienes activos, vas a envejecer bien con el método aunque el ecosistema concreto cambie de nombre cinco veces.

Bloque 4 · La idea-eje final del curso

Si te llevas una sola idea de las quince horas largas que ha durado este curso, que sea esta. La pongo limpia, sin envolver:

El código que sobrevive es el código que alguien fue capaz de escribir como spec antes de escribirlo como código.

Léela despacio.

Hay código que sobrevive y código que no. Sobrevive el que se puede mantener cuando llega el cambio de requisito. Sobrevive el que el equipo nuevo entiende. Sobrevive el que la persona de hace dos años — tú mismo — todavía puede leer hoy y saber por qué hace lo que hace.

¿Qué tienen en común esos códigos que sobreviven? Que la intención está escrita en algún sitio. Sea una spec formal, sean comentarios densos en los puntos clave, sea un README de arquitectura, sea un ADR. Hay algo escrito que el lector puede consultar para reconstruir el porqué.

El código que muere es el que vive en la cabeza de quien lo escribió. Que era todo el contexto. Que pierde el sentido cuando la cabeza se va, o cuando la cabeza misma olvida.

SDD invierte el orden natural del trabajo. Primero la intención, después la implementación. No porque el orden sea más eficiente al inicio (no lo es siempre), sino porque produce código que sobrevive. Y el coste de producir código que muere y hay que reescribir es alto — mucho más alto que los cinco minutos extra de escribir la spec primero.

La frase está pensada para que la lleves grabada. Cuando estés en mitad de algo y dudes si parar a escribir la spec o "darle al enter ya", recuérdala. El código que sobrevive es el que alguien fue capaz de escribir como spec.

Bloque 5 · Lo que se llevan los que apliquen el método

Si aplicas el método con disciplina durante los próximos tres meses, vas a notar cosas concretas:

Menos re-trabajo en code review. Los PRs salen alineados con la intención del cambio porque la spec lo dejó claro antes. El comentario clásico "no era esto lo que pedíamos" desaparece.

Onboarding más rápido. Quien entra nuevo lee specs en lugar de descubrir intenciones leyendo código. La curva de productividad baja un escalón.

Cambios de requisito que duelen menos. El de la sesión 3 que vimos en M6.4 sale en una mañana, no en una semana. La asimetría se nota especialmente cuando el equipo crece y cuando el cliente cambia de idea más de lo esperado.

Confianza para colaborar con la IA en piezas reales. No solo para juguetes y prototipos. Para código que va a producción y tiene que vivir años. El método te da los puntos de control que hacen que confíes en lo que el agente entrega.

No son promesas vagas. Son consecuencias mecánicas del método cuando se aplica.

Lo que no te garantiza SDD: convertirte en mejor dev por ósmosis. El método es un acelerador del criterio que ya tienes. Si tu criterio era bueno, vas a hacer cosas mejores y más rápido. Si tu criterio necesita afinarse, sigue afinándose por su cuenta — leyendo código bueno, viendo proyectos reales fallar y aprender de ellos. SDD no sustituye la cultura de ingeniería; la potencia.

Bloque 6 · Despedida

Llegaste hasta aquí. Lo digo en serio.

Treinta submódulos. Veinte horas largas de contenido. Una práctica de spec del M2.5, una de implementación del M3.6, una de hooks del M4.5, una de MCP del M5.5, y un caso end-to-end completo del M6. Si hiciste las prácticas, no solo has visto el método; lo has aplicado. Esa diferencia es la que separa al que entiende SDD del que lo sabe.

Algunas cosas que te van a venir bien tener cerca a partir de aquí:

- La carpeta `specs/` de tu próximo proyecto. Empieza por una sola spec. La primera siempre cuesta más.
- El `CLAUDE.md` raíz del proyecto. Cuatro líneas básicas el primer día; lo amplías conforme aparecen reglas.
- El kit de tres hooks defensivos del M4.5. Cópialos a tu repo y ajústalos a tu solution.
- La frase del bloque 4. Llévela donde quieras: como comentario en `CLAUDE.md`, como cita pegada en tu monitor, como recordatorio mental.

Gracias por dedicarle tiempo al curso. Cuando hagas tu primer proyecto serio con SDD en .NET y veas el cambio de requisito que se absorbe en una sesión, te acordarás de la sesión 3 y vas a sonreír un poco. Eso es por lo que estaba escrito todo esto.

Buena suerte con lo que sigue.

Glosario rápido del bloque

- **Camino A / B / C** (para empezar SDD): proyecto nuevo desde cero / feature nueva en proyecto existente / bugfix con mini-spec dentro del PR. Tres puntos de entrada según el contexto.
- **Adopción por la base** vs **imposición por la cúpula**: dos formas de introducir un método. La primera funciona; la segunda genera resistencia y dura poco.
- **Pilar de mantenimiento del método**: las tres cosas que aguantan cuando el ecosistema cambia (método inmutable, fuentes oficiales, comunidad práctica).
- **Código que sobrevive**: el que tiene la intención escrita en algún sitio recuperable. La idea-eje final del curso.

Estado al cerrar el curso

Has visto:

- M1: por qué SDD existe ahora (vibe coding deja de escalar; spec como artefacto de primera clase).
- M2: cómo se escribe una spec útil (tres niveles, formatos, plantillas, revisión, práctica).
- M3: Claude Code como motor (modelo mental, Plan Mode, skills, subagentes, sesión, práctica integrada).
- M4: red de seguridad automática (hooks de validación y de protección, hooks como skill de equipo, práctica).
- M5: puente al mundo real con MCP (qué es, anatomía, catálogo útil, diseñar uno propio, práctica de conexión).
- M6: caso end-to-end completo (presentación, sesión de especificación, sesión de implementación, cambio de requisito, cierre).

El método entero ya está cubierto. Lo que falta es aplicarlo.

Fin del curso

"El código que sobrevive es el código que alguien fue capaz de escribir como spec antes de escribirlo como código."

Hasta el próximo proyecto.